BLEND: A Unified Data Discovery System

1st Mahdi Esmailoghli
TU Berlin & BIFOLD
Berlin, Germany
esmailoghli@tu-berlin.de

2nd Christoph Schnell
Leibniz Universität Hannover
Hannover, Germany
schnell@dbs.uni-hannover.de

3rd Renée J. Miller
University of Waterloo
Waterloo, Canada
rjmiller@uwaterloo.ca

4th Ziawasch Abedjan
TU Berlin & BIFOLD
Berlin, Germany
abedjan@tu-berlin.de

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00061

***Abstract*—Most research on data discovery has so far focused on improving individual discovery operators such as join, correlation, or union discovery. However, in practice, a combination of these techniques and their corresponding indexes may be necessary to support arbitrary discovery tasks. We propose BLEND, a comprehensive data discovery system that supports existing operators and enables their flexible pipelining. BLEND is based on a set of lower-level operators that serve as fundamental building blocks for more complex and sophisticated user tasks. To reduce the execution runtime of discovery pipelines, we propose a unified index structure and a rule- and cost-based optimizer that rewrites SQL statements into low-level operators when possible. We show the superior flexibility and efficiency of our system compared to ad-hoc discovery pipelines and stand-alone solutions.**

I. Introduction

The potential for large structured data lakes, such as governmental data lakes [19], [61], enterprise-level corpora [10], [15], and public-access lakes [22], [14], [33], has spurred various avenues of research, all aimed at empowering data scientists with the ability to extract further insights for their diverse tasks, such as machine learning (ML) [23], exploratory data analysis [41], and data cleaning [40]. To support these tasks, various data discovery operatorion have been studied, such as **join discovery** [24], [69], [63], [62], [59], [27], [70], [20], [16], [56], [50], **union search** [45], [11], [36], [13], [25], [34], and **correlation discovery** [23], [49], [9], [17].

Existing research has focused on each operator individually without considering possible opportunities for combining, pipelining, and optimizing them. Yet, fulfilling a real-world data discovery task often requires a combined application of these operators. For instance, in the case of data enrichment for ML, the user needs to use a combination of join, union, and correlation discovery [9], [45]. To achieve this, we need a unified data discovery system.

Requirements for a unified discovery system. A trivial approach to building such a unified system is to federate all indexes and algorithms via a common interface. Even after overcoming the inherent software and interface heterogeneity among prototypes, the execution of complex data discovery tasks that combines these operators across systems is inefficient and inflexible in terms of both storage and maintenance. This stems from several key challenges:

Lack of Flexibility. Introducing new criteria or discovery operators is challenging since new indexes and algorithms must be integrated into an already complex system.

Storage and Maintenance Overhead. Multiple independently developed index structures must be generated and stored. The aggregated system will comprise an incoherent set of potentially redundant structures needing maintenance.

Lack of Optimization. The federation of independent operators under the same abstraction does not lend itself to optimization. To showcase the operational complexity of a rather simple discovery task, consider following example:

Example 1. Fig. 1 illustrates a user table S along with three tables $T1$, $T2$, and $T3$ from a data lake. S contains information about departments in a company and the name of their corresponding leaders, i.e.,"Head". The user wants to fill in missing values of S but already knows that the head of the IT department, "Tom Riddle", has left the company. Therefore, any table that contains the projection ("IT","Tom Riddle") is outdated. Thus, the user defines the data discovery for missing values as follows: *Find top-*1 *table that contains ("HR" and "Firenze" in a row and "HR", "Marketing", "Finance", "IT", ... in a column but does not contain rows with "IT" and "Tom Riddle").*

To solve this task with existing discovery operators, one could use single-column join discovery, such as Josie [69], to look up overlapping column values, and multi-column join discovery, such as MATE [24], to find overlapping column combination values. With MATE, one could find tables containing the projection ("HR", "Firenze") as result set rs_1. Also, MATE can be used to identify tables containing the tuple ("IT", "Tom Riddle") as result set rs_2. Josie can be employed to find tables that join only on the department values "HR", "Marketing", "Finance", ... as result set rs_3. Finally, the equation $((rs_1 \cap rs_3) - rs_2)$ is the answer to the user's query. According to Fig. 1, $rs_1 = \{T2, T3\}$, $rs_2 = \{T2\}$, and $rs_3 = \{T1, T2, T3\}$. Thus, the answer to the discovery query is $\{T3\}$, which contains amendable values for the "Head" of the department column in S. •

The example illustrates that running independent discovery redundant and inefficient. For each look-up, combinations of the three tables $T1$, $T2$, and $T3$ may be processed. However, knowing that $T1$ and $T2$ are not desired tables, we could exclude them from follow-up operators: first, by checking all three tables to find those containing ("HR" and "Firenze"); then, by focusing only on $T3$ and $T2$ and dropping the outdated $T2$; and finally, by verifying $T3$ for overlapping department names. Intuitively, we could obtain the same results by resorting to application code. We aim to

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00061

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:53 UTC from IEEE Xplore. Restrictions apply.



Fig. 1: Discovering up-to-date tables to fill in the NULLs.

achieve the same flexibility within a discovery system with a unified language to optimize custom discovery processes. Such abstraction becomes necessary in case of complex queries with a variety of operators and result sets with millions of tables.

Contributions We introduce BLEND, a unified system that enables users to create complex discovery plans. With no need for compiling, installing, or adapting additional discovery systems, BLEND streamlines the discovery process. Given a data discovery plan, a data lake, and a parameter k, BLEND retrieves the top-k tables from the lake that are most relevant to the defined plan. Our key contributions are as follows:

1) **Introduction of an abstraction layer:** We design a novel abstraction that unifies the building blocks of different discovery tasks. We propose a set of atomic search operators, which users can declaratively compose to efficiently discover relevant tables. These building-blocks are implemented using primitive database functions, allowing the majority of computations to run within the database.
2) **Optimized execution of tasks:** We develop a two-phase plan optimizer that combines rule- and cost-based query optimization techniques. It re-orders the execution of operators based on predefined rules, ensuring efficient execution while preserving the intended logic of the discovery task. Furthermore, BLEND optimizes individual operators by rewriting corresponding queries using intermediate results.
3) **Integrated index:** We studied fifty-three index structures from thirty data discovery papers and propose a unified index that efficiently supports the execution of the introduced operators and their compositions. The index is designed as a single database table, enabling the system to additionally benefit from the database-level query optimizations.
4) **Simplification of complex tasks using SQL operations:** Our proposed index structure allows for the translation of BLEND's operators into SQL queries. Even complex tasks, such as correlation discovery, are reduced to SQL-based operations, eliminating the need for custom algorithms or external implementations. By integrating multiple operators into declarative pipelines, BLEND minimizes user effort and simplifies the creation of discovery tasks.
5) **User study:** We conducted a user study with data experts to understand their discovery preferences and evaluate the effectiveness of BLEND in addressing these needs.

II. Problem Statement

Given the requirements formulated in the introduction, we formalize the following problem statement. Let $\mathcal{D} = \{T_1, T_2, \ldots, T_n\}$ be a data lake, where each T_i is a table. Given a query table S with insufficient data and a discovery plan P by the user, the goal of BLEND is to enable the formulation of the plan and its optimized compilation for execution. The result of the execution is the subset $\mathcal{T}' \subseteq \mathcal{D}$ that enriches S. With this functionality in mind, we strive to solve four technical problems (**Pr.1-4**):.

First, the language for defining a plan should support the formulation of the most prominent discovery operations from the literature (**Pr.1**):

- **Keyword search**: discovered tables must overlap with a given set of keywords.
- **Join discovery**: discovered tables must be joinable with S on one or more columns.
- **Union discovery**: rows in discovered tables must be unionable with rows in S.
- **Correlation discovery**: a column in each discovered table must correlate with the user-defined numerical target in S.

Second, the system should also support any set-based aggregation of these operation's results (**Pr.2**).

Third, we require an index that serves all of the possible discovery operations but is leaner than the ensemble of the required individual indexes. More formally, assuming that an independent system exists for each of the aforementioned discovery operations M_{op_i}, where $1 < i < n$, the storage required for our index to execute any combination of operators in OP must be smaller than $\sum_{i=1}^{n} \text{Storage}(M_{op_i})$, where $Storage(M_{op_i})$ is the storage size required for the state-of-the-art index in M_{op_i} (**Pr.3**).

Finally, the system should satisfy two runtime constraints. The implementation of each (M_{op_i}) in the unified system should yield a comparable runtime while any aggregation and combination of more than two M_{op_i} should yield superior runtime compared to their execution as concatenated black box discovery system. Formally, assuming a discovery plan P is an arbitrary combination of operators in OP, i.e, $P \in OP^*$, the execution runtime of P in BLEND should satisfy

$$\text{Time}(P_{\text{BLEND}}) \leq \sum_{j|op_j \in P} \text{Time}(M_{op_j}).$$

Satisfying the previous constraint should not be a burden on how the user formulates the plan. BLEND must identify a good (near optimal) ordering of the operations based on the expected runtime and intermediate result size (**Pr.4**)

III. System Overview

Fig. 2 depicts an overview of BLEND's architecture depicting its online and offline phases. The online phase involves four stages: task definition, plan formulation, plan optimization, and execution.

First, the user defines their data discovery task using BLEND's API (Fig. 2a). Here, the plan *find_dep_heads* is

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:53 UTC from IEEE Xplore. Restrictions apply.

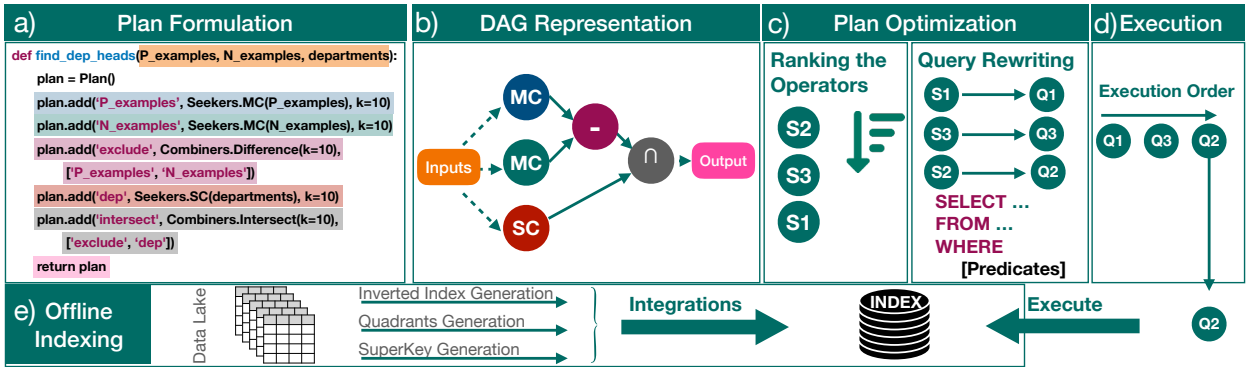


Fig. 2: BLEND's architecture.

written for the use case of Fig. 1. It finds tables containing a set of positive examples and the given department names but not negative examples. The main primitives in BLEND's API are *Plan*, *Seekers*, and *Combiners*. The latter two, *Seekers* and *Combiners*, are the lower-level operators in BLEND, which are implemented in SQL. A discovery task is created by instantiating a *Plan* object, to which one can add *Seekers* and *Combiners*. Seekers are added by specifying an identifier string and parameters, such as input lists and top-k. BLEND transforms the plan into a directed acyclic graph of operators, where edges represent the data flow (Fig. 2b). Once the discovery plan is written and the DAG is generated, BLEND leverages a combination of rule-based and cost-based optimization to re-rank and rewrite operators for faster execution (Fig. 2c). The low-level operators either resemble or contain SQL queries on BLEND's index which is organized as a fact table. Based on the cost model, BLEND rewrites the SQL statements by injecting predicate placeholders enabling the system to take advantage of cardinality estimates of the intermediate results. Once the queries are generated, they are sent to the execution engine to run in the database (Fig. 2d). In the offline phase, BLEND builds the necessary index and data structures (Fig. 2e). The index comprises and unifies three structures: inverted index [5], *super keys* [24], and quadrants [49].

IV. OPERATORS

BLEND's operators are designed to be composable, enabling complex plans, and simple for efficient execution. BLEND leverages two sets of operators: seekers and combiners.

A. Seeker operators

A seeker receives a set of columns, Q, as the input and returns the top-k most relevant tables. k is a user-defined parameter. We propose four seekers: *single-column*, *keyword*, *multi-column*, and *correlation* seekers.

1) Single-Column (SC) Seeker: The SC seeker receives an input column Q, typically from an input (query) table and returns a list of k tables. These tables contain a column that overlaps the most with Q (sorted by number of overlaps).

2) Keyword (KW) Seeker: The KW seeker returns the top-k overlapping tables based on a given set of keywords. The main difference between the KW and SC seekers is that the KW seeker measures the overlap throughout entire candidate tables rather than single columns. Returned tables are sorted based on the number of overlaps.

3) Multi-Column (MC) Seeker: The MC seeker receives Q that has multiple columns and discovers the top-k tables in the data lake that the most with the tuples in Q. The returned tables are sorted based on the overlap.

4) Correlation (C) Seeker: This seeker receives two columns Q, $Q = \{Q_j, R\}$, where Q_j is a join key and R is a numerical target column. The correlation seeker returns the top-k tables that join on Q_j and contain a column that correlates the most with R, sorted by the correlation coefficient.

B. Combiner operators

Combiners enable the discovery task specific composition of seeker results. Combiners receive a set of table collections, where each collection is the result of a seeker or another combiner. A combiner merges the table collections according to its defined set operation and returns a new set of tables. We implemented the most common set operators in BLEND: *Intersection* ($\cap$), *Union* ($\cup$), *Difference*($\setminus$), and *Counter*. Note that the user can introduce new combiners to the system. The **Intersection** and **Union** combiner receive the output of two or more seekers and returns the intersection/union of the tables, respectively. The **Difference** combiner receives only two sets of tables and returns the tables that exist only in the first table set. The **Counter** combiner receives multiple sets of tables, counts the occurrence of the table identifiers, and returns a list of tables sorted in descending order based on their frequencies.

C. Discovery Language Grammar

With the given seekers and combiners, the grammar of the supported discovery plans in BLEND can be defined as follows:

```
expression ::= seeker(Q) |
combiner(expression(,expression)+)
seeker ::= KW | SC | MC | C
combiner ::= ∩ | ∪ | \ | Counter
Q ::= keyword | table
```

Any `expression` based on the grammar above can be subject to optimization of BLEND. Further, it is possible to

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:53 UTC from IEEE Xplore. Restrictions apply.



CHORUS: Foundation Models for Unified Data Discovery and Exploration

Moe Kayali
kayali@cs.washington.edu
University of Washington

Anton Lykov
alykov@cs.washington.edu
University of Washington

Ilias Fountalis
ilias.fountalis@relational.ai
RelationalAI

Nikolaos Vasiloglou
nik.vasiloglou@relational.ai
RelationalAI

Dan Olteanu
olteanu@ifi.uzh.ch
University of Zurich

Dan Suciu
suciu@cs.washington.edu
University of Washington

ABSTRACT

We apply foundation models to data discovery and exploration tasks. Foundation models are large language models (LLMs) that show promising performance on a range of diverse tasks unrelated to their training. We show that these models are highly applicable to the data discovery and data exploration domain. When carefully used, they have superior capability on three representative tasks: table-class detection, column-type annotation and join-column prediction. On all three tasks, we show that a foundation-model-based approach outperforms the task-specific models and so the state of the art. Further, our approach often surpasses human-expert task performance. We investigate the fundamental characteristics of this approach including generalizability to several foundation models and the impact of non-determinism on the outputs. All in all, this suggests a future direction in which disparate data management tasks can be unified under foundation models.

PVLDB Reference Format:
Moe Kayali, Anton Lykov, Ilias Fountalis, Nikolaos Vasiloglou, Dan Olteanu, and Dan Suciu. CHORUS: Foundation Models for Unified Data Discovery and Exploration. PVLDB, 17(8): 2104 - 2114, 2024.
doi:10.14778/3659437.3659461

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at http://github.com/mkyl/CHORUS.

1 INTRODUCTION

Data discovery and exploration are major components of the workflow of analysts and data scientists. A survey conducted by the Anaconda data-science platform in 2021 found that analysts spend 40% of their working hours on data loading and cleaning [2]. Even with this colossal effort, 60-70% of data within an enterprise still goes unused for analytics [21], remaining as *dark data* [23, 63].

Recent developments in large language-models (LLMs) have unlocked human-level performance on diverse domain tasks. The discovery that these models can generalize to diverse domain-specific tasks that they have not been trained on [3, 26, 59, 60] has led to emergence of the term *foundation models* [5].

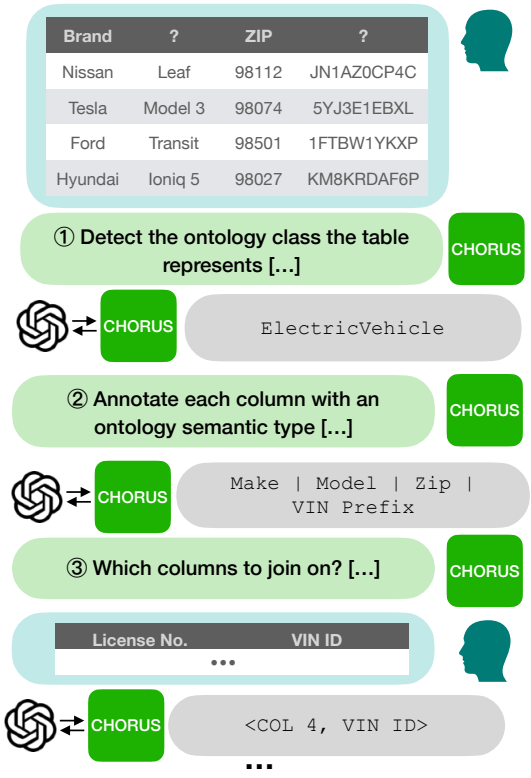


Figure 1: Data discovery tasks considered in this work. Given an ontology, such as DBPedia, ① we assign an overall type to the table and ② we annotate the columns with semantic types. Last, given another table, ③ we predict the join column. The user provides the data while CHORUS interacts with the foundation model. Data from [44], full prompts in Figure 3.

Despite their promise, serious risks have hampered the reception of foundation models. These include: spurious generation (including "hallucination") [24], factual recall limitations [39], bias [19], dataset contamination [14], logical shortcuts [50] and fallacies [38]. Naïve deployment can lead to unanticipated problems: it has already led to legal action [11] and recalls by major corporations [22]. These risks are now acknowledged by the creators of these models [6, 45, 54].

The goal of this paper is to demonstrate the utility of foundation models to the data discovery and exploration while mitigating

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 17, No. 8 ISSN 2150-8097.
doi:10.14778/3659437.3659461

the aforementioned risks. We select three representative tasks to show the promise of foundation models: ① *table-class detection*, ② *column-type annotation* and ③ *join-column prediction*. An outline of our approach is shown in Figure 1. We call our approach CHORUS.

Contributions. We summarize our contributions:

- The first work to use foundation models for the data discovery tasks of table-class detection, column-type annotation and join-column prediction;
- Propose a novel system, CHORUS, whose flexible architecture enables the synthesis of multiple data discovery tasks and deploying risk mitigations;
- Design task-specific approaches that exploit zero- and few-shot strategies and allow information flow between tasks;
- Introduce the novel mitigation of *anchoring* to reduce foundation-model risks specific to this domain;
- Empirically validate CHORUS, comparing its performance with the state-of-the-art baselines across three individual tasks.

Discussion. Prior work has addressed these tasks individually. Landmark approaches like Sherlock [27] trained deep model architectures for a specific task, requiring 100K-1M labeled data points. More recent work such as DoDuo [52] and TaBERT [62] has focused on *representation learning*, learning embeddings for structured data by improving their performance on one or more downstream tasks.

Foundation models allow a substantially different approach: rather than the classical architecture where the outputs of the model are task-specific, the inputs and outputs of the model are natural language text. Training occurs not on tables or data management tasks specifically, but on general text. Performance on domain-specific tasks is solely by generalization.

This results in a high degree of flexibility. Novel tasks can be specified in natural text, without need for expensive data collection—task examples, metadata and constraints are all incorporated into the task easily. Another advantage of this approach is a **unified architecture**: tasks can utilize the overall context and previous outputs. For example, in Figure 1 the table class `ElectricVehicle` helps with deducing the outputs `Make`, `Model` in the next task.

Outline. Section 2 defines the three tasks investigated in this paper. Section 3 describes the architecture of CHORUS and key approaches. We evaluate the performance of CHORUS in Section 4's experiments. In that section, we also investigate the fundamental characteristics of this approach. We offer a discussion of those results in Section 5. This includes a discussion of promising future directions. Finally, we place this work within the literature in Section 6, discussing related works.

2 BACKGROUND

2.1 Tasks

We assume to be given a *data collection* consisting of a number of relational tables T_1,T_2,.... Each table T_i consists of a number of columns, or attributes, A_1,A_2,... and a number of rows, or tuples, r_1,r_2,... The name of a table T_i is, in general, non-informative, for example it may be simply a sequential ID. The columns may optionally have a name H_1,H_2,... or consist only of values.

In addition to the data collection, we are also given a reference ontology of table classes C_1,C_2,..., and a reference ontology of column types τ_1,τ_2,.... For example, the DBPedia.org types for the table classes include https://dbpedia.org/ontology/Actor and https://dbpedia.org/ontology/Continent and column types include https://dbpedia.org/ontology/areaTotal and https://dbpedia.org/ontology/birthDate.

We consider three tasks of interest on the data collection:

Definition 2.1 (① Table-class detection)**.** For each table T_i, determine its appropriate class C_j, such that every row r_1,r_2,... represents an instance of the C_j type. We adopt this definition from [33].

For example, table-class detection on the table given in Figure 1 could output `ElectricVehicle`, since each row of that table is an instance of that class. Alternatively stated, the table is about `ElectricVehicles`.

Definition 2.2 (② Column-type annotation)**.** For each table T_i, find a mapping from its attributes (columns) A_1,A_2,... to the reference column types τ_1,τ_2,..., such that each value in A_i is an instance of the τ_i type. See [1, 13].

For example, column-type annotation on the first column in Figure 1 could output `Manufacturer`, since the values are the respective manufacturers of each `ElectricVehicle`.

Definition 2.3 (③ Join-column prediction)**.** Assume an *execution log* L, a history of user actions including table joins and their join conditions, which maps many $(T_i,T_j) \rightarrow (A_k,A_l)$ where $A_k \in T_i, A_l \in T_j$. Given two tables T and T', with columns A_1,... and A'_1,... respectively, the *join-column prediction* task is to suggest a pair (A_k,A'_l) of columns such that the equality condition $A_k = A'_l$, which can be used to join the the tables, matches with the choice in the execution log L. For more discussion, see [61].

For example, given the table in Figure 1 and another table `car_registration(name, vehicle_id_number)`, join-column prediction could output `(VIN_prefix, vehicle_id_number)`. The correctness of the prediction depends on the ground truth of which columns the user did in-fact join on.

Ontologies Foundation models contain knowledge of ontologies such as DPBedia.org, Freebase and Wikidata. We focus on universal ontologies, that is, ontologies that aim to represent all entities in general. This is in-line with findings that foundation models encode highly technical knowledge, such as clinical reasoning [51] or electrical engineering principles [53].

3 APPROACH

We outline the structure of CHORUS in this section. First, we explore the core idea of ingesting relational data with foundation models and performing data exploration tasks in Subsection 3.1. Next, we describe the necessary post-processing and mitigations we develop in Subsection 3.2.

Figure 2 shows the architecture of the system. CHORUS has a unified architecture which runs multiple tasks in the same context, allowing for information flow. Each task is run sequentially, with the output of one task fed as context into future tasks.

For each task instance, CHORUS generates a prompt by concatenating six inputs: context, demonstration, data samples, metadata,

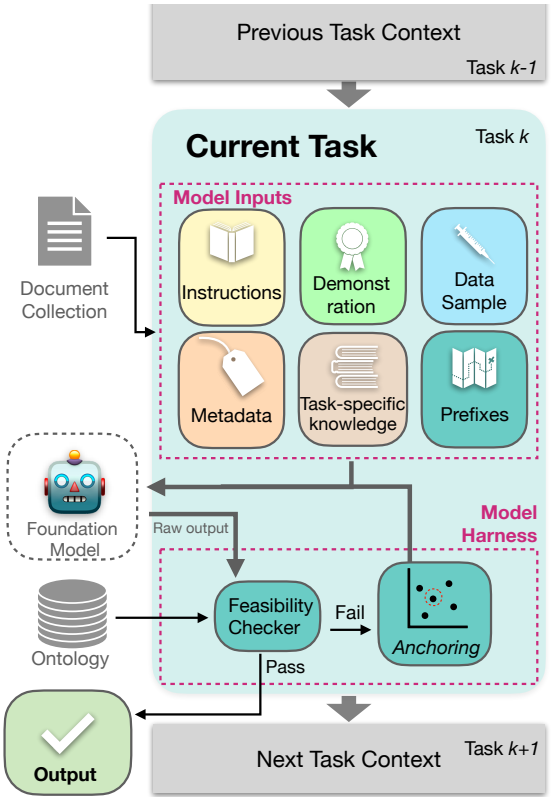


Figure 2: CHORUS system architecture.

task-specific knowledge, and prefixes. They form the "Model Inputs" box in Figure 2 and are color-coded so that they match the colored prompt components in Figure 3. This natural language input is then fed to the foundation model. The output is then subject to post-processing: checks of parsability and feasibility are conducted. If these pass, the output is extracted. Otherwise, we activate a mitigation process, called *anchoring*, in order to repair the error and prevent its propagation.

3.1 Model Inputs

We discuss what inputs are provided to the foundation model and how they are pre-processed and synthesized. We discuss the six components of the Model Inputs module in Figure 2, individually. These correspond to the six color-coded prompt components in Figure 3. Once generated, all the above inputs are concatenated to into a single prompt provided to the model.

Instructions. A description of the specific task (table-class detection, column-type annotation or join-column prediction) is provided to the foundation model in natural language. These are shown in yellow in Figure 3. For example, we translate the formal Definition 2.1 of the first task, table-class detection, into the English sentence "For the following CSV sample, select one DBpedia.org ontology that represents the dataset." For the third task, join-column prediction, we utilize a code-completion approach. We frame the task as code-completing a Pandas fragment that performs a join, with the code to complete shown in Figure 3c. We choose Pandas because it is a very popular framework, with more than millions of example lines of code

Legend: Instruction, Demonstration, Data sample, Metadata, Task-specific knowledge, Prefixes.

```
For the following CSV sample, select one DBpedia.org ontology that
represents the dataset from the following list:
AcademicJournal, AdministrativeRegion, Airline, Airport, [. . . ],
University, VideoGame, Work, Wrestler.
For example, for a dataset about hospitals, return
`https://dbpedia.org/ontology/Hospital`. Begin your answer with
'https://dbpedia.org/ontology'.
```
```
Brand, ␣, ZIP, ␣,
Nissan, Leaf, 98112, JN1AZ0CP4C,
Tesla, Model 3, 98074, 5YJ3E1EBXL,
[...]
```
```
```

(a) Table-class detection

```
Consider this example. Input:
```
```
Name, Famous Book, Rk, Year
Fyodor Dostoevsky, Crime and Punishment, 22.5, 1866
Mark Twain, Adventures of Huckleberry Finn, 53, 1884
Albert Camus, The Stranger, -23, 1942
```
```
Output:
`dbo:author, dbo:title, Unknown, dbo:releaseDate`.
For the following CSV sample, suggest a DBPedia.org Property for each
column from the `dbo:`namespace.
```[...]```
```

**(b) Column-type annotation**

```
Given two Pandas Dataframes, suggest what `pd.merge`
parameters to use to join the dataframes.
df1 =
```[...]```
df2 =
```[...]```
Complete the correct Pandas merge command. `pd.merge(df1, df2, left_on=
```

**(c) Join-column prediction**

**Figure 3: Prompts used in this paper, materialized with examples. Most prompt elements are fixed—only the data sample and metadata change for each instance.**

on the web. This is the *zero-shot prompt* setting: the model can be provided with instructions for a novel task and performs them directly.

*Demonstration.* For the first two tasks, we use the foundation models with task examples as an additional input: this is called the *few-shot prompt* setting. The model is given a few demonstrations of task completion, including inputs and outputs. This is shown in Figure 3 as green text.

*Data sample.* By serializing the input tables, we can input them into foundation models. For example, consider the example table from Figure 1 in the introduction. Serializing the table allows the foundation model to ingest the data. We use the comma-separated values (CSV) format, shown in blue in Figure 3.

Because the models have a limited context window size—typically in the few thousands of tokens—tables cannot always be ingested as a whole. Instead, we always serialize a sample of the rows. We find
```

LakeCompass: An End-to-End System for Data Maintenance, Search and Analysis in Data Lakes

Chengliang Chai*
ccl@bit.edu.cn
Beijing Institute of Technology

Yuhao Deng*
dyh18@bit.edu.cn
Beijing Institute of Technology

Yutong Zhan
zyt@bit.edu.cn
Beijing Institute of Technology

Ziqi Cao
caoziqi@bit.edu.cn
Beijing Institute of Technology

Yuanfang Zhang
zyuanfang@bit.edu.cn
Beijing Institute of Technology

Lei Cao
lcao@csail.mit.edu
University of Arizona/MIT

Yuping Wang
wyp_cs@bit.edu.cn
Beijing Institute of Technology

Zhiwei Zhang
zwzhang@bit.edu.cn
Beijing Institute of Technology

Ye Yuan
yuan-ye@bit.edu.cn
Beijing Institute of Technology

Guoren Wang
wanggr@bit.edu.cn
Beijing Institute of Technology

Nan Tang
nantang@hkust-gz.edu.cn
HKUST (GZ)

ABSTRACT

Searching tables from poorly maintained data lakes has long been recognized as a formidable challenge in the realm of data management. There are three pivotal tasks: keyword-based, joinable and unionable table search, which form the backbone of tasks that aim to make sense of diverse datasets, such as machine learning. In this demo, we propose LakeCompass, an end-to-end prototype system that maintains abundant tabular data, supports all above search tasks with high efficacy, and well serves downstream ML modeling. To be specific, LakeCompass manages numerous real tables over which diverse types of indexes are built to support efficient search based on different user requirements. Particularly, LakeCompass could automatically integrate these discovered tables to improve the downstream model performance in an iterative approach. Finally, we provide both Python APIs and Web interface to facilitate flexible user interaction.

PVLDB Reference Format:
Chengliang Chai, Yuhao Deng, Yutong Zhan, Ziqi Cao, Yuanfang Zhang, Lei Cao, Yuping Wang, Zhiwei Zhang, Ye Yuan, Guoren Wang, and Nan Tang. LakeCompass: An End-to-End System for Data Maintenance, Search and Analysis in Data Lakes . PVLDB, 17(12): 4381 - 4384, 2024.
doi:10.14778/3685800.3685880

1 INTRODUCTION

Many organizations have acknowledged the importance of providing open access to their tabular data for the public good [14]. However, the large amount of tables are often poorly maintained (*e.g.*, lacking of index or schema information). To make sense of this data, a key initial step for the data scientists is to search tables relevant to the task at their hand. To this end, we propose to build an end-to-end prototype system, LakeCompass that driven by the need of the downstream analytics tasks, offers efficient and effective table search over data lakes.

At a high level, LakeCompass features the following three key functionalities.

Index building. We build various types of indexes over tables in data lakes, such as inverted index, local sensitive hash (LSH) and the hierarchical navigable small world (HNSW), which taking both text similarities and semantic similarities into consideration, efficiently and effectively support table search.

Table search. We support three typical categories of table search, *i.e.*, keyword-based, joinable and unionable table search. The former one returns relevant tables based on the given keywords. The latter two take as input a query table, and discovering tables that can be integrated with the query table.

Data Selection for Model analysis. LakeCompass offers a data selection component that taking into consideration the specific need of the downstream applications, *i.e.*, the ML models, intelligently selects the tuples and features from the table search result in a more targeted manner.

Overall, LakeCompass has the following advantages.

[1. *High efficiency.*] Various types of indexes allow us to support efficient search over different queries. In terms of the model analysis module, we propose to use the online learning technique to improve the efficiency of iterative model evaluation.

[2. *High effectiveness.*] We maintain the original data associated with their semantic embeddings, together with various indexes, which allow users to search highly relevant tables customized to their needs. For the model analysis, we propose a multi-armed bandit (MAB) technique to judiciously select beneficial tuples/features among the returned relevant tables.

[3. *User-friendly interaction.*] We encapsulate the above functionalities through flexible Python APIs, where the user can upload

*Both authors contributed equally to this research.
Ye Yuan is the corresponding author.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 17, No. 12 ISSN 2150-8097.
doi:10.14778/3685800.3685880

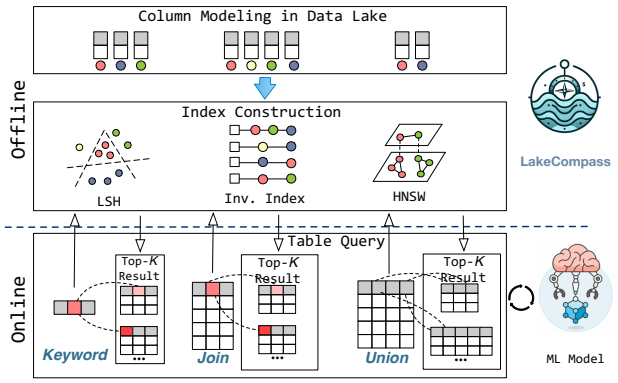


Figure 1: Overall Architecture of LakeCompass.

data, construct indexes, search tables and build ML models. We also provide a Web interface to serve table search and analysis in a more intuitive way.

Existing Works. There exist several prototypes of table search. Aurum [6] takes as input keywords and returns relevant tables based on their schema information. COCOA [5] focuses on searching joinable tables; Auctus [1] supports both joinable and unionable table search; Besides the above table search, Juneau [7] further supports feature extraction and data cleaning based on the returned tables. Similarly, DIALITE [12] can also search tables that can be integrated with the query table, and then conducts some data analysis like entity matching. Although the above prototypes supports table search in data lakes, they do not consider the semantic information of columns/rows of tables, which is rather significant in achieving high effectiveness. What's more, they do not take the machine learning task into consideration, which is the key application for table search in data lakes. Although some works [3, 9] have studied data augmentation for ML, they assume the relevant data is discovered beforehand by another system, hence not offering an end-to-end table search solution.

In summary, LakeCompass is an end-to-end system that is adequate to demonstrate the efficiency and effectiveness of table search for real-world applications.

2 SYSTEM OVERVIEW

At a high level, LakeCompass consists of the following modules: data lake embedding, index construction, online table query processing as well as the model analysis, as shown in Figure 1. The former two modules are offline, which respectively encode the column/schema in the data lake into a vector and index these vectors. Once an online query table arrives, we first encode its column(s) and then use the index to retrieve relevant tables with the highest relevance score from the data lake. If the query is a keyword, we just encode the query and retrieve relevant tables.

Embedding Data Lake. Generally speaking, column representations are significant in table search. The columns of the original tables in data lake are always represented as fixed-length vectors. The vectors are mostly hash codes [6, 17] (*e.g.*, generated by the minhash function) or embeddings [8, 10] (*e.g.*, generated by pre-trained language models) that are effective in finding columns with similar semantics or high overlaps, which are rather significant in searching joinable/unionable tables. However, some methods like Josie and InfoGather [15, 16] directly use the original cell values of each column to search highly overlapping columns rather than using vectors. Besides, we also maintain the metadata of data lake tables (*e.g.*, table name, caption and schema information, etc.) and index them, which are also important in table search, especially for keyword-based search.

Index Construction. Almost all table search methods rely on an index to search large table repositories. Building upon the representations discussed above, several typical ANN indexes like LSH [17] or HNSW [8] can be utilized to index all the embeddings. As another option, inverted index [11, 15, 16] accelerates the process of finding highly overlapping columns, where each cell value is linked to the columns that contain the value. In this case, these colorful circles represent Column IDs rather than vectors. All the types of indexes are supported in LakeCompass.

Online Table Search. For keyword-based search, the user issues the keywords, which are encoded as embeddings, and retrieve tables with similar embeddings on metadata or columns. For joinable search, LakeCompass supports users in issuing a query table through either user-uploaded files or the result of keyword-based search. Once a user issues a table T_J and a specific column c for search, LakeCompass first represents c as a vector. It then uses the index to quickly identify a of top-K columns (tables), denoted by $R(T_J)$, with high relevance scores. The unionable search follows the same procedure. The only difference is that given a table T_U, it has to search over each attribute in T_U and aggregate the results. Specifically, for each column $c_i \in T_U$, similar to the join search, we retrieve from the data lake a set of columns that have high relevance scores with c_i. Then, we compute the union of all retrieved tables. For each table T in the union set, we compute the relevance score using techniques like maximum bipartite graph matching, considering the column relevance between the T_U and T. Finally, a set of top-K tables, denoted by $R(T_U)$ with the highest relevance scores are returned. In LakeCompass, we have implemented various joinable [15–17] and unionable search [8, 11, 15] algorithms.

Iterative Data Selection for Model Analysis. When the query table serves as the train data, due to the data scarcity problem, we expect to augment more tuples/features through unionable/joinable table search. However, although all the returned tables $R(T_J)/R(T_U)$ can be integrated into the query table, integrating all of them may not improve the model performance. The reason is that they come from heterogeneous sources in data lakes, some of which are in fact noisy to the model. Therefore, we apply our proposed tuple/feature augmentation techniques [2, 13] to judiciously select tuples/features (from unionable/joinable tables), iteratively integrate to the query table and evaluate the performance. Here, we summarize the high level idea [2] of selecting tuples from unionable tables.

i). *Clustering.* Given all tables in $R(T_U)$, we first merge all tuples of these tables, and then cluster these tuples based on their feature similarities. Then tuples in each cluster can be seen as homogeneous. Next, we use an iterative MAB solution to judiciously select tuples that can benefit the model from these clusters.

ii). *MAB-based data selection.* We take each cluster as a bandit arm. Each arm is assigned with an upper confidence bound (UCB) score,

```
# build index for the datalake
datalake = LakeCompass.DataLake('/datalake_demo')
keyword_index = datalake.metadata.indexing(
    index_type='HNSW', search_type='keyword',
      config=LakeCompass.HNSWconfig())
# keyword based search
candidate_table = LakeCompass.keyword_search(
    keyword_index, 'education')
```

Figure 2: A Code Segment for Index Building and Keyword-based Search of LakeCompass.

which will be updated after each iteration of tuple selection. At the beginning of each iteration, the arm with the highest UCB score will be selected, then a small batch of tuples will be sampled from the cluster and added to train. If the model performance is tested to be improved on a validation set, a reward will be assigned. At a high level, the UCB score is computed by the summation of an exploration and an exploitation score. The less frequently a cluster is picked, the higher the exploration score. The higher reward the cluster has obtained, the higher the exploitation score. Hence, a combination of the two scores used by MAB-based method can well handle the exploration-exploitation trade-off. Specifically, for each cluster C_i, at iteration k:

$$UCB(C_i, k) = R_i^k + \alpha\sqrt{2\ln n^k/(n_i^k+1)} \quad (1)$$

where α is pre-defined, n_i^k is the total number of times that C_i was assigned non-zeros scores from iterations 1 to k, and n^k is the sum of n_i^k for all clusters at iteration k. In Eq. 1, the former part (R_i^k) refers to the *exploitation*, which means to focus more on the cluster where some tuples have resulted in much performance improvement. The latter part ($\sqrt{2\ln n^k/(n_i^k+1)}$) refers to the *exploration*, *i.e.*, focusing more on the clusters that are rarely picked.

iii). *Iterative model evaluation.* We iteratively apply the MAB-based solution until the model performance does not increase within several successive iterations. In order to accelerate the iterative training, we can apply the online learning technique that only trains over the added tuples.

Similarly, the MAB-based solution can also be extended to augment useful features from joinable tables [13].

3 DEMONSTRATION SCENARIOS

This section showcases our demonstration scenarios, which highlight the main functionalities of LakeCompass. These scenarios illustrate how LakeCompass identifies relevant tables within the data lake to enrich tuples or features, thereby enhancing the performance of downstream models. Users can utilize LakeCompass functionalities via Python APIs or Web interfaces.

Datasets. We support index construction, table search and model analysis over 1TB data collected in our benchmark [4] – **100X** larger than the data lakes used in existing works, and the number of columns is up to **100 million**, on which we implemented and evaluated multiple typical table union/join search methods. Moreover, we also provided a sufficient number of labeled diverse

```
# train configuration
predict_col = 'Educational Category'
model_config = {'model': 'SVM', 'type': 'classification',
                'k': 4, 'model_config': SVMconfig()}
query_table = LakeCompass.read('education_Des_Moines.csv')
query_table_model = DownstreamModel(**model_config)
val_set = pd.read_csv('val_set.csv')
test_set = pd.read_csv('test_set.csv')
# trian a model using query table
query_table_model.train(query_table, predict_col, val_set)
query_table_model.eval(test_set, predict_col)
```

Figure 3: A Code Segment for Training Model based on the Query Table.

```
# unionable table search
union_index = datalake.columns.indexing(
    index_type='HNSW', search_type='union',
    config=LakeCompass.HNSWconfig())
unionable_tables = LakeCompass.union_search(
    union_index, query_table)
```

Figure 4: A Code Segment for Unionable Table Search of LakeCompass

queries (more than 10 thousand) – **10X** more than any other works, so as to test the algorithm performance. To this end, we organized a team of 25 graduate students and spent more than *7,500 human hours* on labeling the queries.

(1) **Index Building.** LakeCompass enables the utilization of various index types (including the LSH, HNSW, and inverted index.) to support the typical three table search tasks. As shown in Figure 2, the user builds the HNSW index over the metadata to support keyword-based search. Note that we also support the users to build their own data lakes and build indexes.

(2) **Keyword-based Search.** Users could obtain the training dataset by keyword search. To be specific, as shown in Figure 2, The user could use the Python APIs to set "education" as a keyword and utilize the "keyword_search" function to retrieve relevant tables. Then, tables related to education in different cities of different states in the U.S. are retrieved from our data lake.

(3) **Unionable Table Search.** As shown in Figure 3, users have the flexibility to either upload a query table or choose a table from the keyword-based search as the query table. In such cases, the model based on the solely query table tends to show poor performance. Therefore, they can then employ the "union_search" function to retrieve unionable tables based on semantic similarities. As depicted in Figure 4, users select the "education_Des_Moines.csv" as the query table. Subsequently, additional tables pertaining to education in other cities within the state of Iowa are retrieved through the union search. These retrieved tables share multiple semantically identical columns with the query table, enabling them to be seamlessly unioned together.

(4) **Joinable Table Search.** Users could also interact with the Web

METAM: Goal-Oriented Data Discovery

Sainyam Galhotra, Yue Gong, Raul Castro Fernandez
Department of Computer Science, The University of Chicago
Email: {sainyam, yuegong, raulcf}@uchicago.edu

2023 IEEE 39th International Conference on Data Engineering (ICDE) | 979-8-3503-2227-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICDE55515.2023.00213

***Abstract*—Data is a central component of machine learning and causal inference tasks. The availability of large amounts of data from sources such as open data repositories, data lakes and data marketplaces creates an opportunity to augment data and boost those tasks' performance. However, augmentation techniques rely on a user manually discovering and shortlisting useful candidate augmentations. Existing solutions do not leverage the synergy between discovery and augmentation, thus underexploiting data.**

In this paper, we introduce METAM, a novel goal-oriented framework that queries the downstream task with a candidate dataset, forming a feedback loop that automatically steers the discovery and augmentation process. To select candidates efficiently, METAM leverages properties of the: i) data, ii) utility function, and iii) solution set size. We show METAM's theoretical guarantees and demonstrate those empirically on a broad set of tasks. All in all, we demonstrate the promise of goal-oriented data discovery to modern data science applications.

I. INTRODUCTION

Augmenting a dataset by joining it with others can improve the utility of data-driven tasks such as causal inference and supervised machine learning. The abundance of tables in open repositories [1], lakes in organizations [2], and even data markets [3], [4] translates into an abundance of *augmentation candidates*. Identifying good augmentation candidates among many is a data discovery problem. To solve this problem, one could use a traditional data discovery system to identify what tables join with the input data, and then, separately, identify what joins increase the task's utility. This *discover-then-augment* approach works when the discovery system returns candidates relevant to the task. Unfortunately, in practice, it is hard to guarantee the discovery system identifies good augmentations because: i) relevant augmentations depend on the task; and ii) analysts may not know what properties make an augmentation relevant, e.g., what features augment the predictive power of a classifier. The disconnection between data discovery and augmentation presents a research opportunity.

We harness that opportunity with a new approach we call **goal-oriented data discovery**, where data discovery is not treated as a one-time process. Instead, it adapts to the task by performing interventions: a process to augment the initial dataset and validate its utility. *Interventional* queries help to identify augmentations that *cause* an increase in the task's utility, thus steering the discovery process to automatically maximize the task's utility. This achieves two objectives: first, analysts do not need to know what criteria make an augmentation good because the approach is automatic. Second, any downstream task with a utility function benefits from this discovery approach. The consequences of goal-oriented data discovery are significant; consider the following anecdote. We used goal-oriented data discovery to predict "housing prices" in a geographical area. Our technique identified some obvious datasets that a social scientist would have been able to identify using a discovery system, such as "income of people staying in the neighborhood" and "crime stats". But crucially, it also identified non-obvious datasets correlated with housing prices such as "presence of grocery stores" and "number of taxi trips" from those areas. Indeed, many sociologists and economists leverage external data to infer causal relationships between attributes of interest [5], [6], [7]. But they rely on domain knowledge and manual effort to identify those relationships. *Goal-oriented data discovery* paves the way to identify new causal relationships from large data repositories automatically.

A trivial but computationally prohibitive way of solving goal-oriented data discovery is to measure the utility of every augmentation candidate and choose the best. The key technical contributions of our paper focus on developing *interventional querying algorithms* for goal-oriented data discovery that exploit the structure of the data, the utility function, and the solution to adaptively prioritize the candidates for querying.

Interventional Queries for Goal-Oriented Data Discovery. Our technical contributions leverage the following properties to optimize the complexity of discovery without loss of quality.

Properties of the Data. A key insight is that *similar* augmentations perform similarly on the downstream task. We exploit this insight by clustering augmentations and judiciously choosing them from different clusters, thus skipping computation. To cluster augmentations, we represent each with a vector of *data profiles*, which are task-independent measures of data and include semantic similarity, correlation, and mutual information, among others. When a combination of data profiles is correlated with the task's utility, clustering narrows down the number of augmentation candidates.

Properties of the Utility function. A task's optimal utility is given by a set of augmentations. Enumerating all subsets is infeasible. Our insight is that when utility functions are monotonic, i.e., utility never decreases with new augmentations, it is possible to find augmentations efficiently, by considering them one by one. And we can make any function monotonic by ignoring augmentations that harm utility using a wrapper around the user-provided task implementation.

Properties of the Solution set. The optimal set of augmentations is a collection of join paths over different datasets. Most augmentations are irrelevant for a downstream task, and useful augmentations are a small subset of the candidate set. We leverage this property to prioritize small subsets of join

2375-026X/23/$31.00 ©2023 IEEE
DOI 10.1109/ICDE55515.2023.00213

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:59:06 UTC from IEEE Xplore. Restrictions apply.

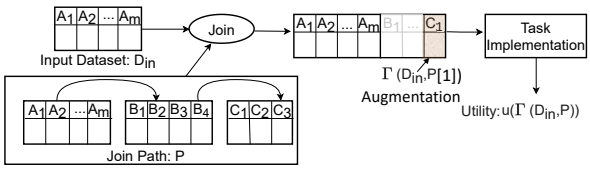


Fig. 1: Notation of a data augmentation pipeline.

paths over larger ones by using combinatorial testing.

METAM: Goal-oriented data discovery We combine the aforementioned properties into an anytime algorithm that finds augmentation candidates by adaptively querying the task. Although goal-oriented data discovery is NP-hard, the algorithm is guaranteed to identify an approximate solution under reasonable assumptions which hold in practice. We also show the efficiency guarantees of the algorithm, and implement it as part of METAM, an end-to-end data discovery approach.

Evaluation Results. METAM automatically finds useful augmentations among millions of them within minutes. We evaluate METAM on large repositories with data from cities in the United States and Kaggle, and for prescriptive analytics tasks such as *what-if* and *how-to* analysis in causal inference, classification and regression in supervised ML, as well as entity linking, and clustering.

Outline. We present notation and discuss the problem in Section II. We present insights for interventional queries in Section III, algorithm in Section IV, and it's analysis in Section V. We present evaluation in Section VI, and related work in Section VII.

II. Goal-Oriented Data Discovery

In this section, we introduce notation and problem statement in Section II-A, and preliminaries in Section II-C.

A. Problem Definition

Figure 1 presents a summary of the notation used in the paper. Let $\mathcal{R}(A_1, \ldots, A_m)$ denote a relation schema over m attributes, where A_i denotes the ith attribute. D comprises a schema $\mathcal{R}(A_1, \ldots, A_m)$ and a list of tuples T where each tuple $t \in T$ is a specific instance of the schema. Using this notation, we now define a noisy dataset and a data repository.

Definition 1 (Noisy Structured Data)**.** *A noisy structured dataset D is characterized by incomplete schema information $\mathcal{R}(A_1, \ldots, A_m)$ where $A_i = \phi$ denotes missing header values and a list of tuples T which may contain less than m values. Additionally, certain tuples may contain duplicate values.*

Definition 2 (Data Repository)**.** *A data repository contains a set of datasets $\mathcal{D} = \{D_1, \ldots, D_n\}$, where different datasets D_i, D_j may contain overlapping values. Additionally, the datasets may lack schema and contain missing values.*

The different datasets that are joined with the input dataset D_{in} are denoted as a join path, defined below.

Definition 3 (Join Path)**.** *A join path P is defined as an ordered list of noisy datasets $P \equiv \{D_1, \ldots, D_t\}$ such that datasets D_j and D_{j+1} join for all $j < t$ with each other forming a chain of join operations. The dataset formed by joining these t datasets is considered to be augmented with the original dataset D_{in}.*

The augmentation of a join path P to a dataset D_{in} in denoted by $\Gamma(D_{in}, P)$. Augmenting a join path P with D_{in} can augment multiple new columns, some of which may not be required by the downstream task. To distinguish between these different augmented columns, we define augmentation as the projection of the join consisting of a single column.

Definition 4 (Augmentation)**.** *An augmentation $\Gamma(D_{in}, P[j])$ is defined as the jth column that is added after joining the path P with the input dataset D_{in}.*

Note that $\Gamma(D_{in}, P) \equiv \{\Gamma(D_{in}, P[j]), \forall j \in \{1, 2, \ldots, l\}\}$, where l is the number of columns after materializing the join-path P. Γ considers materializing a join path to add a new attribute. Other types of data augmentation include unions (addition of additional rows), relational embedding based augmentations, spatio-temporal augmentations, etc. Empirically, we observed that join-path based addition of new attributes is commonly used for popular applications. However, our framework extends to any general set of augmentations. A task t is a black box consisting of the analysis algorithm that takes a dataset (D_{in} or an augmented version of D_{in}) as input and outputs the performance measure of the task. We call "query" the process of obtaining a task's utility on a dataset. We formally define the utility score of the task as follows.

Definition 5 (Utility score)**.** *Given a task t that operates on a dataset D, the utility score $u_t(D)$ is defined as the objective (or the evaluation metric) of the task when operated with D.*

Without loss of generality, we assume that the utility score is normalized $u_t(\cdot) \in [0, 1]$ and that higher utility score means better task quality. Different sets of augmentations may improve the task's utility. We seek a *minimal* set that contains only the augmentations that help to maintain high utility.

Definition 6 (Minimal set of Augmentations)**.** *Given an initial dataset D_{in}, a mechanism to compute $u_t(\cdot)$, a set of augmentations $\mathcal{P} = \{P_1[i_1], \ldots, P_j[i_j]\}$ is considered minimal if removing any attribute from $\Gamma(D_{in}, \mathcal{P})$ reduces task utility.*

Problem II.1 (Goal-oriented discovery)**.** *Given an initial dataset D_{in}, a task t that has a mechanism to compute $u(D')$ for any table D', the problem of identifying augmentations that optimize the task is to identify a minimum augmentation set $\mathcal{T}$ such that the augmented dataset has utility $u(\Gamma(D_{in}, \mathcal{T})) \geq \theta$.*

B. Task implementation and utility calculation

A task takes a dataset as input and returns a utility score that depends, at least in part, on the input dataset. Some examples:

1) Predictive analytics: The task trains a machine learning model over an input training dataset. The training code could be simple, e.g., a random forest, or complex, such as an AutoML solution. The utility correspond to a model-relevant metric such as accuracy, F1 score and may also include fairness and robustness metrics.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:59:06 UTC from IEEE Xplore. Restrictions apply.

2) Prescriptive analytics involves hypothetical analysis (what-if and how-to queries), causal inference, and explainability as the key components [8], [9], [10], [11]. The utility corresponds to a metric that summarizes the quality of generated explanations, support of identified causal dependencies, and total causal effect of attributes.
3) Data preparation and visualization: These include entity resolution, entity linking, data cleaning and visualization. The utility often corresponds to F-score and accuracy.

Note that these are some examples of task implementation and any advances in feature engineering, data transformation and data analysis can be incorporated in the utility function.

C. Preliminaries and Problem Discussion

We use previous data discovery techniques [12], [13], [14], [15] to obtain a set of augmentations from large data repositories; we use Aurum [12]. Although existing solutions may generate noisy candidates (due to the use of approximate techniques and semantic ambiguity), our approach works even when different sources of noise are present. We also leverage existing profiling techniques [16], [17], [18], [19] to describe the augmentation candidates $\Gamma(D_{in}, \mathcal{P})$. For example, a data profile $\langle Corr, C_1, C_2 \rangle$ refers to the correlation between columns C_1 and C_2. We use the term *data profile value* to denote the value of the characteristic/property. A data profile may refer to a single attribute (e.g. domain of an attribute) or multiple attributes (correlation between attributes), and these properties may be correlated with downstream utility.

Definition 7 (Data Profile)**.** *A data profile X is defined as the property of a dataset D such that the tuples in D satisfy X.*

We consider the following data profiles in our implementation.
• **Correlation and Mutual Information (MI)**: MI is often used to evaluate causal relationship between attributes [20]. Correlation and MI are also predictors of the quality of attributes in machine learning tasks. This profile estimates the Pearson correlation (MI) of the candidate augmentation P and the attributes of D_{in}.
• **Semantic-embedding based distance** profile captures the semantic similarity between the considered datasets. This profile is computed as the cosine similarity between the embeddings of both datasets constructed from pre-trained models such as BERT [21]. The dataset embedding is constructed by averaging the embedding vectors of tokens present in the table.
• **Dataset metadata/attributes** profile calculates the similarity between datasets based on their source and attributes. Unlike the semantic embeddings, this profile captures syntactic similarity between attributes and dataset source, which is commonly used to estimate the quality of augmentations [22]
• **Dataset overlap** profile calculates the cardinality of the final dataset identified after augmentation.

Extending to other data profiles. Metam can be extended with new profiles to cater for new downstream tasks (e.g., anomaly detection [23], conditional independence for fairness [24]) or to leverage advances in profiling techniques [23], [25], [26], [27].

Problem Discussion: We discuss how various discovery-then-augment baselines fail to solve the problem efficiently:
• **Join Everything**: This technique joins D_{in} with every join path identified. This approach may bring irrelevant attributes that deteriorate the utility. Furthermore, the approach is infeasible in large-scale scenarios with thousands of augmentations.
• **Uniform sampling**: This technique samples join paths out of the identified joinable datasets uniformly at random and uses them to augment D_{in}. This approach does not guarantee that the chosen sample will improve the utility score.
• **Join Path overlap ranking**: A common technique is to rank join paths based on the cardinality of the augmented datasets (used by S4 [14] and Ver [22]). This technique identifies datasets that contain fewer missing values, but does not guarantee to optimize the task.
• **Using data profiles for selection**: Analysts may use intuition to rank augmentations based on profiles that are expected to be useful. This approach may work if the profiles are accurate and the domain scientist's intuition is correct. But this approach will not work well otherwise, and it relies on accurate estimation of profiles, which are hard to compute in large-scale data repositories, where approximate techniques are often necessary for scalability.

The crux of the problem is that these baselines are unaware of the task, so there is no guarantee that the chosen augmentations will improve utility. Solving goal-oriented data discovery requires an interventional approach that seeks to understand what augmentations *cause* the task's utility to increase.

III. Intervention-based Querying

In this section, we first describe the limitations of baseline techniques and then discuss how the properties of data, utility function, and solution let us design an efficient technique.

A. Baseline Interventional Techniques

Goal-oriented data discovery can be solved by enumerating every subset of candidate augmentations, computing their utility, and choosing the minimal subset that achieves utility of at least θ. With n candidate augmentations, this process may require up to $O(2^n)$ queries (does not finish for $n > 30$), so this approach is unfeasible. We consider two improvements:
• **Utility-based selection:** Given a ranking of augmentations (e.g., based on some data profiles) this solution queries, iteratively, in ranking order until the utility obtained is higher than the desired threshold, θ. This solution will be inefficient every time the data profile is not related to the task. And choosing a data profile related to the downstream task is non-obvious for many of the interesting data-driven tasks considered.
• **Prediction from expert advice:** To avoid selecting a profile a priori, different data profiles can be treated as experts and we can use expert selection techniques to solve the problem. For example, the multiplicative weights update method (MW) [28] estimates the ability of different experts to improve utility. MW guarantees the selection of the best expert in hindsight. In its simplest form, considering each profile as a different expert fails to identify combinations of profiles that best

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:59:06 UTC from IEEE Xplore. Restrictions apply.

Selecting Sub-tables for Data Exploration

Yael Amsterdamer
Bar-Ilan University
amstery@cs.biu.ac.il

Susan B. Davidson
University of Pennsylvania
susan@cis.upenn.edu

Tova Milo
Tel Aviv University
milo@post.tau.ac.il

Kathy Razmadze
Tel Aviv University
kathyr@mail.tau.ac.il

Amit Somech
Bar-Ilan University
somecha@cs.biu.ac.il

2023 IEEE 39th International Conference on Data Engineering (ICDE) | 979-8-3503-2227-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICDE55515.2023.00192

***Abstract*—Data scientists frequently examine the raw content of large tables when exploring an unknown dataset. In such cases, small subsets of the full tables (*sub-tables*) that accurately capture table contents are useful. We present a framework which, given a large data table T, creates a sub-table of small, fixed dimensions by selecting a subset of T's rows and projecting them over a subset of T's columns. The question is: Which rows and columns should be selected to yield an *informative* sub-table?**

Our first contribution is an informativeness metric for sub-tables with two complementary dimensions: *cell coverage*, which measures how well the sub-table captures prominent data patterns in T, and *diversity*. We use association rules as the patterns captured by sub-tables, and show that computing optimal sub-tables directly using this metric is infeasible. We then develop an efficient algorithm that indirectly accounts for association rules using *table embedding*. The resulting framework produces sub-tables for the full table as well as for the results of queries over the table, enabling the user to quickly understand results and determine subsequent queries. Experimental results show that high-quality sub-tables can be efficiently computed,and verify the soundness of our metrics as well as the usefulness of selected sub-tables through user studies.

***Index Terms*—Interactive data exploration and discovery**

I. Introduction

Data exploration is the process of understanding an unfamiliar dataset and determining what part of the data is relevant to an analysis task. The process is often done by iteratively applying exploratory queries, examining their results, and interpreting them with data visualizations.

Numerous lines of work have studied solutions for different aspects of the exploration process, such as query suggestions (e.g., [1], [2]), query formulation for non-programmers [3]–[6], insights-discovery [7]–[9], and visualization recommendations [10]–[12].

In this work, we focus on the task of *tabular data display*: before applying additional queries or visualizations, users often manually examine the raw table or query results. This is done using commands such as the table display of the popular Pandas library[1], which shows a small subset of the raw table rows and columns that is easier for a human user to inspect than the full table. However, a disadvantage of the Pandas tabular display is that its choice of rows and columns to display is arbitrary: it includes a few of the first and last rows and columns. As a result, the sub-tables may include arbitrary data values and cross-column value combinations rather than ones that reflect prominent trends; it may also elide columns that are important for further exploration and analysis.

[1]Pandas: Python Data Analysis Library. https://Pandas.pydata.org/

Example I.1. *Consider a table T taken from the Kaggle* flights *dataset [13] which contains 31 columns and ∼6M rows. T may be used for analysis tasks such as predicting flight delays and cancellations.*

The analyst starts by inspecting the data using Pandas display(T)*, which yields the sub-table displayed at the top left of Figure 1.*

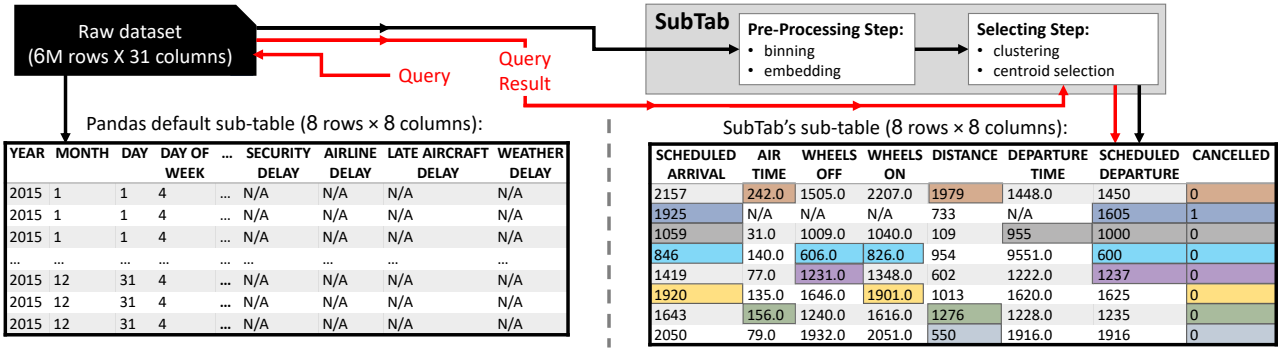
The usefulness of this display for data exploration is intuitively limited: for instance, the first rows, last rows, and last columns are repetitive, and the displayed values are arbitrary. In addition, the chosen columns may not be relevant for the analyst's goals: for instance, the analyst may be interested in flight cancellations, yet there is no reason to assume that the columns in the Pandas *sub-table are relevant for this goal.* □

Our goal is to develop a foundational approach for an informed selection of sub-tables that characterizes a given table as a whole and captures multiple diverse patterns. Concretely, given a table T with n rows and m columns, our goal is to create a sub-table T_{sub} with $k << n$ rows and $l << m$ columns which is a subset of k rows projected over a subset of l columns of T. To do this, we introduce a novel notion of sub-table informativeness which reflects (1) the extent to which the sub-table captures prominent *data patterns* and trends in the full table, including cross-column patterns. This is based on *association rules* (ARs) as a standard type of pattern that applies to a subset of a row's cells and hence allows several such patterns to be captured is a small sub-table. (See a discussion of other types of patterns in Section VII.); (2) the *diversity* in the sub-table, including representative values from each selected column and minimizing data repetitions. We also allow users to specify *target columns* that affect the choice of sub-tables so that it captures columns and ARs relevant to the target column.

Example I.2. *Continuing with the* flights *dataset, an* informative *sub-table is shown at the right of Figure 1. Unlike the sub-table at the top left, this sub-table captures several prominent ARs that hold over the input table and that include the target column* CANCELLED*. For purposes of illustration, we show this by highlighting cells that participate in one of the ARs that hold for the row (many additional ARs hold). For example, the highlighted rule in the first row states that long flights (*AIR_TIME$\in [198.0, 422.0]$ *and* DISTANCE$\in [1546.0, 2724.0]$*) are likely* not *to be cancelled (*CANCELLED$= 0$*). The highlighted rule in the second row states that short afternoon flights (according to the* SCHEDULED_DEPARTURE *and* SCHED-

2375-026X/23/$31.00 ©2023 IEEE
DOI 10.1109/ICDE55515.2023.00192

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:58:35 UTC from IEEE Xplore. Restrictions apply.



YEAR	MONTH	DAY	DAY OF WEEK	...	SECURITY DELAY	AIRLINE DELAY	LATE AIRCRAFT DELAY	WEATHER DELAY
2015	1	1	4	...	N/A	N/A	N/A	N/A
2015	1	1	4	...	N/A	N/A	N/A	N/A
2015	1	1	4	...	N/A	N/A	N/A	N/A
...	...	...	...	...	...	...	...	...
2015	12	31	4	...	N/A	N/A	N/A	N/A
2015	12	31	4	...	N/A	N/A	N/A	N/A
2015	12	31	4	...	N/A	N/A	N/A	N/A

SCHEDULED ARRIVAL	AIR TIME	WHEELS OFF	WHEELS ON	DISTANCE	DEPARTURE TIME	SCHEDULED DEPARTURE	CANCELLED
2157	242.0	1505.0	2207.0	1979	1448.0	1450	0
1925	N/A	N/A	N/A	733	N/A	1605	1
1059	31.0	1009.0	1040.0	109	955	1000	0
846	140.0	606.0	826.0	954	9551.0	600	0
1419	77.0	1231.0	1348.0	602	1222.0	1237	0
1920	135.0	1646.0	1901.0	1013	1620.0	1625	0
1643	156.0	1240.0	1616.0	1276	1228.0	1235	0
2050	79.0	1932.0	2051.0	550	1916.0	1916	0

Fig. 1: System Workflow. SubTab takes as input a raw table used throughout the EDA session. The sub-table on the left is the output of the display command in Pandas, a random subset of the table's rows and columns. The sub-table on the right is the output of SubTab, in which some of the ARs are highlighted for purposes of illustration. SubTab has two-steps: (1) Pre-Processing Step - the table is binned and embedded into vectors; (2) Selecting Step - triggered by a display of the input table or as the result of a query over the table, an *informative sub-table* is displayed. This is done by clustering the row and column vectors of the query result in the embedded space and selecting the centroids as representatives.

ULED_ARRIVAL columns) are likely to be canceled. The sub-table also shows diverse values per column by including representatives of diverse sub-groups in the value distribution, e.g., morning, afternoon and night arrivals/departures, short, medium and long flight distances, etc. □

We formalize (Section II) our notion of informativeness based on a combination of two complementary metrics: *cell coverage* and *diversity*. Cell coverage measures how well the sub-table T_{sub} represents data patterns in the full table T, which are defined via ARs.

Concretely, given a set of prominent ARs that hold over the full table T,[2] our metric reflects the number of cells in T that are describable by ARs that are captured in T_{sub}. If one or more *target columns* are known in advance to be the focus of the analysis, they will be included in the l selected columns, and we measure cell coverage only according to ARs that include one or more target columns. For the second metric, *diversity*, we split the value distribution of a column (continuous or categorical) into meaningful *bins*. We then compute a Jaccard-like row similarity score based on these bins, and use the average pairwise row distance as the diversity score. Intuitively, in a small sub-table we cannot capture the full distribution of each column, and our metric is geared towards increasing the number of represented bins and reducing bin repetitions. We use a score combining these metrics to measure the informativeness of a sub-table.

Unfortunately, we show (Section III) that optimizing our informativeness measures directly is generally infeasible, and discuss the limitations of approximate solutions. In particular, although there are several efficient techniques for mining ARs (e.g., [14]–[18]), they are still too time-consuming for large datasets in an interactive setting.

We therefore propose a sub-table selection method which *indirectly* accounts for ARs using *table embedding* [19], [20], and that allows us to avoid calculating ARs thereby speeding up and simplifying the entire process (Section IV). Concretely, given a table T we use *binning* [15] to split each column's values into a small set of meaningful groups. We then compute an embedding of table cells as vectors. As we demonstrate in Section IV-B, the embedding captures bin co-occurrences, and therefore implicitly corresponds to ARs. To select rows and columns for a sub-table we derive from the cell vectors a vector representation for rows and columns, cluster them (separately), and select the centroids as rows and columns that represent diverse patterns in the data.

An important benefit of our solution design is in *responding to queries over* T: during the exploratory data analysis (EDA) session, users typically issue different queries on a given table T (red arrows of Figure 1). Our computation of cell embedding may be viewed as a part of the pre-processing step of a given data table T, along with the binning of its values (first blue box in Figure 1). This step is executed once upon loading the table. Then, for subsequent selection-projection query Q on T, we can compute the vector representation of rows and columns in $Q(T)$ based on the cells of T that appear in them, and re-execute clustering and centroid selection (*Selecting* step in Figure 1, shown as the second blue box). This significantly speeds up sub-table computation compared with computing everything from scratch (a few seconds instead of up to a minute for large tables).

Contributions: Main contributions of this work include:

1) A formal notion of *informativeness* for sub-tables, designed to measure how well a sub-table characterizes the full table. The capturing of prominent data patterns in the sub-table is measured by *cell coverage* (for ARs), while the ratio of non-repetitive, representative values is measured by *diversity*.

2) A *complexity analysis* of the problem of selecting an optimal sub-table, which shows the infeasibility of computing sub-tables that optimize our informativeness metrics.

3) A *greedy algorithm* (CCSG), which traverses column combinations, and greedily selects rows for each such combination. This algorithm approximates the optimal cell coverage if all column combinations can be traversed within a given

[2]There are standard metrics we can use to measure the prominence of ARs in T, such as Support and Confidence [14]. Also see Section V-D.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:58:35 UTC from IEEE Xplore. Restrictions apply.

time limit. We also present RCCSG, a variant that samples rows to speed up row selection. We discuss the practical limitations of both algorithms, and use them as baselines against which we compare the quality of computed sub-tables.

4) A *practical algorithm*, SubTab, for computing informative sub-tables, which accounts for ARs indirectly using table embedding. The algorithm has two phases: a pre-processing phase that performs binning and embedding, and can be executed as soon as the data is loaded; and a clustering and centroid selection phase that is called for each sub-table display, e.g., on query results.

5) An *implementation* for SubTab as a local Python library that hooks into Pandas, and displays tables and query results as informative sub-tables. Its UI includes optional highlighting of ARs (as shown in Figure 2).

6) *Experimental results* that measure the sub-table quality and running time of SubTab compared with several baselines. Among the interactive time algorithms, SubTab achieves the best quality by a large gap. Among the algorithms that achieve the best quality, including ones that directly optimize our metric, SubTab is the only one which executes in interactive speed. Experiments with real EDA sessions and user studies show that our quality metrics are compatible with both subjective user experience and objective performance in insight discovery.

Organization: The rest of the paper is organized as follows. We start in Section II by defining our metrics of cell coverage and diversity. Section III gives hardness results for the problem of optimal sub-table selection and greedy algorithms. Section IV presents our sub-table computation method based on table embeddings. Section V describes our experimental study. Related work is discussed in Section VI. We conclude in Section VII.

II. Model and Metrics

In this section, we formalize the notion of "sub-table informativeness" via metrics of cell coverage and diversity and define an optimization problem based on their combination.

A. Model

A relational schema $U = \{u_1, \ldots, u_{|U|}\}$ is a finite set of columns, such that each column u_i allows values from a subset of the global domain $\mathcal{D}_i \subseteq \mathcal{D}$ (e.g., for a binary column, $\mathcal{D}_i = \{0, 1\}$). A relational table over U is a finite set $T \subseteq U \to \mathcal{D}$ of tuples such that $t(u_i) \in \mathcal{D}_i$ is the value of the cell in the row corresponding to a tuple $t \in T$ and the column $u_i \in U$.

Definition II.1 (Sub-table)**.** *Given a table T over schema U, a sub-table T_{sub} is a table over schema $U_{sub} \subseteq U$ such that each tuple $t \in T_{sub}$ is the projection of some tuple $t' \in T$ over the columns of U_{sub}, i.e., for every $u \in U_{sub}$, $t(u) = t'(u)$.*

We next define two standard notions that will be useful in the sequel: binning and ARs.

In a schema U, each column u_i may be *categorical*, namely, $\mathcal{D}_i$ is discrete, e.g., a column of airline names; or *continuous*, namely, $\mathcal{D}_i$ is a continuous range, e.g., a column of flight distance. Moreover, in a table T over U a different distribution

```
from SubTab import subTab
sdf = subTab(flights_df, use_rules=True)
df1 = flights_df[(flights_df['DISTANCE']>2000) & (flights_df['DEPARTURE_DELAY']>10)]
sdf.display(df1)
```

For creating new table vectors, it took 0:00:02.282
For summary creation, it took 0:00:00.45
{'cell_cov': 0.31. 'jaccard':0.72}

Destination Airport	Arrival Time	Wheels On	Airline Delay	Departure Time	Wheels Off	Scheduled Departure	Late Aircraft Delay	Distance	Departure Delay
MCO	1842.0	1832.0	102.0	1050.0	1107.0	855	0.0	2218	115.0
JFK	742.0	736.0	N/A	2333.0	2345.0	2303	N/A	2475	28.0
Rule #2: {(Destination Airport -'SEA'), (Arrival Time -'high'), (Distance -'high')} →{(Airline Delay -'low'), (Departure Delay -'high')}, support-0.11, confidence-0.5, lift-1.5									
SEA	2300.0	2253.0	20.0	1939.0	2012.0	1807	72.0	2402	92.0
SAN	1941.0	1938.0	N/A	1649.0	1702.0	1629	N/A	2588	20.0
BOS	1550.0	1545.0	N/A	752.0	806.0	730	N/A	2611	22.0
SFO	2136.0	2130.0	N/A	1820.0	1834.0	1808	N/A	2704	12.0
JFK	16.0	3.0	0.0	1603.0	1615.0	1535	17.0	2475	28.0

Fig. 2: Informative 10X10 sub-table for a large query result

of values (e.g. uniform or skewed) may occur in each column. *Binning* the column values is a technique commonly used to allow a uniform treatment of columns with different ranges and distributions. Formally,

Definition II.2 (Binning)**.** *Given a table T over schema U, a* binning *function $\mathcal{B}$ maps each column $u_i \in U$ to a finite set of bins $\mathcal{B}_i = \{B_1^i, \ldots, B_{|\mathcal{B}_i|}^i\}$ such that for every $t \in T$, $t(u_i)$ belongs to exactly one bin $B_j^i \in \mathcal{B}_i$.*

Example II.3. *In the flights' dataset, we split the (continuous) range of the DISTANCE column into the bins short, medium, and long-distance. Depending on the column value distribution, we may obtain $B_{long}^{DIST} = [1546.0, 2724.0]$. The CANC. column is binary, hence we can use its categories as bins.*

Next, we recall the notion of *association rules* (ARs), which we use to capture patterns in the data. ARs will be used to measure and compare the quality of sub-tables. Formally,

Definition II.4 (Association rules (ARs) [15])**.** *Given a table T over schema U, an AR R has the form $\{(u_1, v_1), \ldots (u_r, v_r)\} \rightarrow \{(u_{r+1}, v_{r+1}), \ldots (u_{r+p}, v_{r+p})\}$ where each $u_i \in U$ is a column and each $v_i \in \mathcal{D}_i$ is a cell value. Denote by $U_R = \{u_1, \ldots, u_{r+p}\} \subseteq U$ the set of columns used in R. We say R* holds *for a tuple $t \in T$ if $t(u_i) = v_i$ for every $1 \leq i \leq r + p$. Denote by $T_R \subseteq T$ the subset of tuples for which R holds.*

The use of binning may improve mined ARs [15]: given a table T, we can replace each cell value $t(u_i)$ with an identifier of its matching bin B_j^i. Consequently, one may be able to mine ARs that apply to more tuples.

Example II.5. *Using the bins from example II.3, the AR from Example I.2 stating that long flights are likely* not *to be cancelled can be written as:*
AIR_TIME$\in B_{long}^{AT}$,*DISTANCE*$\in B_{long}^{DIST} \nrightarrow$*CANC.*$\in B_0^{CANC}$.

B. Informativeness Metrics

We now develop quality metrics for sub-tables. Out of various possible informativeness metrics, we base our choice on the role of a sub-table as a compact view of the raw data that characterizes the table as a whole. The first type of metric

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:58:35 UTC from IEEE Xplore. Restrictions apply.

Ver: View Discovery in the Wild

Yue Gong*, Zhiru Zhu*, Sainyam Galhotra, Raul Castro Fernandez
Department of Computer Science, The University of Chicago
Email: {yuegong, zhiru, sainyam, raulcf}@uchicago.edu
*Yue Gong and Zhiru Zhu contributed equally

2023 IEEE 39th International Conference on Data Engineering (ICDE) | 979-8-3503-2227-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICDE55515.2023.00045

***Abstract*—We present `Ver`[1], a data discovery system that identifies project-join views over large repositories of tables that do not contain join path information, and even when input queries are inaccurate. `Ver` implements a reference architecture to solve both the technical (scale and search) and human (semantic ambiguity, navigating a large number of results) problems of view discovery. We demonstrate users find the view they want when using `Ver` with a user study and we demonstrate its performance with large-scale end-to-end experiments on real-world datasets containing tens of millions of join paths.**

I. Introduction

The existence of large repositories of data such as those that originate from the combination of disparate databases [1], data lakes [2], open data portals [3], and cloud repositories [4] has the upside of offering opportunities to find valuable data for tasks such as machine learning, reporting, and data analytics. The downside is the resulting data discovery problem: identifying a combination of datasets useful for the downstream task even when these reside in different databases. For example, a machine learning engineer may need a training dataset that requires combining a table in a database with the one sitting on the enterprise data lake. Large volumes of often incomplete and noisy data without any join information, which we call *pathless table collections*, make solving data discovery difficult and thus hampers productivity.

There are several approaches to identify project-join views (PJ-views) over pathless table collections. Discovery systems such as Aurum [5], Goods [6], Auctus [7], Juneau [8], Josie [9], Table-Union [10], D3L [11] and libraries such as LSHEnsemble [12], and Lazo [13] help with identifying datasets that satisfy some relevance criteria whether via keywords [6] or programs [5]. Analysts can then combine the datasets to verify that it satisfies the view they need. Another approach is query-by-example (QBE) interfaces [14], [15] that lets users provide examples of the view they need. These can be adapted, with effort, to run on top of data discovery systems. Whether via keywords, programs, or QBE interfaces, the result of discovery queries over pathless table collections leads to ambiguous results that include semantically distinct results, duplicates or near-duplicates, and different versions of the data. This ambiguity further complicates identifying the right view. More generally, solving view discovery in the wild requires addressing the following challenges:

- **Challenge 1. Noisy Queries.** Users provide queries that represent their best knowledge of the data in the form of keywords, programs, and examples. User-provided input may or may not appear in the table collection and they may be noisy and incorrect.
- **Challenge 2. Noisy Join Paths.** Pathless table collections do not include join paths. Identifying true join paths automatically is impossible. We resort to identifying inclusion dependencies, a proxy to join paths, i.e., a join path implies an inclusion dependency but not the other way around.
- **Challenge 3. Large Number of Join Paths.** Large volumes of data result in a large number of join paths that require efficient data structures to represent and navigate them.
- **Challenge 4. Noisy Result Views.** A large number of join paths implies many views may satisfy a user query. Such result views will be noisy due to ambiguity in the user query and noisy join paths. Concretely, there will be duplicate result views, views that are contained within each other, others that are complementary, and others that will show contradictory values for the same key.
- **Challenge 5. Result View Navigation.** Result views will contain semantically ambiguous results, e.g., views with "work address" and "home address". Only users know the right context so the challenge is to elicit that context and use it to choose the view they need among all result views.

In this paper, we introduce a *reference architecture to identify PJ-views in the wild*. Discovering a PJ-view over pathless table collections requires understanding human preferences and requirements (which we refer to as a *human* problem) and solving a *technical* problem. The architecture divides the larger problem into smaller ones, each of which we tackle with a different component. Reference architectures help conceptualize problems and have been influential in advancing the field. For example, [16], [17] for relational databases and [18], [19], [20], [21] for data integration. The reference architecture we propose tackles the five challenges above. While challenges 1-3 are addressed by existing work, we introduce new techniques to address Challenge 4 and 5 in this paper, and demonstrate them as part of an implementation of the reference architecture. We make the following contributions:

- An end-to-end system, **`Ver`**, that identifies PJ-views among tens of millions of join paths. **`Ver`** implements the reference architecture for QBE-based interfaces. We choose QBE because it permits users to declare the table they would like to find, even when they do not know examples and can only provide attribute names (Section III). **`Ver`** relies on existing work [5] to address Challenges 1-3.

[1]code is available at https://github.com/TheDataStation/ver

2375-026X/23/$31.00 ©2023 IEEE
DOI 10.1109/ICDE55515.2023.00045

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:58:08 UTC from IEEE Xplore. Restrictions apply.

- A **view presentation** component that helps humans to identify a good view among many. The approach uses a novel bandit-based approach to *learn* user-specific discovery preferences, and it uses abduction-based reasoning to quickly narrow down the search space, reducing the number of human interactions (Section IV).
- A **view distillation** component that automatically filters out views by classifying them into **4C categories**: *compatible*, *contained*, *complementary*, and *contradictory*. Besides helping with filtering, these categories produce information used by the view presentation component (Section V).

We conduct an IRB-approved user study (Institutional Review Board) [22] to validate **Ver**'s approach to the human problem. We conduct thorough experiments on queries from open data repositories that lack join paths.

II. Definitions and Problem Statement

$\mathcal{R}(A_1, \ldots, A_m)$ is a relation schema over m attributes, where A_i denotes the ith attribute. A table D comprises a schema $\mathcal{R}(A_1, \ldots, A_m)$ and a set of tuples T where each tuple $t \in T$ is a specific instance of the schema.

In practice, tables do not look like the ideal defined above because they may *lack header information*, have *ambiguous names* and contain *dirty and noisy data*. More formally:

Definition 1 (Noisy structured data): A noisy data D is characterized by an incomplete schema information $\mathcal{R}(A_1, \ldots, A_m)$ where $A_i = \phi$ for missing header values and tuples T such that each tuple $t \in T$ contains at most m values.

In addition, a pathless collection may contain tables with contradictory values, e.g., two census tables with different population counts for the same states of the country. Formally, two tables D_i and D_j contradict if the tables contain different values for the same key. We discuss the detection of contradictions in Section V.

Definition 2 (Pathless table collection): A pathless table collection contains a set of noisy tables $\mathcal{D} = \{D_1, \ldots, D_n\}$ where each D_i is a noisy table and tables D_i, D_j may contain contradictory values.

PJ-example-query. A PJ-query (hereafter called query) contains (possibly noisy) example tuples of the desired output. The examples are a proxy to user's discovery requirements. The quality of examples depends on user's knowledge. Given a query q, there may be many tables that contain the input examples, and many combinations of these may satisfy q, resulting in a large number of candidate PJ-views.

Definition 3 (Noisy query): An input query q is a noisy table consisting of l example tuples, $\chi = \{\chi_1, \ldots, \chi_l\}$ where each χ_i is a noisy tuple denoting example values that are expected to be present in the desired output. The different columns in the examples χ are denoted by $\chi.A_i, \forall i \in \{1, \ldots, \tau\}$, where τ is the number of attributes in the input query.

PJ-views are constructed by joining datasets through keys. We first define a join path and then use it to discuss the effects of noise, followed by a formal problem statement.

Definition 4 (Join path): A join path P is defined as an ordered set of noisy tables $P \equiv \{D_1, \ldots, D_t\}$ such that tables D_j and D_{j+1} join for all $j < t$ via a key column $k \in D_j, D_{j+1}$ forming a chain of join operations.

Joinable datasets can be identified in the presence of key information, which is generally absent in pathless scenarios. A PJ-view V is the dataset formed after materializing the path P followed by projection, i.e., choosing the relevant columns.

Problem 1 (Project-Join view discovery over pathless table collections): Consider a pathless table collection $\mathcal{D}$ and a query q with examples χ. The goal is to construct a minimal candidate set of PJ-views $\mathcal{D}'$ that satisfy the user requirements.

Users may require the PJ-view to contain all examples χ, or any of them, depending on user and application. Formalizing these requirements is outside the scope of this work.

III. A Reference Architecture

Why a Reference Architecture? From a systems engineering perspective, a reference architecture is the materialization of a "divide and conquer" strategy that splits complex engineering problems into smaller parts. Thus, a reference architecture describes a collection of components and their interactions. In making a reference architecture concrete, we state our understanding of the problem and represent it as a concrete artifact that the community can scrutinize and improve.

Design Overview

Finding a PJ-view in the wild requires solving a *human* and a *technical* problem. We present a reference architecture for view discovery that contains components targeting both *human* and *technical* problems. Along with each component, we discuss implementation options, including those made by **Ver**. We also briefly discuss two novel components, View Distillation and View Presentation.

Overview. Algorithm 1 shows pseudocode (human components are highlighted in blue) and Fig. 1 shows **Ver**'s architecture inside a funnel, denoting the gradual reduction of views as data flows downstream. **Ver** builds a discovery index offline.

Discovery Engine and Index Creation (technical). *This component builds indices over pathless table collections: i) a join path index, which can be approximate; ii) retrieval indices over table names, values, attribute names and column similarity. The indices are available online, via the Engine's API to other components. (**Challenge 2**).*

The indices can be built using state-of-the-art methods such as Aurum [5] (that **Ver**'s implementation uses), Auctus [7], JOSIE [9], LSHEnsemble [12]. After the indices are built, users design and submit queries via a VIEW-SPECIFICATION component (line 1).

View Specification (human). Discovery interfaces include spreadsheet-style, keyword search, APIs, natural language, and combinations of these. The reference architecture supports these interfaces via the View-Specification component. For QBE-based interfaces, as implemented by **Ver**, the input is a set of examples, χ, and the output of this stage is a set of example attributes and values. View-Specification

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:58:08 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: **Ver** Design Overview

Input : Pathless table collection $\mathcal{D}$, Discovery Index I
Output: PJ-views $\mathcal{V}$

1 $\chi \leftarrow$ VIEW-SPECIFICATION($\mathcal{D}$)
2 CAND $\leftarrow \phi$
3 **for** $\chi.A_i \in$ COLUMNS(χ) **do**
4 CAND(A_i) $\leftarrow$ COLUMN-SELECTION($\chi.A_i, \mathcal{D}, I$)
5 **if** MODE==*Interactive* **then**
6 CAND(A_i) $\leftarrow$ Query CAND(A_i)
7 CAND $\leftarrow$ CAND $\cup$ CAND(A_i)
8 $\mathcal{V}_{PJ} \leftarrow$ JOIN-GRAPH-SEARCH(CAND, χ)
9 $\mathcal{S} \leftarrow$ VIEW-DISTILLATION($\mathcal{V}_{PJ}$)
10 **if** MODE==*Interactive* **then**
11 $\mathcal{V} \leftarrow$ VIEW-PRESENTATION($\mathcal{V}_{PJ}, \mathcal{S}$)
12 **else**
13 $\mathcal{V} \leftarrow$ Rank $\mathcal{V}_{PJ}$ based on overlap score
14 **return** $\mathcal{V}$

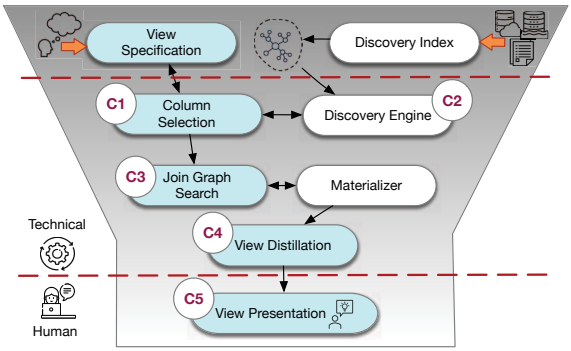


Fig. 1: **Ver** Reference Architecture. The funnel illustrates the progressive reduction of data as it flows downstream.

may interact with users when it detects the provided examples are ambiguous, or to offer examples to choose from.

Next, the COLUMN-SELECTION component selects the subset of tables containing user-provided examples, χ (lines 3-7).

COLUMN SELECTION (TECHNICAL). *This component must be designed to identify relevant data even when the input query is noisy, addressing* ***Challenge 1*** *above. The output of this component is a collection of candidate tables and columns.*

The candidate columns are processed by the JOIN-GRAPH-SEARCH component (line 8) to enumerate and materialize candidate PJ-views (addresses Challenge 3). The candidate view search space reduces, as shown in Fig. 1.

JOIN GRAPH SEARCH (TECHNICAL). *Given a set of candidate tables, an input query, and the discovery index providing join paths, this component identifies all* join graphs *that, when materialized, produce candidate PJ-views. The main goal of this component is to address the large join path space (****Challenge 3****). To materialize candidate PJ-views, the* JOIN-GRAPH-SEARCH *component uses a* MATERIALIZER*, a data processing component with the capacity to execute PJ queries.*

JOIN-GRAPH-SEARCH returns many *candidate views*. Ranking the views is hard because of users' differing search criteria. The VIEW-DISTILLATION component (line 9) summarizes the candidate views, further reducing the view search space as shown in Fig. 1.

VIEW DISTILLATION (TECHNICAL). *This component computes* categories *from the candidate PJ-views that include redundancy and containment in the views, as well as opportunities for unioning views and more. Some categories can be used to distill/summarize the views (****Challenge 4****). Others are shared with the downstream component.*

VIEW-PRESENTATION receives the distilled views. It can *rank* the views and return top-1 (for a full automated mode) or it can leverage the categories computed by VIEW-DISTILLATION to help users find the right data (lines 10-13).

VIEW PRESENTATION (HUMAN). VIEW-PRESENTATION *uses different question interfaces to elicit information from users via data questions. The questions are designed to narrow down the space until users find the desired view (****Challenge 5****). The component chooses what questions to ask, sequentially, using a bandit-based approach.*

Next, we present VIEW-PRESENTATION (Section IV), VIEW-DISTILLATION (Section V).

IV. VIEW PRESENTATION

Ideally, a query results in one PJ-view. In practice, ambiguity, redundancy, erroneous join paths, and large table repositories mean there may be hundreds of result views. **Ver** uses novel techniques to reduce the number of views automatically (Section V), but there is a limit to automation. Semantic ambiguity requires involving users to obtain the final view. The VIEW-PRESENTATION component analyzes the views and generates questions that, when answered, help rank and select views. For example, this component will ask a user if they want "home address" or "work address" in their output when it detects both in the views. By asking questions, users learn more about the schemas and datasets available, and thus refine their preferences and discovery needs. A key challenge is that different users may be able to respond to different questions and that their preferences evolve as they interact with the system. **Ver**'s VIEW-PRESENTATION component is based on two design principles that cater to varied user needs.

• **Mixed-Initiative Interface Design:** Lack of knowledge about available datasets inhibits users from effectively querying the identified set of views. However, users can answer questions about their desired view. Each user has a different understanding of the requirements and finds different interfaces to be more appropriate depending on that. Therefore, users may need different interface designs to answer questions. For example, some users could recognize phone number from the area code, while others might look for a pattern across values. Because different users will be able to answer different questions, **Ver** supports different question interfaces, each asking a different question type. This is motivated by previous mixed-initiative designs [23], [24]. Besides, **Ver** learns which interface to offer to a user according to their previous actions.

• **Adapt to evolving users' knowledge:** Users' understanding of their discovery need evolves as they interact with questions and learn about the schemas and data contained in the repository. **Ver** is designed so users can change their

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:58:08 UTC from IEEE Xplore. Restrictions apply.



Cross Modal Data Discovery over Structured and Unstructured Data Lakes

Mohamed Y. Eltabakh
Qatar Computing Research Institute (QCRI)
Doha, Qatar
meltabakh@hbku.edu.qa

Mayuresh Kunjir
Amazon Web Services
Berlin, Germany
mkunjir@amazon.de

Ahmed K. Elmagarmid
Qatar Computing Research Institute (QCRI)
Doha, Qatar
aelmagarmid@hbku.edu.qa

Mohammad Shahmeer Ahmad
Qatar Computing Research Institute (QCRI)
Doha, Qatar
mohammadshahmeerah@hbku.edu.qa

ABSTRACT

Organizations are collecting increasingly large amounts of data for data-driven decision making. These data are often dumped into a centralized repository, e.g., a data lake, consisting of thousands of structured and unstructured datasets. Perversely, such mixture makes the problem of *discovering* tables or documents that are relevant to a user's query very challenging. Despite the recent efforts in *data discovery*, the problem remains widely open especially in the two fronts of (1) discovering relationships and relatedness across structured and unstructured datasets–where existing techniques suffer from either scalability, being customized for a specific problem type (e.g., entity matching or data integration), or demolishing the structural properties on its way, and (2) developing a holistic system for integrating various similarity measurements and sketches in an effective way to boost the discovery accuracy.

In this paper, we propose a new data discovery system, named CMDL, for addressing these two limitations. CMDL supports the data discovery process over both structured and unstructured data while retaining the structural properties of tables. As a result, CMDL is the only system to date that empowers end-users to seamlessly pipeline the discovery tasks across the two modalities. We propose a novel multi-modal embedding representation that captures the similarities between text documents and tabular columns. The model training relies on labeled datasets generated though *weak supervision*, and thus the system is domain agnostic and easily generalizable. We evaluate CMDL on three real-world data lakes with diverse applications and show that our system is significantly more effective for cross-modality discovery compared to the search-based baseline techniques. Moreover, CMDL is more accurate and robust to different data types and distributions compared to the state-of-the-art systems that are limited to only the structured datasets.

PVLDB Reference Format:
Mohamed Y. Eltabakh, Mayuresh Kunjir, Ahmed K. Elmagarmid, and Mohammad Shahmeer Ahmad. Cross Modal Data Discovery over Structured and Unstructured Data Lakes. PVLDB, 16(11): 3377 - 3390, 2023.
doi:10.14778/3611479.3611533

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 11 ISSN 2150-8097.
doi:10.14778/3611479.3611533

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/qcri/CMDL

1 INTRODUCTION

The democratization of data science has resulted in enterprises drowning in their own collected data. Large enterprises routinely have a huge collection of structured and unstructured data in their data repositories, referred to as *data lakes*. Performing *data discovery* where the goal is to find relevant elements (e.g., tables or documents) for a given user's query or an analytical task from such massive and unorganized data lakes is a major challenge faced by data scientists [3, 17, 39, 42, 56]. Data discovery is one of the initial steps of data preparation that plays an important role in various downstream data processing tasks including data exploration, data integration, and data enrichment [28, 32, 33, 55, 62].

Data discovery is emerging as a practical and core problem to modern enterprises as exemplified by the recent interest from both academia and industry [1, 2, 10, 22, 29, 40]. Nevertheless, it remains a challenging problem due to several factors. First, data is collected and published by disparate sources, and thus do not conform to a common format or structure. A vast amount of this data is in the form of unstructured text documents, e.g., research articles, social media posts, emails, news reports, etc. [12, 19]. As a result, both data modalities (structured and unstructured) are equally critical, and any system that overlooks either of them is lame. The second factor is the lack of consistent and sufficient data description and metadata. Therefore, a robust data discovery system must reason about and establish relationships of relatedness using multiple and diverse similarity signals, where each could be possibly weak. Lastly, a data discovery system needs to scale well to very large volumes of data as more organizations and governments are publishing more data for transparency [54].

The majority of existing work limits the discovery process to only the structured portion of data lakes, e.g., [10, 22, 24, 25, 59]. And thus, these systems do not extend to unstructured data and hit the wall of the first challenge mentioned above. In contrast, there are recent efforts trying to extend the discovery process across both structured and unstructured data sets, however they inherit several limitations. For example, some of these systems shift the focus entirely to the unstructured domain by transforming the

structured data to unstructured documents [12, 13, 19]. This way, they demolish the structural properties and cut the way towards having a holistic discovery system across both modalities. Other systems move the two modalities to a third common space where everything is encoded as a set of subject-predicate-object triplets, e.g., [23]. Again, they lose the ability for supporting discoveries such as table joinability and unioinability in addition to encountering high overheads that hinder its scalability.

The goal of this work is to develop a more holistic data discovery system that can: (1) bring large unstructured data into the framework of data discovery, (2) enable analysts to intermix their structured and unstructured discovery tasks in a single pipeline seamlessly, and (3) be robust through novel integration of various content-based and metadata-based similarity signals–this is especially required because unstructured data inherit very different characteristics compared to structured tables.

Motivation Example (Pharmaceutical Usecase): *Pharmaceutical databases such as DrugBank consist of an exhaustive listing of drugs, enzymes, and genes along with their interactions with other drugs or related proteins [4]. Domain experts periodically curate such databases with findings from research literature, MedLine reports [6], FDA MedWatch reports [5], and other sources. Assume our data lake encompasses all these rich data sets, and the analysts are exploring the data in the wild, i.e., without any prior knowledge of the available data sources. Now, an analyst studying enzyme* "Thymidylate Synthase" *would like at first to find documents related to this enzyme (Q1 in Figure 1). The system returns such documents as depicted in the left side of the figure. Then, she poses the second question (Q2) trying to find curated tables in the database related to the 1st document (or sub-document by highlighting sentences of interest). The system should provide table* "Enzyme_Targets" *as an answer. Then, progressing forward with her exploration, she finds another returned document that talks about drug* "Pemetrexed" *and its relation to enzyme* "Thymidylate Synthase". *As such, she poses Q3 to potentially find more tables in the database supporting this relationship. The system should provide the two tables* "Enzyme_Targets" *and* "Drugs" *as high likelihood candidates to Q3. One step further, the analyst would like to know if there are more relationships in the database around this drug. And thus, she raises Q4 and the system provides two potentially joinable tables (i.e.,* "Enzyme_Targets" *and* "Drug_Interactions"*). The former table seems central to this exploration as it has been reached from multiple paths, and hence the analyst issues Q5 to find more related and unionable tables.*

Evidently, the kind of relationships, similarity measures, and sketches needed to be maintained and exploited to answer Question Q1 (within the unstructured data modality) are very different from those needed to answer Questions Q2 and Q3 (across modalities), and also very distinct from those needed to answer Question Q4 (within the structured data modality). To the best of our knowledge, there is no single holistic system that can support such data discovery chain as highlighted above.

Solution overview: We propose a data discovery system, called CMDL[1], that enables the data discovery tasks demonstrated in the motivation example over a data lake of structured and unstructured data sets. To support cross-modal questions such as Q2 and Q3 (from Figure 1), we introduce a novel embedding-based joint representation that brings both unstructured documents and relational tables, more specifically the individual columns in each table, into a common vector space. We propose several optimizations over the model's loss function to achieve scalability and avoid biases as will be explained later. The model that creates these joint representations relies on supervised learning and requires the presence of a labeled training data set–which may not exist in the first place. Therefore, CMDL is equipped with a weakly-supervised training process that integrates multiple similarity signals over documents and columns to construct a labeled training data set. As such CMDL does not require a pre-existing labeled datasets, and hence, easily generalizable to different applications.

To support other types of data discovery questions such as Q1, Q4, and Q5 (from Figure 1), CMDL utilizes several types of signatures and sketches on both the actual data content and the contextual metadata. For example, for structured data, schema information and tables names are types of metadata. Similarly, for unstructured documents, the documents' names and their sources are the metadata. Then, we build appropriate indexes using state-of-the-art techniques on them, e.g., BM25 elastic search index [49] for keyword search and Locality Sensitive Hashing (LSH) index [61] for the set containment operations. While prior data discovery solutions are also based on similar sketches and indexes, e.g., [10, 13, 22, 24, 25], CMDL goes one step beyond by first leveraging them as similarity signals for the weak-supervised training process mentioned above, and second integrating these complementary elements into a single unified discovery system.

The various relationships discovered by the system form an Enterprise Knowledge Graph (EKG) on which a comprehensive query interface is developed. It extends the *SRQL* query engine from Aurum [22] with APIs and retrieval methods specific to CMDL, e.g., for supporting cross-modality discovery tasks.

In summary, we make the following contributions:

(1) Introducing a holistic data discovery system that treats both structured tabular data and unstructured documents as first-class citizens. Within a single discovery pipeline, end-users can seamlessly formulate discovery questions ranging from simple keyword search over either modality to complex discoveries spanning both modalities.
(2) Proposing a novel embedding-based joint representation for multi-modal data sets. The model training relies on weak supervision by combining various types of similarity signals. As such, the system can be adopted even under the absence of labeled data.
(3) Integrating existing sketches with a few alterations to improve the discovery accuracy even for the traditional joinability and unionability tasks by up to 30%.
(4) Evaluating CMDL on three real-world data lakes ranging from drug discovery to ML data augmentation. CMDL is not only more effective in discovering the complex relationships across the unstructured documents and the structured tables, but also more robust for the traditional discovery tasks (e.g., joinability and unionability) over the structured data compared to the state-of-the-art systems.

[1] CMDL–Cross Modal Data Discovery over Structured and Unstructured Data Lakes–is pronounced as kam·dl.

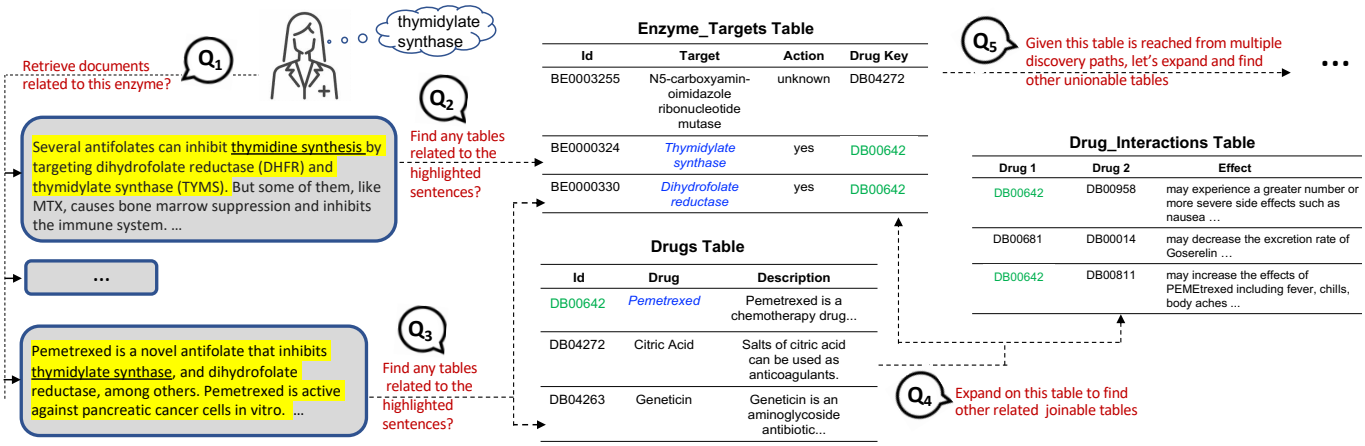


Figure 1: An example data discovery scenario on a lake of structured and unstructured pharmaceutical data.

2 CMDL OVERVIEW

In order to support data discovery pipelines similar to that presented in Figure 1, we need four building block elements. First, the notion of *discoverable elements* (*DE*s) that are the output from the discovery queries. Second, semantics for identifying the relationship between the *DE*s. Third, algorithms and data structures to efficiently discover and store the *DE*s and their relationships. Finally, an expressive query language for expressing the discovery tasks.

2.1 Discoverable Elements and Relationships

CMDL Discoverable Elements. We borrow the concept of Discoverable Elements from Aurum [22], which is the abstract unit of data discovery. In CMDL, we treat each **column** and each **document** as the basic units of discovery for the structured and unstructured data sets, respectively. Based on columns, each **table** in the data lake is also treated as a higher-order DE. The intuition behind the column-level granularity is multifold. (1) Working at this granularity allows the system to scale well to massive datasets, (2) The column-level discovery is already the basic unit of discovery over structured data (e.g., joinability and unionability) [10, 26, 59], and thus it would work seamlessly in the entire CMDL framework, and (3) Columns typically have coherent semantics, e.g., drug names or city names, that embeddings can easily capture (unlike tuples which could have many attributes with very diverse semantics).

For simplicity, we assume each unstructured document is short, i.e., several sentences as illustrated in Figure 1. From a use case and end-user point of view, this assumption is realistic, because otherwise starting a discovery search with a large document, e.g., several pages, is very unfocused and will most probably lead to countless relationships of no interest. Moreover, from the system's backend point of view, this assumption is not restrictive because large documents can still be uploaded as physical units, however, CMDL will (logically) break each document into smaller DE units, e.g., paragraphs, for the discovery purpose.

CMDL DE Relationship Types. In this paper, we emphasize three key relationship types between DEs as motivated by our example in Section 1.

Doc*$_{to}$ *Table *(From Document to Tables).* A Table T with column set $\mathbf{A}$ is related to a text document D iff $\exists A_i \in \mathbf{A}$ s.t. D and A_i are related via overlapping values, semantic similarity, or metadata similarity, each with a relatedness score. The combined scores of the links $D \rightarrow A_i$ represent the strength of the relationship.

Table*$_J$ *Table *(Joinable Tables).* Table T with column set $\mathbf{A}$ is joinable to Table T' with column set $\mathbf{A'}$ iff $\exists A_i \in \mathbf{A}$ and some $A'_j \in \mathbf{A'}$ s.t.: (a) A_i and A'_j have value overlap suggesting syntactic join, or (b) A_i and A'_j have semantic overlap suggesting semantic join.

Table*$_U$ *Table *(Unionable Tables).* Table T with column set $\mathbf{A}$ is unionable to Table T' with column set $\mathbf{A'}$ iff a one-to-one mapping $H : \mathbf{A} \Rightarrow \mathbf{A'}$ exists wherein $\exists h \in H$ s.t. the column pair given by h exhibits name, value, or semantic similarity. The combined similarity score of the column mapping gives the strength of the relation.

2.2 CMDL Architecture

Figure 2 illustrates the overall system's architecture. From left to right, both the text documents and structured tables go through a preprocessing phase. More specifically, each text document goes through an NLP-based pipeline for transformation, which ultimately results in converting a document into a column format consisting of a bag of words. In contrast, each column in the tabular data goes through heuristic-based tagging that labels the column based on the potential relationships and discovery tasks it will participate in. The next step is the *profiler*, which creates various types of sketches and statistics to be leveraged later in the discovery tasks for capturing syntactic and semantic relationships. These sketches then go to the *indexing framework* for building the appropriate indexes according to the individual sketch type (e.g., for metadata and content data, an elastic search index is constructed, whereas for min hash statistics an LSHEnsemble index is constructed).

The sketches created so far by the *profiler* are independent of and agnostic to any relationships between the text documents and the tabular columns. The *Multi-Modal Joint Representation* module is responsible for creating joint sketches that encode such relationships, i.e., related document-column pairs should have similar

LIFTus: An Adaptive Multi-aspect Column Representation Learning for Table Union Search

Ermu Qiu[†], Jun Gao[†*], Yaofeng Tu[‡], and Jingru Yang[§*]
[†]Key Laboratory of High Confidence Software Technologies, CS, Peking University, China [‡]ZTE Corporation,
[§]National Key Laboratory of Data Space Technology and System
qem@stu.pku.edu.cn, gaojun@pku.edu.cn, tu.yaofeng@zte.com.cn, okiyang@pku.edu.cn

***Abstract*—Table union search (TUS) represents a fundamental operation in data lakes to find tables unionable to the given one. Recent approaches to TUS mainly learn column representations for searching by introducing Pre-trained Language Models (PLMs), especially on columns with linguistic data. However, a significant amount of non-linguistic data, notably represented by domain-specific strings and numerical data in the data lake, are still under-explored in the existing methods. To address this issue, we propose LIFTus, an adaptive multi-aspect column representation for table unionable search, where aspect refers to a concept more flexible than data types, so that a single column can exhibit multiple aspects simultaneously. LIFTus aims at combining different aspects of a column (including both linguistic and non-linguistic aspects) to promote the effectiveness and generalization of TUS in a self-supervised manner. Specifically, besides employing PLMs to extract the linguistic aspects from an individual column, LIFTus trains a pattern encoder to learn possible character-level sequential patterns for the column, and builds a number encoder to capture numerical aspects of the column, including the distribution and magnitude features. LIFTus further utilizes a hierarchical cross-attention aided by aspect-relevant statistics to combine these aspects adaptively in producing the final column representations, which are indexed by vector retrieval techniques to achieve efficient search. Extensive experimental results demonstrate that LIFTus has outperformed the current state-of-the-art methods in terms of effectiveness, and achieved much better generalization capability to support unseen data.**

***Index Terms*—Table Union Search, Data Lake, Aspect, Non-Linguistic Column, Generalization**

I. Introduction

With the rapid growth of data from various sources, data lake [1]–[3] becomes an attractive infrastructure to accommodate flexible types of data in their raw form directly. Unlike data warehouses [4], data lakes allow storing data including relational tables (sometimes with metadata unavailable) and other types without transforming into a predefined schema [5]. Thus, a fundamental step for an analytic operation in data lake is to locate the relevant data. TUS is such an operation to identify a set of unionable tables in a data lake that share similar columns with the given query Table [6].

TUS methods attempt to achieve effectiveness, efficiency and generalization. Data instances collected in tables within a data lake often exhibit considerable heterogeneity, encompassing linguistic data (*e.g.* words, long texts, dates), non-linguistic data (*e.g.* domain-specific strings, numerical values), and even their combinations [2], [7], [8]. It is challenging to find the desired unionable target tables effectively. In addition, the column representation learning, a key step in the TUS, needs to capture features of flexible data instances and utilize external knowledge, indicating high time consumption. Moreover, TUS methods should be adaptive to new and evolving data, as the volume of data in the lake grows rapidly. Consequently, a generalized model for column representation is highly desirable.

In the context of the TUS problem, unionability refers to the potential for attributes or tables to be unioned based on shared domain features [9]. Early approaches to TUS have focused on designing rules to measure the unionability between two tables [2], [9], [10], but face challenges in capturing latent linguistic information. With the development of PLMs such as BERT [11] and its variants [12], [13], recent works [14]–[17] have employed these models into TUS, as the data in the lake have flexible format including the textual features [18]. Besides PLMs, knowledge bases (KBs) [19], [20] can also provide information for the determination of unionability [7]. Some studies further notice the roles of the inter-column relationships and compose a token sequence by sampling data from different columns into the BERT-like models, thereby learning the context-aware column representations [14], [15]. The learned column representations are then fed into the vector index for fast similarity search [14]. Some studies calculate pairwise similarities of columns to build a semantic graph for TUS [19], achieving promising performance but at the cost of additional preprocessing and online search time [7].

While PLM-based methods become mainstream in TUS, their preference for linguistic data imposes potential limitations on the performances, especially in the context of the relational tables. We illustrate these issues in Fig. 1. Table A and Table B can be unionable as they both pertain to the entities of washing machines manufactured by the same company and their corresponding columns are unionable, while Table C serves as an example that is prone to model misjudgment. We can see that three tables possess linguistic data in their *Location* and *Description* columns, and Table C also includes the *Color* column as part of the linguistic data. Besides, Fig. 1 illustrates the widespread presence of non-linguistic data in the data lake, including but not limited to identifiers (*e.g. Product Code* column) and numerical data (*e.g. ID* column, *Capacity* column, *Price* column, etc.). For

* Corresponding authors

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00165

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00165

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:40:17 UTC from IEEE Xplore. Restrictions apply.

example, the strings from the *Product Code* and *Function Abbr* columns (*e.g.* "SDL/JHX/BP/ZH" and "MB65V35E") are not tokens in natural language but highly domain-specific, as they are codes designed by washing machine manufacturers.

Table A

ID	Product Code	Capacity(L)	Function Abbr	Location	Price	Description
20001	EMS75B37mate6	7.5	SDL/JHX/BP/ZH	WalMart	3100	This is a top-loading washing machine that deli...
20002	ES100B35Mate5	10	DDL/JHX/DP	WalMart	2950	Featuring a 10L capacity, it can handle large loa...
20003	ES90B26Mate6	9	SDL/BP/FCR	WalMart	2200	The machine's smart sensors adjust the water...

Table B

ID	Product Code	Capacity(L)	Function Abbr	Location	Price	Description
20981	EMS65B26Mate6	6.5	DDL/DP/DLCJ	Carrefour	1850	With a variety of wash cycles, including a quick...
20982	EB80B37Mate5	8	DDL/DP	Carrefour	2100	Its durable stainless steel drum is gentle on fab...
20983	ES95B36PLUS5	9.5	SDL/BP/FCR/ZH	Carrefour	3500	Equip with a self-cleaning function, eliminating...

Table C

ID	Product Code	Power(W)	Color	Location	Price	Description
301	MB65V35E	340	White	WalMart	1900	The machine's sleek design and compact size m...
302	MB75V35E	400	Silver	WalMart	2150	For those with allergies, the UltraClean 3000's...
303	MB65V55E	360	Black	WalMart	2350	With a quiet operation and a 5-year warranty, t...

Fig. 1. An example of data lake. In this data lake, Table A and Table B are unionable with each other, while Table C is easily misjudged as being unionable with Table A. This misjudgment often stems from an inadequate understanding of non-linguistic columns (*e.g. ID*, *Product Code*, *Function Abbr*), which are widely distributed across the data lake and play a crucial role in determining the unionability of tables. The metadata present here is for easy understanding, but not used in the following processing.

We illustrate the issues faced by the existing PLM-based methods. First, the domain-specific strings, represented by commonly used identifiers, fall outside the typical token set in the PLMs [11]–[13], and cannot be captured by PLMs effectively. Taking Fig. 1 as an example, as the token sets in the PLMs do not cover the values in *Product Code* column and *Function Abbr*, PLMs handle them using the frequent sub-strings, like BPE [21], computed from the large common corpus, which may be different from the key sub-parts in these domain-specific strings. We can see that the similarity between "EMS100B37mate6" and "EMS65B26Mate6" should be significantly greater than that between "EMS100B37mate6" and "MB65V35E". However, PLMs may decompose the string into different parts (*e.g.* "E", "MS1"), which damages the understanding of the domain-specific string. It is imperative to employ character-level sequential modeling for domain-specific strings to capture their similarities.

Second, the accurate perception of the numerical values to match columns is an open problem [22]. It is a live issue for PLMs to adequately understand numerical values [23]. PLMs' token sets can not cover infinite numerical values distributed in continuous space. Thus, the PLM-based methods [14]–[16] struggle to fully understand real numbers, which leads to a loss in the similarity computation over arbitrary numbers in real-life applications. The set-coverage-based methods [9] may yield Jaccard similarity scores nearly zero (*e.g.* {7.5, 10, 9} in *Capacity* column in Table A, {6.5, 8, 9.5} in *Capacity* column in Table B in Fig. 1), despite the fact that these numbers differ only slightly. We also notice that there has been research on encoding numerical types in tables [24]. However, there remains a significant gap between encoding individual numbers and encoding entire numerical columns in TUS, where the latter needs to capture the crucial criteria to determine the similarity among columns, including distribution and magnitude features.

Third, columns may include combined features, and relying solely on data-type-specific approaches may be too coarse to accurately capture column features. As we can see from Fig. 1, some columns in a data lake are rich in semantic information, characterized by a high proportion of natural language words, such as the *Description* columns, which rely more on external corpora or PLMs. In contrast, the *Function Abbr* columns in Table A and Table B should guide the model to prioritize character-level similarity during unionable search. Additionally, some columns are predominantly numerical, such as the *ID* and *Price* columns across the three tables. In such a case, the determination should focus on the distribution and magnitude features of the numbers. Moreover, certain columns contain a mixture of numerical data and non-language strings, such as the *Product Code* columns in the three tables. We can see that even if we can obtain the data type for the column, the data-type-specific methods are not sufficient to capture the flexible features inside the column.

To overcome the aforementioned challenges and issues of the existing methods, we make the following design choices. (1) We introduce a notation of *column aspect*, which is finer than column data type, as each column has only one data type but with multiple aspects. Thus, instead of invoking one method after determining the column data type, we apply multiple aspect extractors on one column simultaneously, so as to handle the different forms of the same typed column or mixed-typed data. We also expect that the full exploration of aspects in an individual column can enhance the model's generalization to unseen data. (2) We do not solely rely on PLM to handle all kinds of columns, but introduce effective methods to capture distinct features in columns, including the linguistic data, domain-specific strings, and numerical values. For aggregating these aspects, we design a learning method rather than heuristic rules to enable robust adaptation to scenarios involving tables with diverse information. (3) We mainly study the exploration of features in one column, and leave the inter-column relationships into future work, since different features inside one single column, we believe, are still underexplored. (4) The method will yield the column embedding, instead of the pairwise similarity model in order to leverage the existing vector index like HNSW (Hierarchical Navigable Small World) [25] to improve the efficiency of online search.

This paper proposes **LIFTus**, an adaptive mu**l**ti-aspect column representat**i**on **f**or **t**able **u**nionable **s**earch. The contributions are as follows:

- We design methods to capture the key non-linguistic aspects in LIFTus, including a pattern encoder for domain-specific strings to infer similarities between strings, and a number encoder to convert the numerical aspect into embedding that preserves the distribution and magnitude features.
- We devise a hierarchical cross-attention integrator, to adaptively combine different aspects of columns, in which

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:40:17 UTC from IEEE Xplore. Restrictions apply.

the statistics indicating the weights of different aspects are extracted and fed into a hierarchical cross-attention framework to produce the column representations.
- We conduct experiments on four real-world datasets, and the results indicate that LIFTus has achieved a 3.2% improvement in effectiveness on average (**4.7% improvement** for 2 large datasets on average) over the second to the best model at a similar offline preprocessing time cost. The results also show much better generalization capability of LIFTus, which is trained on a small dataset but can be applied to a large dataset, thereby saving training time. For example, after training on SANTOS small only, LIFTus can achieve even better performance on SANTOS large than Starmie, which takes **5.6X** more time training on SANTOS large directly. The code is available on https://github.com/MsiQue/LIFTus.

II. Related Work

In this section, we first review the advances of approaches to TUS, then introduce recent progress in a more generalized table representation learning, and finally discuss the methods used to process non-linguistic data within tables.

A. Table Union Search

TUS is a frontier research topic and has witnessed rapid advancements recently [2], [7], [10], [14]. We review the existing methods from various design choices, ranging from the measurement of the unionable table, different forms of external knowledge, the consideration of inter-column relationships, and training strategies.

There are explicit and implicit measurements for the unionable table. Nargesian et al. [9] have introduced the concept of TUS, and proposed three aspects of unionability. Bogatu et al. [10] have introduced D^3L, extending the determination of unionability to a five-dimensional model. Other recent works, including SANTOS [7] and Starmie [14], learn the unionable tables in an implicit way, in which the previous factors are constructed as a column relation graph or fed into PLM.

Different external knowledge can be incorporated into the TUS. The early works [2], [10] mainly utilize external knowledge bases [19], [20] or word embeddings [26]. SANTOS [7] leverages both existing knowledge bases [19] and a synthesized one, generated by exploiting the inherent knowledge within the data lake. Starmie [14] relied on PLMs [11]–[13], which are based on the transformer architecture [27] to learn the attention weights between token pairs.

The inter-column relationships are considered by SANTOS [7] and Starmie [14]. They assume that the neighboring columns provide a context for an individual column. Specifically, SANTOS acquires semantic relationships using the knowledge base, and Starmie vectorizes each column by concatenating tokens sampled from neighboring columns and feeding them into a language model.

The model can be trained in a supervised or self-supervised fashion. D^3L [10] treats the determination of unionable tables as a binary classification problem in a supervised manner. To reduce the high cost of manual labeling, Watchog [28] and Starmie [14] have shifted to the self-supervised fashion, allowing researchers to expand the column encoder for various downstream tasks.

The existing methods bring the following inspiration to LIFTus. LIFTus follows a self-supervised fashion to train a column encoder for TUS with sufficient factors considered. In addition, LIFTus chooses the PLM to handle the linguistic data as a part of producing column representations. LIFTus puts off the inter-column relationship to the future work, as we observe that the column features are not fully explored in the current data preprocessing.

B. Table Representation Learning

TUS is a specific downstream task of the generalized table representation learning, which can support other tasks like column classification, data integration, etc. We can roughly categorize the existing methods into basic neural network based approaches [29]–[32], graph representation learning based approaches [33]–[35], and pretrained model based approaches [22], [36]–[40]. Among them, Sherlock [29] employs a multi-input deep neural network to integrate various feature categories and learns column embeddings in a supervised manner. SATO [30] extends Sherlock [29] by incorporating inter-column information and leverages Latent Dirichlet Allocation (LDA) [41]. EmbDI [33] and HierGAT [34] learn embeddings for both columns and rows by utilizing graph-based random walk and attention mechanism, respectively. Pretrained model based approaches can mainly be divided into two categories. One is to use tables as a corpus to pre-train the models [38]–[40], and the other is to directly utilize PLMs to understand tables for solving downstream tasks [22], [28], [36], [37].

C. Non-linguistic Column Understanding

Domain-specific String. Existing works handle the domain-specific strings by extracting their formatting rules or regular expressions. Cornet [42], [43] leverages user examples to automatically learn formatting rules for a specific column. Bogatu et al. [10] establish a collection of rules to transform columns into a set of regular expressions and measure the similarity of two columns using the Jaccard similarity. Sherlock [29] focuses on the statistics of characters in strings (*e.g.* mean, max, sum, etc.), while the order of characters within a string is not extensively explored in computing similarity between strings. In the context of TUS, we actually do not require explicit regular expressions for strings, and an implicit model is sufficient for the similarity among strings.

Numerical Data. Some works measure the correlation between two numerical columns using statistics. STEER [44] proposes a steered-labeling approach to handle numerical columns effectively with Earth Mover's Distance for similarity assessment, which leverages non-numerical column labels to guide context-aware clustering. Bogatu et al. [10] measure the correlation by computing the Kolmogorov-Smirnov (KS)

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:40:17 UTC from IEEE Xplore. Restrictions apply.



Semantics-aware Dataset Discovery from Data Lakes with Contextualized Column-based Representation Learning

Grace Fan
Northeastern University
United States
fan.gr@northeastern.edu

Jin Wang
Megagon Labs
United States
jin@megagon.ai

Yuliang Li
Megagon Labs
United States
yuliang@megagon.ai

Dan Zhang
Megagon Labs
United States
dan_z@megagon.ai

Renée J. Miller
Northeastern University
United States
miller@northeastern.edu

ABSTRACT

Dataset discovery from data lakes is essential in many real application scenarios. In this paper, we propose Starmie, an end-to-end framework for dataset discovery from data lakes (with table union search as the main use case). Our proposed framework features a contrastive learning method to train column encoders from pre-trained language models in a fully unsupervised manner. The column encoder of Starmie captures the rich contextual semantic information within tables by leveraging a contrastive multi-column pre-training strategy. We utilize the cosine similarity between column embedding vectors as the column unionability score and propose a filter-and-verification framework that allows exploring a variety of design choices to compute the unionability score between two tables accordingly. Empirical results on real table benchmarks show that Starmie outperforms the best-known solutions in the effectiveness of table union search by 6.8 in MAP and recall. Moreover, Starmie is the first to employ the HNSW (Hierarchical Navigable Small World) index to accelerate query processing of table union search which provides a 3,000X performance gain over the linear scan baseline and a 400X performance gain over an LSH index (the state-of-the-art solution for data lake indexing).

PVLDB Reference Format:
Grace Fan, Jin Wang, Yuliang Li, Dan Zhang, and Renée J. Miller. Semantics-aware Dataset Discovery from Data Lakes with Contextualized Column-based Representation Learning. PVLDB, 16(7): 1726 - 1739, 2023.
doi:10.14778/3587136.3587146

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/megagonlabs/starmie.

1 INTRODUCTION

The growing number of open datasets from governments, academic institutions, and companies have brought new opportunities for innovation, economic growth, and societal benefits. To integrate and analyze such datasets, researchers in both academia and industry have built a number of dataset search engines to support the application of dataset discovery [3, 7, 17, 19, 33, 40, 45]. One popular example is Google's dataset search [3] which provides keyword search on the metadata. However, for open datasets, simple keyword search might suffer from data quality issues of incomplete and inconsistent metadata across different datasets and publishers [1, 16, 41, 42]. Thus it is essential to support *table search* over open datasets, and more generally data lake tables (including private enterprise data lakes), to boost dataset discovery applications, such as finding related tables, domain discovery, and column clustering.

Finding related tables from data lakes [11, 25, 39, 46, 57] has a wide spectrum of real application scenarios. There are two sub-tasks of finding related tables, namely table union search and joinable table search. In this paper, we mainly focus on the problem of *table union search*, which has been recognized as a crucial task in dataset discovery from data lakes [2, 24, 39, 41, 42, 57, 61]. Given a query table and a collection of data lake tables, table union search aims to find all tables that are unionable with the query table. To determine whether two tables are unionable, existing solutions first identify all pairs of unionable columns from the two tables based on column representations, such as bag of tokens or bag of word embeddings. They then devise some mechanism to aggregate the column-level results to compute the table unionability score.

State-of-the-art: Early work on finding unionable tables used table clustering followed by simple syntactic measures such as the difference in column mean string length and cosine similarities to determine if two tables are unionable [4]. Table union search [42] improved on this by applying a rich collection of column representations including syntactic, semantic (leveraging ontologies), and natural language (based on word-embeddings) column representations. Two important innovations of this work were the modeling of data lake context to create an *ensemble unionability score* which models the surprisingness of a score given the score distributions within a data lake and the use of LSH indices to make table union search fast over large data lakes [42]. More recently D^3L [2] added additional column representations based on regular expression matching and SANTOS [24] added to the column representations, representations of binary relationships. In parallel to these search-based approaches, the mighty hammer of deep learning has been applied to the problem of column matching (determining the semantic type of a column) [22, 56]. Since these

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 7 ISSN 2150-8097.
doi:10.14778/3587136.3587146

Table A:

Name	Mode of Travel	Purpose	Destination	Day	Month	Year	Expense
Philip Duffy	Air	Regional Meeting	London	10	April	2019	189.06
Jeremy Oppenheim	Taxi	Exchange Visit	Ottawa	30	Jul	2019	8.08
Mark Sedwill	Air	Evening Meal	Bristol	02	September	2019	50

Table B:

Name	Date	Destination	Purpose
Clark	23/07	France	Discuss EU
Gyimah	03/09	Belgium	Build Relations
Harrington	05/08	China	Discuss Productivity

Table C:

Bird Name	Scientific Name	Date	Location
Pine Siskin	Carduelis Pinus	2019	Ottawa
American Robin	Turdus migratorius	2019	Ottawa
Northern Flicker	Colaptes auratus	2019	London

Figure 1: An example of table union search on Open Data.

approaches are supervised, they can only be applied to finding a limited set of semantic types (78 in their experiments), and while not a general solution for unionability in data lakes, they can be used in an offline fashion to find unionable tables containing the types on which they are trained.

However, there are still plenty of opportunities to further improve the performance of table union search. One important issue is to learn sufficient contextual information between columns in tables so as to determine the unionability. This point can be illustrated in the following motivation example.

Example 1.1. Figure 1 shows an example of finding unionable tables. Given the query Table A, existing approaches first find unionable columns. In this example, the column `Destination` in Table A will be deemed more unionable with `Location` from Table C than with `Destination` from Table B. This is because the syntactic similarity score, e.g. overlap and containment Jaccard, between the two `Destination` columns is 0; while the average word embedding of cities (Table A) is also not as close to that of nations (Table B). Similarly, if an ontology is used, Table A and Table C shares the same class while the values in B are in different (though related) classes. Meanwhile, looking at the tables as a whole we observe that Table A is actually irrelevant to Table C. But as existing solutions only look at the pair of single columns when calculating column unionability score, the columns `Year/Date` and `Destination/Location` of the two tables might be wrongly aligned together. Even techniques that look at relationships [24] can be fooled by the value overlap in this relationship and determine the relationship `Year-Destination` in Table A to be unionable with `Date-Location` in Table C. This kind of mistake can be avoided by looking at a table's context, i.e. information carried by other columns within a table. Looking at the table as a whole, a method should be able to recognize that the `Year` in Table A is part of a travel date while in Table C it is the date of discovery of a bird; and `Destination` in Table A refers to the cities to which the officers are traveling; whereas `Location` in Table C is the city where a bird is found.

From the above example, we focus on the following challenges in proposing a new solution. Firstly, it is essential to learn richer semantics of columns based on natural language domain. To this end, we require a more powerful approach to learn the column representation so as to capture richer information instead of relying on simple methods like the average over bag of word embeddings utilized in previous studies [2, 14] or even the similarity of the word embedding distributions [42]. Secondly, we argue that it is crucial to utilize the contextual information within a table to learn the representation of each column, which is ignored by previous studies. Even proposals for capturing relationship semantics do not use contextual information to learn column representations [24]. Finally, due to the large volume of data lake tables, it is also a great challenge to develop a scalable and memory-efficient solution.

We propose Starmie, an end-to-end framework for dataset discovery from data lakes with table union search as the main use case. Starmie uses pre-trained language models (LMs) such as BERT [13] to obtain semantics-aware representations for columns of data lake tables. While pre-trained LMs have been shown to achieve state-of-the-art results in table understanding applications [12, 31, 47], their good performance heavily relies on high-quality labeled training data. For the problem setting of table union search [41, 42], we must come up with a fully unsupervised approach in order to apply pre-trained LMs to such applications, something not yet supported by previous studies. Starmie addresses this issue by leveraging *contrastive representation learning* [10] to learn column representations in a **self-supervised manner**. An innovation of this approach is to assume that two randomly selected columns in a data lake can be used as negative training examples. For positive examples, we propose and use novel data augmentation methods. The framework defines a learning objective that connects the same or similar columns in the representation space while separating distinct columns. As such, Starmie can apply the pre-trained representation model in downstream tasks such as table union search without requiring any labels. We also propose to combine the learning algorithm with a novel multi-column table transformer model to learn *contextualized* column embeddings that model the column semantics depending on not only the column values, but also their context within a table. While a recent study SANTOS [24] can reach a similar goal by employing a knowledge base, our proposed methods can automatically capture such contextual information from tables in an unsupervised manner without relying on any external knowledge or labels.

Based on the proposed column encoders, we use cosine similarity between column embeddings as the column unionability score and develop a bipartite matching based method to calculate the table unionability score. We propose a filter-and-verification framework that enables the use of different indexing and pruning techniques to reduce the number of computations of the expensive bipartite matching. While most previous studies employed LSH index to improve the search performance, we also make use of HNSW (Hierarchical Navigable Small World) index [36] to accelerate query processing. Experimental results show that HNSW can significantly improve the query time while only slightly reducing the MAP/recall scores. Besides table union search, we further conduct two case studies to show that Starmie can also support other dataset discovery applications such as joinable table search and column clustering. We believe these results show great promise in the use of contextualized, self-supervised embeddings for many table understanding tasks.

Our contributions can be summarized as the following.

- We propose Starmie, an end-to-end framework to support dataset discovery over data lakes with table union search as the main use case.
- We develop a contrastive learning framework to learn contextualized column representations for data lake tables without requiring labeled training instances. Starmie achieves an improvement of 6.8% in both MAP and recall compared with the best state-of-the-art method, with a MAP of 99%, a significant margin compared with previous studies.

- We design and implement a filter-and-verification based framework for computing the table-level unionability score which can accommodate multiple design choices of indexing and pruning to accelerate the overall query processing. By leveraging the HNSW index, Starmie achieves up to three orders of magnitude in performance gain for query time relative to the linear scan baseline.
- We conduct an extensive set of experiments over two real world data lake corpora. Experimental results demonstrate that the proposed Starmie framework significantly outperforms existing solutions in effectiveness. It also shows good scalability and memory efficiency.
- We further conduct case studies to show the flexibility and generality of our proposed framework in other dataset discovery applications.

2 OVERVIEW

2.1 Problem definition

A data lake consists of a collection of tables $\mathcal{T}$. Each table $T \in \mathcal{T}$ consists of several columns $\{t_1, \ldots, t_m\}$ where each column t_i can be from different domains. Here m is the number of columns in table T (denoted as $|T| = m$). We will use the notation T to denote both the table and its set of columns if there is no ambiguity. To determine the unionability between two columns, following previous studies, we employ *column encoders* to generate the representations of columns. Then the *column unionability score* can be computed to measure the relevance between those representations. A column encoder $\mathcal{M}$ takes a column t as input and outputs $\mathcal{M}(t)$ as the representation. Given two columns t_i and t_j, the column unionability score is computed as $\mathcal{F}(\mathcal{M}(t_i), \mathcal{M}(t_j))$, where $\mathcal{F}$ is a scoring function between two column representations.

Based on the column unionability scores, we compute the table unionability score between two tables, which is obtained by aggregating the column unionability scores introduced above. Given two tables S and T, we define a table unionability scoring mechanism as $U = \{\mathcal{F}, \mathcal{M}, \mathcal{A}\}$, where $\mathcal{M}$ and $\mathcal{F}$ are the column encoder and scoring function for two column representations, respectively. Here $\mathcal{A}$ is a mechanism to aggregate the column unionability scores between all pairs of columns from the two tables. We will introduce the details of $\mathcal{A}$ later in Section 4.

Following the above discussions, we can formally define the table union search problem as a top-k search problem as Definition 2.1:

Definition 2.1 (Table Union Search). Given a collection of data lake tables $\mathcal{T}$ and a query table S, top-k table union search aims at finding a subset $\mathcal{S} \subseteq \mathcal{T}$ where $|\mathcal{S}| = k$ and $\forall T \in \mathcal{S}$ and $T' \in \mathcal{T} - \mathcal{S}$, we have $U(S, T) \geq U(S, T')$.

2.2 System architecture

Figure 2 shows the overall architecture of Starmie that solves table union search in two stages: offline and online.

During the offline stage, Starmie pre-trains a column representation model that encodes columns of data lake tables into dense high-dimensional vectors (i.e., column embeddings). Then, we apply the trained model to all data lake tables to obtain the column embeddings via model inference. We store the embedding vectors in

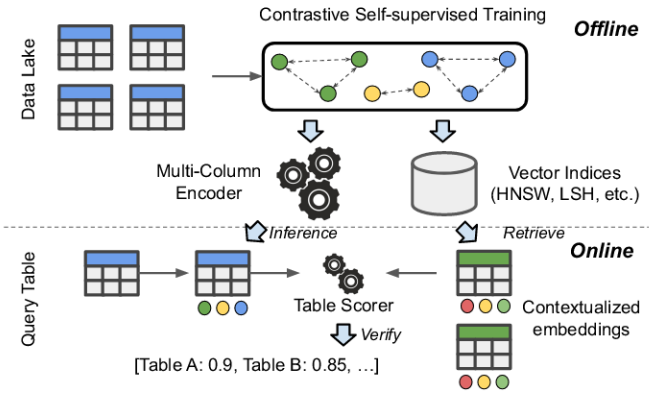


Figure 2: During the offline phase, Starmie **pre-trains a multi-column table encoder using contrastive learning and stores the embeddings of data lake columns in vector indices like HNSW. During online processing,** Starmie **retrieves candidate tables with similar contextualized column embeddings then verifies their table-level unionability scores using column alignment algorithms.**

efficient vector indices for online retrieval. A key challenge for the offline stage is to train high-quality column encoders that capture the semantics of tabular data. In Starmie, we follow a recent trend [12, 31, 47] of table representation learning that encodes tabular data using pre-trained language models (LMs). Pre-trained LMs have achieved state-of-the-art performance on table understanding tasks such as column type and relation type annotation [47]. However, the good performance of pre-trained LMs requires fine-tuning on high-quality labeled datasets, which are always not available in table search applications such as table union search. Using pre-trained LMs off-the-shelf is also problematic as the column embeddings cannot capture (ir-)relevance between columns or the contextual information within tables. To this end, in Section 3, we propose a contrastive learning framework for learning high-dimensional column representations in fully unsupervised manner. We combine the framework with a multi-column table model that captures column semantics from the column values while taking the table context into account. Then we apply the column encoder to all tables to convert each table into a collection of embedding vectors.

During the online stage, given an input query table, we retrieve a set of candidate tables from the vector indices by searching for data lake column embeddings of high column-level similarity with the input columns. Starmie then applies a verification step for checking and ranking the candidates for the top-k tables with the highest table-level unionability scores. The first challenge for the online stage is how to efficiently search for unionable columns. This is not a trivial task due to the massive size of data lakes. We address this challenge by allowing different design choices of state-of-the-art high-dimensional vector indices. Yet another challenge is designing a table unionability function that can effectively aggregate the column unionability scores. As in other studies, we employ weighted bipartite graph matching. To address its limitation of high computation complexity, we introduce a novel algorithm to reduce the number of expensive calls to the exact matching algorithm by deducing lower and upper bounds of the matching score (Section 4).

TabSketchFM: Sketch-based Tabular Representation Learning for Data Discovery over Data Lakes

Aamod Khatiwada[⋆▶] Harsha Kokel[⋆] Ibrahim Abdelaziz Subhajit Chaudhury Julian Dolby
Northeastern University *IBM Research* *IBM Research* *IBM Research* *IBM Research*
khatiwada.a@northeastern.edu harsha.kokel@ibm.com ibrahim.abdelaziz1@ibm.com subhajit@ibm.com dolby@us.ibm.com

Oktie Hassanzadeh Zhenhan Huang[▶] Tejaswini Pedapati Horst Samulowitz Kavitha Srinivas
IBM Research *Rensselaer Polytechnic Institute* *IBM Research* *IBM Research* *IBM Research*
hassanzadeh@us.ibm.com huangz12@rpi.edu tejaswinip@us.ibm.com samulowitz@us.ibm.com kavitha.srinivas@ibm.com

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00118

***Abstract*—Enterprises have a growing need to identify relevant tables in data lakes; e.g. tables that are unionable, joinable, or subsets of each other. Tabular neural models can be helpful for such data discovery tasks. In this paper, we present TabSketchFM, a neural tabular model for data discovery over data lakes. First, we propose novel pre-training: a sketch-based approach to enhance the effectiveness of data discovery in neural tabular models. Second, we finetune the pretrained model for identifying unionable, joinable, and subset table pairs and show significant improvement over previous tabular neural models. Third, we present a detailed ablation study to highlight which sketches are crucial for which tasks. Fourth, we use these finetuned models to perform table search; i.e., given a query table, find other tables in a corpus that are unionable, joinable, or that are subsets of the query. Our results demonstrate significant improvements in F1 scores for search compared to state-of-the-art techniques. Finally, we show significant transfer across datasets and tasks establishing that our model can generalize across different tasks and over different data lakes.**

I. Introduction

Enterprises store critical data in *data lakes*, large repositories of tabular data, for both governance and analytics [1]. It is essential to effectively find relevant tables (e.g., joinable, unionable, subsets) within lakes for reporting, decision-making, statistical analysis, and more [2], [3]. For instance, *Unionable table search* helps to augment an existing table with new rows, and to enrich analytics over it [4]. *Joinable table* search is useful to run analytics that needs data in columns from multiple tables, e.g., to identify how many customers in some region purchased a product due to an email campaign. Also, identifying *subsets* or *supersets* of a table is essential, for example, when searching for potential copies of personal data to comply with the General Data Privacy Regulation of the European Union[1]. Motivated by such use cases, we focus on the problem of identifying tables from the data lakes that can be *unionable, joinable, and/or subsets of each other* [4]–[6]. Specifically, in this work, we propose a pre-trained tabular model that could help in enhancing the effectiveness of data discovery techniques.

⋆ Joint first-authors ▶ Work done while at IBM Research.

[1]https://gdpr-info.eu

Central to the problem of identifying relevant tables in the data lakes is identifying similar columns in tables. However, the columns in a table could be ambiguous. For instance, a column called `Age` is different when it is in a table about buildings from when it is in a table about people. Furthermore, values within a column `Age` play a key role in determining semantics; `Age` of buildings in New York are unlikely to match `Age` of historical buildings in Egypt. Contextualizing columns could be useful in resolving such ambiguities. Language Models (LMs) are shown to be successful in contextualizing words in a sentence [7]. Consequently, we use them to contextualize columns and use such contextualization to determine similar columns or tables.

Existing pre-trained tabular models [8]–[11] based on LMs [7] do not focus on identifying relevant tables. Instead, they focus on language-style tasks such as fact checking [10], question answering [12], converting natural language to SQL [11], table cell content population [8], and table metadata prediction [8], [9]. In addition, input to these models is the actual cell values or a row because of the focus on language-style tasks. However, this has two limitations for understanding relevant tables: (a) only a small number of values can be passed in as input because of severe limits on token length in encoder-only models that can generate embeddings for search (e.g. 512 tokens in BERT [7]), and this potentially limits the information available to the model to understand the input table [2] ; (b) treating numerical values in the table as text tends to lose their semantics; it may be better to pass numerical information directly as vectors into the model.

To address these limitations on input length and numerical representation, we introduce a novel transformer-based tabular model—*TabSketchFM*—exploiting both table metadata and

[2] Decoder only models have larger context lengths. However, it has been difficult until recently to use decoder-only models for embeddings [13]. Moreover, these models are expensive to deploy and provide only a minimal improvement in accuracy over encoder-only models such as BERT. This is why embeddings for search are mostly based on BERT-style models. Further, even with large context lengths, it is not practical to serialize tables with millions of rows as sentences.

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00118

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:19 UTC from IEEE Xplore. Restrictions apply.

cell values that creates different sketches over them instead of linearizing cell values. Specifically, we abstract column cell values using data sketches, as has been used in the data discovery literature (e.g., MinHash sketches [14], and numerical sketches [15]). Such sketches capture tabular features, and we input them into the neural models. Inputting numerical sketches bypasses feeding table cell values to the model, addressing the token limit issue, as well as the numerical representation of cell values.

After pretraining using sketches, we finetune the models for individual data discovery tasks such as union, join, and finding subsets. The embeddings given by the finetuned models are then used for the respective retrieval tasks. While some in the literature [16] have used self supervision for fine tuning, self supervision applies better for some tasks such as unions compared to say joinability. To finetune models for the data discovery tasks, we used a suite of 8 benchmarks across the 3 tasks *LakeBench* that we have open sourced for use [17]. On the fine tuning tasks, *TabSketchFM* outperformed other neural methods, by as much as 55% on F1. We then used the fine tuned models for search tasks, and showed that *TabSketchFM* outperforms the state-of-the-art neural and traditional methods by as much as 70% on F1.

Importantly, we also compare the performance of specialized traditional and neural models for data discovery against an off-the-shelf pretrained model that encodes column values as a single sentence. To our surprise, the off-the-shelf model performs much better than existing systems - in fact, for certain tasks such as union, it is sufficient to use that model for discovering unionable tables; our tabular specific model does worse, as do all other models that target unionability. The insight from this finding is that while some data discovery tasks need values to match across columns (e.g. join or subsets), for union, it is sufficient when columns have the same semantic meaning; indeed there is no need for any value overlap. Sentence embeddings capture those semantics well. Because certain tasks seem to benefit from these sentence embeddings (e.g. union), and other do not (e.g., subset), we propose a way to concatenate the embeddings from sentence encoders with tabular model embeddings, and show that one can augment tabular embeddings with value embeddings to improve performance, at least in some cases.

In summary, our contributions are as follows:

- **A novel tabular pretrained model.** We are the first to present a sketch-based tabular pretrained model, *TabSketchFM*, which contextualizes columns based on their data sketches and tabular metadata. The sketches help to better handle input token limits and numerical cell values. To achieve this, we modified the text-based transformer architecture to take in numerical and MinHash sketches as input. A purely sketch based approach though misses the opportunity to include table semantics from column values. Given that the inputs to the transformer architecture in our system are numeric vectors, it is difficult to combine these numeric inputs with tokens derived from cell values. We therefore combine the sketch based model embeddings with column value embeddings from existing pretrained sentence transformer models. We show that this combination boosts performance on some data discovery tasks.
- **Finetuning models for downstream tasks.** We develop cross-encoders from pretrained TabSketchFM, which we fine-tune for a range of downstream tasks from the LakeBench collection [17]. We fine tune not only *TabSketchFM* but other tabular pretrained models to see if they can perform well on these downstream tasks and present the comparision.
- **Detailed ablation study.** We assess the effectiveness of different sketches for identifying different types of table relevance. Our study presents interesting insights such as Minhash sketches of column values are crucial for identifying joins, but numerical sketches are crucial for identifying subsets; once again highlighting the value of using sketches for table representation.
- **Demonstration of TabSketchFM on search.** We compared the performance of *TabSketchFM*'s fine tuned models against neural and non-neural baselines. In the case of join search, *TabSketchFM* outperformed the nearest systems geared for join search by almost 70% in F1 scores, but tellingly, the sentence encoding model outperformed existing systems as well by 63%. By combining the two embeddings (from TabSketchFM and sentence encodings), we gain almost 73% over other systems in F1.
- **Illustrate generalization.** Finally, we illustrate the main advantage of learning a pretrained foundational model by showing the generalization of search performance across tasks. For example, we fine tune the pretrained model on join identification on one dataset and use it for union search on another dataset. Our evaluation shows that our finetuned models generalize well.

II. Related Work

Tabular Pretrained Models. Neural models are pretrained on tabular datasets to either recover masked tokens in the table or to detect corrupt values [8]–[11]. [18] surveys neural models for tabular data representation and their applications. [8] combined the Masked Language Model (MLM) objective from BERT [7] with a novel Masked Entity Recovery to train TURL. [10] use MLM with novel Cell-level Cloze and Table Context Retrival for TUTA. [11] used Masked Column Prediction and Cell Value Recovery for TABERT while [9] repurposed ELECTRA's objective function [19] for TABBIE. These models are finetuned and evaluated for different downstream tasks such as table metadata prediction, table content population, fact-checking, question answering, semantic parsing, and so on. For example, TABERT is evaluated on neural semantic parsing; TABBIE for column, row, and column type population tasks; and TUTA is finetuned and evaluated for cell and table type classification tasks. These downstream tasks take either a table or a table with a question as input; they do not consider the inter-table tasks such as dataset discovery, which we consider in this paper. Furthermore, they adopt

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:19 UTC from IEEE Xplore. Restrictions apply.

serialization techniques from LMs and represent table values as sentences, whereas we represent using sketches.

Dataset Discovery Methods. Different methods such as keyword search [20], [21] and table search [22]–[24] have been used for the dataset discovery over data lakes. Table Union Search (TUS) [4] presented finding top-k data lake tables that are unionable with a query table. They determine the column unionability using three statistical tests based on value overlap, semantic overlap, and word embedding similarity, and aggregate them to infer table unionability. D^3L [25], adopted value overlap and word embedding measures from TUS and added three additional measures to measure column unionability: numerical column distributions, column header similarity, and regular expression matching. SANTOS [6] considers the relationships between the column pairs in both query and data lake tables. Fan et el. [16] developed a contrastive-learning-based table union search technique called Starmie that captures the context of the whole table in the form of column embeddings. AutoTUS [26] searches for unionable tables by contextualizing entire tables with embeddings of the relationship between the column pairs. Furthermore, there are other table search techniques that focus on finding joinable tables, i.e., given a query table marked with a query column, find the data lake tables that contain a column with overlapping values with the query column. For instance, LSH Ensemble [14] defines joinable table search as a problem of finding data lake tables that have a column having high set containment with the query column. They propose a novel index that approximates set containment computation. JOSIE [5] searches for top-k joinable tables based on exact set containment. AutoJoin [27], on the other hand, determines the fuzzy join between rows of a given pair of joinable tables by employing string transformations such as matching n-grams of cell values and concatenating cell values. PEXESO [28] efficiently searches for the top-k semantically joinable tables. Along with a join on exact values, they also consider fuzzy join, i.e., join on abbreviations and synonyms. DeepJoin [29] is a deep-learning-based technique that also searches for semantically joinable tables. WarpGate [30] is also a system for searching for top-k semantically joinable tables by embedding each column to vector space using pretrained embeddings. They index column embeddings using Locally Sensitive Hashing [31] for efficient search. Recently, Chorus [32] even demonstrated the ability of Large Language Models to identify joinable columns as a task of completing 'pd.merge' command when two tables are presented as pandas dataframe. Our work addresses the task of discovering tables from the data lakes that can be unionable, joinable, and/or subsets of each other. Notably, we specialize data discovery neural models based on a single pretrained model. The advantage of our approach over above mentioned task specific approaches is that, as in natural language processing, much smaller datasets are sufficient to finetune the model for new data discovery tasks; by leveraging the existing tabular knowledge in the pretrained model.

Benchmarks. Several tabular benchmarks exist in the literature [18], [33], [34]. Koutras et al. [35] created benchmarks for schema matching [36] using open data tables and evaluated the existing matching techniques over them. Mudgal et al [37] open-sourced 13 datasets such as the DBLP-Google Scholar author dataset and Walmart-Amazon product datasets for entity matching task [38]. The WDC Web Table corpus [39], containing around 233 million web tables extracted from different web pages, has been widely used for diverse tasks such as Question Answering [12], [40], Semantic Parsing [11], [40], [41], Table Retrieval [42], Table Metadata Prediction [8], [10], [43] and more [44]. Efthymiou et al. [45] used knowledge graphs to create tabular benchmarks for column type prediction and column-to-column binary relationship prediction tasks. The VizNet Benchmark [46] has been used to evaluate column type prediction tasks [47], [48]. GitTables [33] is a large table corpus in the literature. However, they contain web tables that are small entity tables with fewer rows and columns than enterprise tables and do not represent enterprise data lake scenarios [5]. But the most closely related benchmark to our work is the LakeBench collection of datasets [17] for identifying relation between pair of tables (Sec. III-D) Hence, we use that benchmark for finetuning our model. Further, there are publicly available table search benchmarks that are also relevant to our work which we use for illustrating the use of TabSketchFM for data discovery. TUS [4] and SANTOS [6] are table union search benchmarks which we include in our experiments. At the time of writing this paper, there were no search benchmarks for table join search and table subset search, so we generated *Wiki join search* and *Eurostat subset search* benchmarks (Sec. IV-C). However, recently two new benchmarks have appeared in this space. One is manually annotated table join and union search benchmark [49] (also called LakeBench[3]) and another is a semantic table search [50] benchmark that used wikipedia categories and navigational links to establish semantic join and union relation.

III. TabSketchFM

In this section, we detail the architecture of TabSketchFM. Like existing tabular models [10], [12], we pretrain the BERT model [7]. But, unlike them, we adapt the BERT input embeddings to accommodate the numerical inputs.

A. Sketches Preparation

While other models linearize table cells as text, we prepare *sketches*: numerical vectors that capture different features of the table. We use MinHash signature to convert sets of strings to numerical vectors. MinHash is a technique that uses hash functions to map similar sets to similar hash values, allowing for fast estimation of Jaccard similarity between two sets [51].[4] Specifically, we create one table-level sketch called a content snapshot and two types of column-level sketches: numerical sketch and minhash sketches.

Content Snapshot. Recognizing that row information could be crucial in detecting similarity of tables, we create a sketch from the first 10000 rows. We convert each row into a string

[3]Not to confuse with Srinivas et al. [17] LakeBench used for finetuning.

[4]We use datasketch library [52] to compute MinHash.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:19 UTC from IEEE Xplore. Restrictions apply.

DeepJoin: Joinable Table Discovery with Pre-trained Language Models

Yuyang Dong
NEC Corporation
dongyuyang@nec.com

Chuan Xiao
Osaka University
Nagoya University
chuanx@ist.osaka-u.ac.jp

Takuma Nozawa
NEC Corporation
nozawa-takuma@nec.com

Masafumi Enomoto
NEC Corporation
masafumi-enomoto@nec.com

Masafumi Oyamada
NEC Corporation
oyamada@nec.com

ABSTRACT

Due to the usefulness in data enrichment for data analysis tasks, joinable table discovery has become an important operation in data lake management. Existing approaches target equi-joins, the most common way of combining tables for creating a unified view, or semantic joins, which tolerate misspellings and different formats to deliver more join results. They are either exact solutions whose running time is linear in the sizes of query column and target table repository, or approximate solutions lacking precision. In this paper, we propose DeepJoin, a deep learning model for accurate and efficient joinable table discovery. Our solution is an embedding-based retrieval, which employs a pre-trained language model (PLM) and is designed as one framework serving both equi- and semantic (with a similarity condition on word embeddings) joins for textual attributes with fairly small cardinalities. We propose a set of contextualization options to transform column contents to a text sequence. The PLM reads the sequence and is fine-tuned to embed columns to vectors such that columns are expected to be joinable if they are close to each other in the vector space. Since the output of the PLM is fixed in length, the subsequent search procedure becomes independent of the column size. With a state-of-the-art approximate nearest neighbor search algorithm, the search time is sublinear in the repository size. To train the model, we devise the techniques for preparing training data as well as data augmentation. The experiments on real datasets demonstrate that by training on a small subset of a corpus, DeepJoin generalizes to large datasets and its precision consistently outperforms other approximate solutions'. DeepJoin is even more accurate than an exact solution to semantic joins when evaluated with labels from experts. Moreover, when equipped with a GPU, DeepJoin is up to two orders of magnitude faster than existing solutions.

PVLDB Reference Format:
Yuyang Dong, Chuan Xiao, Takuma Nozawa, Masafumi Enomoto, and Masafumi Oyamada. DeepJoin: Joinable Table Discovery with Pre-trained Language Models. PVLDB, 16(10): 2458 - 2470, 2023.
doi:10.14778/3603581.3603587

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 10 ISSN 2150-8097.
doi:10.14778/3603581.3603587

1 INTRODUCTION

Given a table repository and a query table with a specified join column, joinable table discovery finds the target tables that can be joined with the query. Due to the demonstrated usefulness in data enrichment [10, 16], joinable table discovery has become a key procedure in data lake management and serves various downstream applications, especially those involving data analysis.

For joinable table discovery, early attempts mainly targeted equi-joins [64, 66], which are the most common way of combining tables for creating a unified view [15] and can be easily implemented using SQL. To deliver more joins results for heterogeneous data, recent approaches [16, 17] studied semantic joins, which join on cells with similar meanings via word embedding, so as to handle data with misspellings and discrepancy in formats/terminologies (e.g., "American Indian & Alaska Native" v.s. "Mainland Indigenous"). There are two major limitations in these solutions. First, they only apply to a single join type. Second, most of them are exact algorithms with a worst-case time complexity linear in the product of query column size and table repository size, and thus their scalability is dubious. Despite the existence of an approximate algorithm for equi-joins [66], it is based on MinHash sketches [6] and has to convert the joinability condition to a Jaccard similarity condition, which is non-equivalent and introduces many false positives. Moreover, it is sometimes even slower than an exact algorithm [64].

Seeing the limitations of existing solutions, we propose DeepJoin, a deep learning model designed in a *two-birds-with-one-stone* fashion such that both equi- and semantic joins can be served with one framework. In particular, DeepJoin targets textual attributes with fairly small cardinalities that can fit with language models. To cope with semantic joins, it works on a similarity condition of word embeddings and finds similar textual columns as exactly as possible. To resolve the efficiency issue, it finds joinable tables via an **embedding-based retrieval**. In particular, we employ an embedding model to transform columns to a vector space. By metric learning, columns with high joinability are close to each other in the vector space. Then, to find the top-k target columns ranked by joinability, we resort to a state-of-the-art approximate nearest neighbor search (ANNS) algorithm [38], whose time complexity is *sublinear* in the table repository size.

DeepJoin utilizes a **pre-trained language model** (PLM) for column embedding. PLMs, such as BERT [14], have gained popularity in various data management tasks that involve natural language processing. A salient property of modern PLMs is that they are

transformer networks [55] featuring the attention mechanism, thus not only good at capturing the semantics of column contents for semantic joins, but also able to focus on the cells that are more probable to match in equi-joins, assuming that the query column has a similar distribution to those in the repository. As such, our model gains the capability of handling both join types, only needing the PLM to be fine-tuned on the data labeled for either equi- or semantic joins. In addition, PLMs produce a fixed-length vector, meaning that the following index lookup and search are *independent* of the column size, hence along with the ANNS, addressing the scalability issue in existing solutions. Since PLMs take a text sequence as input, by prompt engineering, we propose a set of options that contextualizes a column to a text sequence. To train the model in a self-supervised manner, we devise a series of techniques to effectively generate positive and negative examples. Moreover, our training features a data augmentation technique, through which our model can learn that the joinability is insensitive to the order of cells in a column.

We conduct experiments on two real datasets and evaluate DeepJoin equipped with two state-of-the-art PLMs. We show that by training on a small subset (30k columns) of the corpus, DeepJoin *generalizes* well to large datasets (1M columns). In particular, DeepJoin outperforms alternative approximate solutions in all the settings, and reports an average precision of 72% for equi-joins and 91% for semantic joins and an average NDCG of 81% for equi-joins and 75% for semantic joins. To test the effectiveness of semantic joins, we also evaluate DeepJoin using data labeled by our database researchers, and the results show that DeepJoin is even better by a margin of 0.105 – 0.165 F1 score than PEXESO [17], an exact solution we use to label DeepJoin's training data. An ablation study demonstrates the usefulness of the proposed contextualization and data augmentation techniques. For scalability test, we vary dataset size from 1M to 5M columns. Even if equipped with a CPU, DeepJoin exhibits superb scalability and is 7 – 57 times faster than existing solutions. With the help of a GPU, DeepJoin outperforms them by up to two orders of magnitude.

Contributions. (1) We propose DeepJoin, a framework for joinable search discovery in a data lake. Our solution targets targets textual attributes with fairly small cardinalities, and is able to detect both equi-joinable and semantically joinable tables. (2) We design the search in DeepJoin as an embedding-based retrieval which employs a fine-tuned PLM for column embedding and ANNS for fast retrieval. The search time complexity is sublinear in the table repository size, and except the column embedding, the search time is independent of the column size. (3) We propose a prompt engineering method to transform column contents to a text sequence fed to the PLM. (4) We devise techniques for training data preparation and data augmentation, as well as a metric learning method to fine-tune the PLM in a self-supervised manner. (5) We conduct experiments to show that our model generalizes well to large datasets and it is accurate and scalable in finding joinable tables.

Furthermore, we would like to mention the following facts: (1) For the embedding-based retrieval in our model, the results of ANNS are directly output as the results of joinable table discovery, whereas more advanced paradigms exist, such as two-stage retrieval [11] which finds a set of candidates by ANNS and ranks the candidates by a more sophisticated model. (2) DeepJoin is not limited to the two PLMs evaluated in our experiments, because the PLM can be regarded as a plug-in in our framework. As such, we expect the performance of DeepJoin can be further improved by using more advanced retrieval paradigms or PLMs.

2 PRELIMINARIES

2.1 Problem Definition

Given a data lake of tables, we extract all the columns in these tables, except those unlikely to appear in a join predicate (e.g., BLOBs), and create a repository of tables, denoted by $\mathcal{X}$. Given a query column Q, our task is to search $\mathcal{X}$ and find the columns joinable to Q. In this paper, we target equi-joins and semantic joins. Next, we define the joinability for these two types, respectively.

Given a query column Q and a target column X in $\mathcal{X}$, the joinability from Q to X is defined by the following equation.

$$jn(Q, X) = \frac{|Q_M|}{|Q|}, \tag{1}$$

where $|\cdot|$ measures the size (i.e., the number of cells) of a column, and Q_M is composed of the cells in Q that have at least one match in X. Here, the term "match" depends on the join type, i.e., an equi-join or a semantic join. We normalize the size of Q_M by the size of Q to return a value between 0 and 1. Moreover, the joinability is not always symmetric, depending on the definition of Q_M.

For equi-joins, we model each column as a set of cells by removing duplicate cell values, and define the equi-joinability as follows.

Definition 2.1 (Equi-Joinability). The equi-joinability from the query column Q to a target column X in $\mathcal{X}$ counts the intersection between Q and X, normalized by the size of Q; i.e., in Equation 1,

$$Q_M = Q \cap X. \tag{2}$$

The equi-joinability defined above counts the the distinct number of cells in Q that match those in X, and thus can be used to measure the equi-joinability [64]. Our method can be also extended to the case when columns are modeled as multisets, so as to support one-to-many, many-to-one, and many-to-many joins. In this case, we may measure the joinability by the number of join results and normalize it by the product of $|Q|$ and $|X|$, instead of $|Q|$ in Equation 1.

For semantic joins, we consider string columns and embed the value of each cell to a metric space $\mathcal{V}$ (e.g., word embedding by fastText [19]). As such, each string column is transformed to a multiset of vectors. We abuse the notation of a column to denote its multiset of vectors. Then, the notion of vector matching is defined as follows.

Definition 2.2 (Vector Matching). Given two vectors v_1 and v_2 in $\mathcal{V}$, a distance function d, and a threshold τ, v_1 matches v_2 if and only if the distance between v_1 and v_2 does not exceed τ. We use notation $M_\tau^d(v_1, v_2)$ to denote if v_1 matches v_2; i.e., $M_\tau^d(v_1, v_2) = 1$, iff. $d(v_1, v_2) \leq \tau$, or 0, otherwise.

Given a query column Q and a target column X, the semantic-joinability is defined using the number of matching vectors [1].

[1] Besides this definition, semantic joins are also investigated in [23], yet it studies the problem of performing joins rather than searching for joinable columns.

Definition 2.3 (Semantic-Joinability). The semantic-joinability from Q to X counts the number of vectors in Q having at least one matching vector in X, normalized by the size of Q; i.e., in Equation 1,

$$Q_M = \{\, q \mid q \in Q \wedge \exists x \in X \text{ s.t. } M_\tau^d(q,x) = 1 \,\}. \tag{3}$$

An advantage of the above definitions is that for both equi- and semantic-joinability, the training data can be labeled by an exact algorithm (e.g., JOSIE [64] and PEXESO [17]) rather than experts, so that our model can be trained in a self-supervised manner. Following the above definitions, we model the problem of joinable table discovery as the following top-k search problem, where joinability jn is defined using either Definition 2.1 or 2.3.

Problem 1 (Joinable Table Discovery). *Given a query column Q and a repository of target columns $\mathcal{X}$, the joinable table discovery problem is to find the top-k columns in $\mathcal{X}$ with the highest joinability from Q. Formally, we find a subset $\mathcal{R} \subseteq \mathcal{X}$, $|\mathcal{R}| = k$, and* $\min\{\, jn(Q,X) \mid X \in \mathcal{R} \,\} \geq jn(Q,Y), \forall Y \in \mathcal{X} \backslash \mathcal{R}$.

In this paper, we focus on dealing with textual columns. For numerical columns, a typical solution is utilizing the statistical feature vector in Sherlock [28], which converts a numerical column to a vector, and then we can invoke a vector search to look for joinable columns having similar statistics to the query column.

2.2 State-of-the-Art

JOSIE [64], a state-of-the-art solution to the equi-joinable table discovery problem, regards the problem as a top-k set similarity search with overlap $|Q \cap X|$ as similarity function, and builds its search algorithm upon prefix filter [7] and positional filter [59], which have been extensively used for solving set similarity queries. JOSIE creates an inverted index over $\mathcal{X}$, which regards each cell value as a token and maps each token to a postings list of columns having the token. Then, it finds a set of candidate columns by retrieving the postings lists for a subset of the tokens in Q (called prefix). Candidates are verified for joinability and the top-k is updated. While index access and candidate verification are processed in an alternate manner, JOSIE features the techniques to determine the their order, so as to optimize for long columns and large token universes, which are often observed in data lakes.

PEXESO [17] is an exact solution to semantic-joinable table discovery. It employs pivot-based filtering [8], which selects a set of vectors as pivots and pre-computes distances to these pivots for all the vecotrs in the columns of $\mathcal{X}$. Then, given the vectors of the query Q, non-matching vectors can be pruned by the triangle inequality. A hierarchical grid index is built to filter non-joinable columns when counting the number of matching vectors.

As for the weakness, JOSIE inherits the limitation of prefix filter, whose performance highly depends on the data distribution and yields a worst-case time complexity of $O(|\mathcal{X}| \cdot (|Q| + \overline{|X|}))$, where $\overline{|X|}$ stands for the average size of the columns in $\mathcal{X}$. For PEXESO, despite a claimed sublinear search time complexity of $O(\log |\mathcal{X}_V| \cdot \log |Q|)$, where $\mathcal{X}_V$ denotes the multiset of all the vectors in the repository, it relies on a user-specific threshold for the count of matching vectors. This does not apply to the top-k case, and the algorithm is downgraded to be linear in $|\mathcal{X}_V| \cdot |Q|$, because at the early stage of search, due to the low count of matching vectors in the temporary top-k results, the pruning power of the grid index is next to none.

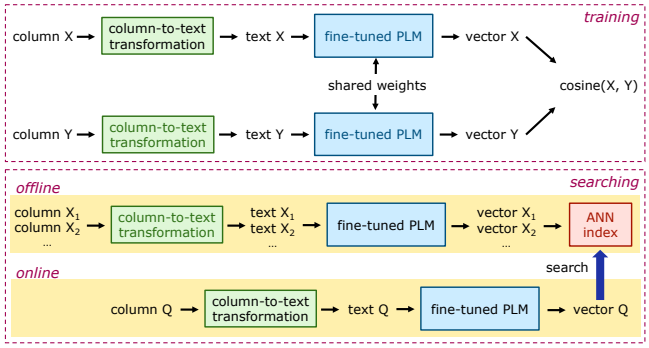


Figure 1: Overview of the DeepJoin model.

Table 1: Column-to-text transformation options.

Name	Pattern
col	$cell_1$, $cell_2$, . . ., $cell_n$
colname-col	$column_name$: col.
colname-col-context	$colname-col$: col. $table_context$
colname-stat-col	$column_name$ contains n values (max_len, min_len, avg_len): col
title-colname-col	$table_title$. $colname-col$
title-colname-col-context	$title-colname-col$. $table_context$
title-colname-stat-col	$table_title$. $colname-stat-col$.

In general, for search time, both JOSIE and PEXESO are linear in the product of column size and repository size, which compromises their scalability to long columns and large datasets. On the other hand, it is unnecessary to always find an exact answer, because data scientists are usually concerned with only part of the top-k results and will choose a subset of them for the join. For this reason, we will design our solution with the following two goals: (A) it returns an approximate answer with *sublinear* time in $|\mathcal{X}|$, and (B) by encoding the query column to a fixed length, it is *independent* of $|Q|$ and $\overline{|X|}$ during index lookup and search.

Apart from exact solutions, LSH Ensemble [66] is an approximate solution to equi-joinability. It partitions the repository and computes MinHash sketches [6] with parameters tuned for each partition. Unlike JOSIE, it targets the thresholded problem (i.e., $\frac{|Q \cap X|}{|Q|} \geq t$), which requires a user-specified threshold t and thus is less flexible than computing top-k. Although adaptation for the top-k problem is available, it suffers from low precision due to the many false positives introduced by transforming overlap similarity to Jaccard similarity for the use of MinHash, and it sometimes runs even slower than JOSIE [64].

3 THE DEEPJOIN MODEL

Figure 1 illustrates the overview of our DeepJoin model. In DeepJoin, the joinable table discovery is essentially an embedding-based retrieval, which has recently been adopted in various retrieval tasks, such as long text retrieval [33] and search in social networks [27]. An embedding-based retrieval usually employs metric learning or meta embedding search to learn embeddings for target data such that the semantics can be measured in the embedding space, especially when target data are highly sparse or the semantics is hard to define by an analytical model. Another key benefit is, by

SANTOS: Relationship-based Semantic Table Union Search

AAMOD KHATIWADA*, Northeastern University, USA
GRACE FAN*, Northeastern University, USA
ROEE SHRAGA, Northeastern University, USA
ZIXUAN CHEN, Northeastern University, USA
WOLFGANG GATTERBAUER, Northeastern University, USA
RENÉE J. MILLER, Northeastern University, USA
MIREK RIEDEWALD, Northeastern University, USA

Existing techniques for unionable table search define unionability using metadata (tables must have the same or similar schemas) or column-based metrics (for example, the values in a table should be drawn from the same domain). In this work, we introduce the use of semantic relationships between pairs of columns in a table to improve the accuracy of the union search. Consequently, we introduce a new notion of unionability that considers relationships between columns, together with the semantics of columns, in a principled way. To do so, we present two new methods to discover the semantic relationships between pairs of columns. The first uses an existing knowledge base (KB), and the second (which we call a "synthesized KB") uses knowledge from the data lake itself. We adopt an existing Table Union Search benchmark and present new (open) benchmarks that represent small and large real data lakes. We show that our new unionability search algorithm, called SANTOS, outperforms a state-of-the-art union search that uses a wide variety of column-based semantics, including word embeddings and regular expressions. We show empirically that our synthesized KB improves the accuracy of union search by representing relationship semantics that may not be contained in an available KB. This result hints at a promising future of creating synthesized KBs from data lakes with limited KB coverage and using them for union search.

CCS Concepts: • **Information systems → Information integration**.

Additional Key Words and Phrases: Data lakes, Table Discovery, Knowledge Graphs

ACM Reference Format:
Aamod Khatiwada*, Grace Fan*, Roee Shraga, Zixuan Chen, Wolfgang Gatterbauer, Renée J. Miller, and Mirek Riedewald. 2023. SANTOS: Relationship-based Semantic Table Union Search. *Proc. ACM Manag. Data* 1, 1, Article 9 (May 2023), 25 pages. https://doi.org/10.1145/3588689

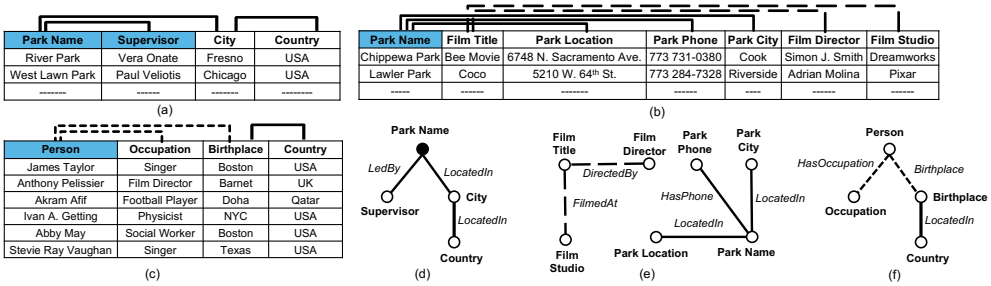
1 INTRODUCTION

Table search is of growing practical relevance [7, 23, 30, 49]. It allows data scientists to find datasets needed for analysis or training of machine learning algorithms [14, 33]. Google's dataset search, for example, uses keyword search over metadata which Google incentivizes data owners to use in

* Khatiwada and Fan are lead authors.

Authors' addresses: Aamod Khatiwada*, Northeastern University, USA, khatiwada.a@northeastern.edu; Grace Fan*, Northeastern University, USA, fan.gr@northeastern.edu; Roee Shraga, Northeastern University, USA, r.shraga@northeastern.edu; Zixuan Chen, Northeastern University, USA, chen.zixu@northeastern.edu; Wolfgang Gatterbauer, Northeastern University, USA, w.gatterbauer@northeastern.edu; Renée J. Miller, Northeastern University, USA, miller@northeastern.edu; Mirek Riedewald, Northeastern University, USA, m.riedewald@northeastern.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2836-6573/2023/5-ART9 $15.00
https://doi.org/10.1145/3588689



Park Name	Supervisor	City	Country
River Park	Vera Onate	Fresno	USA
West Lawn Park	Paul Veliotis	Chicago	USA
-------	------	-------	--------

(a)

Park Name	Film Title	Park Location	Park Phone	Park City	Film Director	Film Studio
Chippewa Park	Bee Movie	6748 N. Sacramento Ave.	773 731-0380	Cook	Simon J. Smith	Dreamworks
Lawler Park	Coco	5210 W. 64th St.	773 284-7328	Riverside	Adrian Molina	Pixar
-----	------	-------	------	----	------	------

(b)

Person	Occupation	Birthplace	Country
James Taylor	Singer	Boston	USA
Anthony Pelissier	Film Director	Barnet	UK
Akram Afif	Football Player	Doha	Qatar
Ivan A. Getting	Physicist	NYC	USA
Abby May	Social Worker	Boston	USA
Stevie Ray Vaughan	Singer	Texas	USA

(c)

Fig. 1. (a) A table about parks. Relationships between `Park Name` and `Supervisor` (`ledBy`) and between `Park Name` and `City` (`locatedIn`) are found and depicted by the solid lines above the table. We have also found a relationship between `Country` and `City` (`locatedIn`). (b) A table about parks and films shown in the park. Some of the relationships we found are shown by solid and dashed lines above the table. (c) A table about famous people. (d) A semantic graph for Table (a) with the relationships found related to the root `Park Name`. (e), (f) Semantic graphs for Tables (b), (c), respectively.

a standard way [4]. However, when dealing with enterprise data or open data, keyword search via textual queries or over metadata is challenging because metadata may be missing, inconsistent, or incomplete [1, 10, 32, 47]. Hence, table search for those domains must rely on the data available within given tables instead of relying on well-curated metadata [7, 23].

In this work, we focus on *table union search*, which is the problem of finding tables that can union with a query table and possibly extend it with more rows. Prior work by Nargesian et al. [33] considers two tables to be *unionable* if "they share attributes from the same domain". We believe that this is a necessary, but not sufficient condition. In particular, we argue that the relationships modeled by pairs of attributes in a table should share a common semantic meaning. We illustrate this with an example.

Example 1. *Consider Tables (a), (b) and (c) in Fig. 1 describing (a) parks, (b) films shown in different parks, and (c) the birth places of famous people, respectively. Suppose the user supplies Query Table (a) and Tables (b), (c) reside in a data lake. The user wants to find data lake tables that union with the Query Table (a). Typically, union search has been defined to permit either the full query table or a projection of it to union with the full data lake table or a projection of it [3]. Now, if we only consider attribute unionability, Table (c) may be considered the most unionable table with three unionable attributes (*`Person`, `Birthplace`, `Country`*) having high attribute unionability scores with the attributes (*`Supervisor`, `City` *and* `Country`*) in the query table. This may be true if we are considering data values [33] or if we consider the attribute names and values [3, 21, 24]. However, if we consider the relationships in the tables, then we would notice that in Query Table (a),* `City` *is where the* `Park` *is located (and the person in* `Supervisor` *works), but in Table (c),* `Birthplace` *contains cities where the people in the* `Person` *column were born. Deciding unionability based only on attribute unionability, without considering the semantics of the relationships between attributes, can lead to false positives. Even worse, it can lead to the omission of good answers like Table (b) which has information about parks and the city in which they are located, but might be considered less unionable than (c) based solely on column semantics.*

We present a new definition of unionability based on both the attribute semantics and the *relationship semantics between attributes*. Relationships have been used to a limited extent in web table search. Their tables are often assumed to be "entity tables" (i.e., all attributes are properties of a known common entity represented by a single attribute) [7, 8, 28] or they use a subject column and a single relationship from that column to search for tables [43]. However, we do not assume

that every table has a single subject attribute that all other attributes describe, nor that we can automatically detect such an attribute. While the entity-table assumption may be mostly true in web tables, data lake tables (in both open data and enterprise lakes) tend to be much wider and may describe the interplay between more than one entity.

In this work, we provide a new definition of table union search. Given a query table Q and a set of data lake tables $\mathcal{T}$, the top-k table union search problem is to find the best k tables from $\mathcal{T}$ that can be unioned with the query table by taking the column and relationship semantics of all involved tables into account.

Example 2. *The columns* `City` *and* `Country` *in Table (a) are unionable with* `Birthplace` *and* `Country` *in Table (c) because the columns are unionable and their relationships are the same (*`locatedIn`*). If a user is interested in amassing a collection of cities and their properties like location, then these tables are unionable. However, if a user wants to collect a set of park supervisors and their properties (the city and country in which they work), then Tables (a) and (c) are not unionable. This is because the relationship between* `Supervisor` *and* `City` *(*`worksIn`*) is not the same as the relationship between* `Person` *and* `Birthplace` *(*`bornIn`*). A user may consider Table (b) to be unionable with Table (a), but only on* π*(*`Park Name, City, Country`*). If we only consider attribute semantics, we might mistakenly say* `Park Name, Supervisor, City` *is unionable with* `Park Name, Film Director, Park City`*. But the relationship semantics reveal that* `Supervisor` *has a different relationship to* `Park Name` *than* `Film Director` *has to* `Park Name`*.*

1.1 Contributions

Our contributions can be summarized as follows.

• *Relationship-based Semantic Table Union Search.* We present a new definition of table union search that comprehensively integrates column and relationship semantics.

• *KB Solution.* We present a solution to the relationship-based semantic table union search problem that uses a knowledge base (KB) to determine column and relationship semantics. Our solution creates semantic graphs for tables whose nodes are columns and edges are binary relationships found in the KB (depicted in Fig. 1(d)-(f) and explained in Sec. 3). We present a scoring function to match the semantic graph of a data lake table to that of a query table.

• *Synthesized KB Solution.* We present a second, novel solution to the search problem that uses the data lake itself to determine if a set of columns have the same or similar semantics. Our solution determines if the binary relationship between two columns has similar semantics with other binary relationships in the data lake. We show how this *synthesized KB* can be used to create semantic graphs and solve the table union search problem.

• *Empirical Evaluation.* Our two solutions were designed to be used in concert with the Synthesized KB compensating for a curated KB that has only partial coverage over a data lake (which is typical when using both open KBs and proprietary enterprise KBs). We show that when used together, our solutions outperform a SOTA table union baseline. We also experiment with using each solution independently. Finally, we use the Synthesized KB together with different sized samples of the curated KB to understand how accurate our solution is as the coverage of the curated KB decreases.

• *Benchmarks.* We develop (and openly share) three new relationship union benchmarks. In the first, we reuse part of the TUS benchmark [33] and label it with relationship semantics. In the second, we use real open data tables and queries, and label a groundtruth. The third is a larger data lake with open data tables and queries, where we only label the query results that either our technique or a baseline technique returns. Hence, this larger benchmark can be used to evaluate precision and efficiency, but not recall.



Searching Data Lakes for Nested and Joined Data

Yi Zhang*
AWS AI Labs
imyi@amazon.com

Peter Baile Chen*
MIT
peterbc@mit.edu

Zachary G. Ives
University of Pennsylvania
zives@cis.upenn.edu

ABSTRACT

Exploratory data science is driving new platforms that assist data scientists with everyday tasks, such as integration and wrangling, to assemble training datasets. Such tools take scientists' work-in-progress data as a *search object* (table or JSON) and find relevant supplementary data from an organizational *data lake*, which can be unioned or joined with the current data. Existing data lake search tools find *single*, relational tables to match or join with a search object. Yet many data science applications revolve around hierarchical data, which can only be matched by creating views that simultaneously *join and transform several* tables in the data lake. In this paper, we extend the Juneau data lake search system [46] for this broader class of matches *at scale*. Our contribution is a *general* framework for efficiently merging ranked results to match hierarchical data, leveraging novel techniques for indexing and sketching, and incorporating existing single-table search techniques and ranking functions. We experimentally validate our methods' benefits and broad applicability using real data from data science computational notebooks. Our results indicate that, with different ranking functions, our approach can return the optimal set of views up to 4.8x faster and 43% more related compared to heuristics, and increase the data domain coverage by up to 28%. In a case study to show the utility of our results to data science downstream tasks, we reduce regression error by up to 6.6%, and improve classification accuracy by up to 19.5%.

PVLDB Reference Format:
Yi Zhang, Peter Baile Chen, and Zachary G. Ives. Searching Data Lakes for Nested and Joined Data. PVLDB, 17(11): 3346 - 3359, 2024.
doi:10.14778/3681954.3682005

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/juneau-project/juneau.

1 INTRODUCTION

Data scientists are increasingly using tools that assist with data exploration [19], machine learning model training, and workflow definition tasks. These scientists search over tables and code from public repositories such as GitHub [44], as well as enterprise or public data lakes [34, 46–48] to find supplemental data that can augment their working dataset(s). In the same spirit as Code Llama [39] or GitHub CoPilot [18] for software engineering, such tools assist programmers with data engineering tasks such as discovery and data augmentation [34, 46], as well as data wrangling and cleaning steps [6, 46]. Such tools complement large language models like GPT, Claude, and Gemini. Unlike LLMs that make suggestions based on their parametric knowledge, search tools do not "hallucinate" results. Moreover, they can incorporate policies for domain-specific data types (e.g., gene sequence similarity) or for preferring one source over another (e.g., based on freshness).

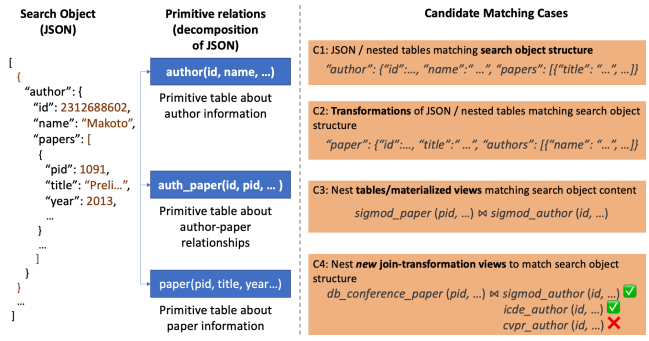


Figure 1: Schema-compliant matches to a JSON search object may come directly or through joins followed by nesting transformations.

The core of such tools is a *search system over data lakes* [13, 32, 34]: Here, the user invokes a query via a *search object* (in existing systems, a table) and the system queries data lake resources for (1) union-compatible tables [13, 33, 46, 48] that can extend training or test datasets with new tuples, (2) joinable tables [16, 46, 47] that integrate data or add machine learning features, or (3) common computational steps found as used in prior workflows [22, 24, 44, 46]. In contrast to web table search [5, 37], the critical challenges of developing such tools lie in understanding the (semantic) compatibility of the schema and detecting the primary-key overlap with the query table [13, 46] . There are two broad classes of existing systems: union-compatible search strives to find tables that *match the search object's schema* while providing novel rows; join-compatible search tends to be non-goal-directed, opportunistically finding tables that appear to join with the existing search table. Recent work [26] shows how to use *full disjunction* to knit the results together to form one output schema.

However, the paradigm of single-table matches to a search object is overly restrictive. Data scientists frequently work with JSON files, nested Dataframes, and composite tabular results (views) that join, pivot, and explode imported tables. Here, to provide the best search results — *the search system must combine subsets of indexed data into new hierarchical or joined data matching the search object structure.* New combinations of indexed tables (i.e., automatically discovered views) better leverage existing data and provide additional insights.

*This work was done while the authors were at the University of Pennsylvania.
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 17, No. 11 ISSN 2150-8097.
doi:10.14778/3681954.3682005

This requires that the data lake system can *not only find 1NF tables* but *explore ways of creating query transformations over combinations of indexed tables to match the query.*

Example 1.1. Snapshots of the popular DBLP publications database (dblp.org) have been exported for use in data science tasks, e.g., in notebooks shared on kaggle.com. These snapshots are often subsets of the entire database, taken at different points in time, and their representation may range from flat CSVs[1] to nested JSON[2]. A data lake could also have complementary publication data (e.g., venues not indexed in DBLP) extracted from conference pages or the ACM Digital Library. Now (see Figure 1), if a data scientist takes a JSON snapshot of DBLP and searches for similar data to train a classifier that recognizes CS papers, we would like to return *all* relevant author-publication data (converted to the appropriate structure), including (C1) objects that directly match the nested schema, authors with nested papers; (C2) "pivoted" JSON, in a different structure with semantically compatible content (e.g., papers with nested authors); (C3) nesting of flat tables or views representing the same concepts as the JSON; (C4) nesting of join queries over 1NF relations, matching the JSON schema.

In a similar fashion, a data scientist may take portions of the above data and create an intermediate table for machine learning training, which joins data from different levels of the DBLP hierarchy. At this point, they may want to find matches, which, while "flat," can be matched by joining 1NF data (which, in turn, could have been gathered by decomposing other JSON structures). Unlike prior join-compatible search approaches, this task is *goal-directed* in terms of what joins; unlike prior union-compatible search approaches, this task requires more than single-table matches.

Conventional table search within a data lake introduces two key challenges: identifying semantic and value overlap across thousands to millions of tables, which has generally been addressed using sketch and embedding techniques [16, 45, 47]; and efficiently ranking the results, especially if there are multiple ranking measures [46]. Extending these ideas to heterogeneous and hierarchical *compositions* of tables and JSON/XML results in three additional critical challenges:

(1) Matches to the search object may be *partial*, requiring joins to produce a complete search result. In our example, the JSON can be normalized into three "primitive" relations: *author*, *paper*, *auth_paper*. Individual data objects might contain the information from all these (C1-3 in Figure 1); or might require us to find *other* tables to join, to form a complete match (C4). Efficiently returning the top-k results, when each result has an irregular number of joins, explodes the search space.
(2) Simply querying the primitive relations in top-k order, then joining them, does not produce the top-k answers: intuitively, high-scoring matches to papers may not actually join with high-scoring matches to authors (as per database papers with CVPR authors in Figure 1.) Therefore, the ranking function should measure the compatibility of a composition of primitive relations, and the score should be independent of the order of evaluation.
(3) Automatically and efficiently identifying which relations should join (without introducing large numbers of false positives) is nontrivial – this is aided tremendously by identifying fields that are good keys and also highly indicative of semantic content, e.g., DOIs, SSNs, etc.

[1]https://www.kaggle.com/jakboss/chunk-of-dblp-dataset
[2]https://www.kaggle.com/mathurinache/citation-network-dataset

Searching data lakes for nested and joined data requires novel indexing and pruning strategies. Broadly, as in Figure 1, the problem requires that we decompose the nesting structure of our hierarchical data and find matches to each nesting level (columns in 1NF tables, or in decompositions of other nested tables); then we return a ranked set of queries (views) over these matches as the answers. Note that among the search cases we have discussed, C4 is particularly challenging, as queries need to be created first. To address these problems, we must innovate in how we index content in tables, and perform top-k computations, with the assistance of the indices, across multiple tables while joining them.

A significant body of empirical work has been developed on finding the best features for ranking individual tables considering schema overlap, table overlap, and joinability (see Section 2). In this paper, we develop a general query system that can "plug in" existing individual table ranking strategies (with different degrees of benefits depending on the specific setup), and our focus is on scaling up running times of returning the (new) views over these tables. To facilitate such a query system, we address the following challenges.

Enumerating plans to join and nest data decomposed into 1NF, for a broad class of ranking functions. We *compose* partial matches (1NF primitive tables) from our data lake using nesting and join operations to match search object structure — favoring joins of results that originally came from the same sources. However, our system also considers cross-source joins, which are critical to applications in Linked Open Data and scientific settings with private data referencing entries in public databases.

Incremental maintenance of sketches and profiles. Existing data lake search systems developed *in-memory* sketches and data profiles. To support data compositions in millions of tables, these must be persisted and incrementally maintained.

Automatic index selection. To scale up the exploration of potential joins to produce a match, we develop solutions to determine combinations of fields to index across the data lake, in the form of automatic *data profile* selection.

To sum up, this paper makes the following contributions:

- A new class of data lake search tools over joined and nested data, whose task is to generate unions of nest-join queries over data lake tables to match a search object, with *customizable ranking functions or modules.*
- Identification of key requirements for ranking functions over candidate *views (over multiple tables)*, enabling an efficient *correlated top-k matching algorithm* to rank them.
- *Automatic* selection techniques for *profiling and sketching* millions of tables.
- Experimental validation of the viability and the generality of our methods, over a diverse benchmark suite of real data and notebooks, using state-of-the-art table scoring functions.

2 PRIOR WORK

Semantics of Table Search. The problem of searching a data lake for related tables has generated a significant interest [34].

Value-based table search (keyword search over tables). The earliest table search systems look for matches to words or values *within* tables. They rank table matches by how well the terms represent the tables. This line of work predates the data lake, and ranks tables by TF/IDF-like measures; tables are matched without considering the compatibility between their schemas. Such tools were largely targeted at enterprise networks [3, 15, 16, 20, 21, 43] and/or tables that appear on the web [4, 5, 37].

Unionable table search (table augmentation). Given a search table, these systems use schema alignment techniques to find similar columns. They rank tables based on schema similarity and how many new rows are added [48]. At scale, columns are usually compared using sketch-based techniques including LSH [16, 34, 46], as well as techniques borrowed from schema matching [27] and column or table embeddings [8, 10, 14] with Approximate Nearest Neighbor Search (ANNS). D^3L [2] further combined all these techniques. Beyond those ideas, SANTOS [25] recently proposed to use a knowledge base to discover the semantic relationships between pairs of columns in tables to find unionable tables. There are also efforts considering how data from data science computational notebooks can be leveraged as a data lakes [46], how computation over that data might be reorganized [23], and even how schema constraints might be inferred [41].

Joinable table search. Given a search table, this class of system identifies columns that can be joined to other columns from tables in the data lake, and ranks tables based on the estimated join compatibility [7, 17, 46, 47]. This line of work focuses on tables with matching join columns, but otherwise is not generally directed by a target schema. Similar to union-compatible table search, it is intractable to do pairwise comparisons of columns to find joinable pairs of columns, and therefore LSH and other sketch-based techniques are often used. Recently, DeepJoin [11] proposed an embedding-based retrieval method, which employs a pre-trained language model fine-tuned on equi- or semantic join data for column embedding and uses ANNS, i.e., HSNW [31] for fast retrieval.

The ALITE system [26] explores how to merge and align different (tabular) search results (as a query post-processing), using Full Disjunction as a means of null-padding tables as necessary to achieve union compatibility among the matches. However, since the returned matches are only constructed with tables returned by the query for individual tables, the search space is limited.

Prior efforts match an *individual* table against other "raw" tables. Our work addresses union-compatible *hierarchical* data and investigates techniques for scalably exploring joining and nesting *queries* over primitive tables.

Foundations of our work. As the foundations of our work, we extend an open-source, data lake search system, Juneau [46] [3], which encompasses many of the basic techniques described above. More specifically, the open-source version of Juneau provides three core capabilities, which are integrated into the Jupyter Notebook data science environment.

[3] https://github.com/juneau-project/juneau

Searching: Given a request from the user with a search table S, Juneau searches for other tables that can be combined with S by unioning or joining. It compares S against other tables using a series of "pluggable" metrics, including schema overlap, value overlap, and more. These are combined in a weighted linear fashion, and new metrics can be added, so long as the final function is monotonic with regards to the individual metrics. Juneau works as a general engine for *top-k query processing*, which ranks tables via this similarity score. Juneau's extensibility differentiates it from alternatives in the previous section. Compared to those single-similarity-metric systems [47], Juneau provides significant flexibility and customizability, but at the cost of more expensive search. For our problem which involves multiple matching criteria, this customizability is essential as well — but Juneau's algorithms are not scalable enough.

Indexing and index selection: To scale its search and top-k ranking algorithms to tens of thousands of tables, Juneau relies on additional strategies for indexing. A key capability is *data profile*, a means of identifying and indexing subsets of columns in each table belonging to a particular domain, which often co-occur. Intuitively, a data profile captures the **domain** that identifies semantically meaningful columns (e.g., a country) or combinations of columns (e.g., a country + continent). However, Juneau requires a significant extension to handle millions of tables, as we target.

Loading: As Juneau encounters tables (as they are created or loaded in the Jupyter Notebook environment), it automatically loads these into the data lake, in the process also extending any existing data profiles to consider the data in each new table.

The Juneau system provides a foundation and platform implementation for exploring query-based data lake search — but we also leverage other ideas from the literature.

A key aspect of table matching is the use of sketches for computing column compatibility (both for schema alignment and record linking): LSH ensembles allow us to find overlapping strings [48]; Kolmogorov-Smirnov sketches allow us to compare value distributions for numeric fields [42]. Our platform indexes and stores hierarchical data, including XML and JSON. To enable maximum flexibility during search (where the search object's hierarchy may not exactly match the stored object's hierarchy), we normalize all hierarchical data into 1NF relations, but maintain information about the original structure, as well as the foreign keys required to recompose them into original or pivoted hierarchies. To do this, we adopt ideas developed for storing XML in relations [9, 40]. For XML, this occurs by consulting the XML schema and creating or reusing relations (with appropriate schema elements) at $1:n$ nesting boundaries; then, on-the-fly as the XML is parsed, different components of the XML tree are directly inserted into the appropriate tables. Finally, our work on top-k query processing adapts ideas from Fagin's Threshold Algorithm [12] as well as the J^* algorithm [35] to iteratively explore options.

3 ENUMERATING POSSIBLE QUERIES

Our work explores how to create and rank *join-nest queries* over combinations of tables in a data lake, which can be unioned with a (hierarchical, i.e., non-1NF) search object. Our goal is a general architecture that supports many potential scoring functions (modules) from the literature. In this section, we identify (1) the potential

Table Overlap Estimation through Graph Embeddings

FRANCESCO PUGNALONI, Hasso Plattner Institute, University of Potsdam, Germany
LUCA ZECCHINI, University of Modena and Reggio Emilia, Italy
MATTEO PAGANELLI, University of Modena and Reggio Emilia, Italy
MATTEO LISSANDRINI, University of Verona, Italy
FELIX NAUMANN, Hasso Plattner Institute, University of Potsdam, Germany
GIOVANNI SIMONINI, University of Modena and Reggio Emilia, Italy

Discovering duplicate or high-overlapping tables in table collections is a crucial task for eliminating redundant information, detecting inconsistencies in the evolution of a table across its multiple versions produced over time, and identifying related tables. Candidate duplicate or related tables to support this task can be identified via the estimation of the largest table overlap. Unfortunately, current solutions for finding it present serious scalability issues for heavy workloads: Sloth, the state-of-the-art framework for its estimation, requires more than three days of machine time for computing 100k table overlaps.

In this paper, we introduce Armadillo, an approach based on graph neural networks that learns table embeddings whose cosine similarity approximates the *overlap ratio* between tables, i.e., the ratio between the area of their largest table overlap and the area of the smaller table in the pair. We also introduce two new annotated datasets based on GitTables and a Wikipedia table corpus containing 1.32 million table pairs overall labeled with their overlap. Evaluating the performance of Armadillo on these datasets, we observed that it is able to calculate overlaps between pairs of tables several times faster than the state-of-the-art method while maintaining a good quality in approximating the exact result.

CCS Concepts: • **Information systems** → *Information retrieval*; *Information integration*; *Deduplication*.

Additional Key Words and Phrases: table representation learning, graph neural networks, data lakes, web tables, table overlap, table matching

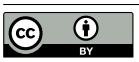
ACM Reference Format:
Francesco Pugnaloni, Luca Zecchini, Matteo Paganelli, Matteo Lissandrini, Felix Naumann, and Giovanni Simonini. 2025. Table Overlap Estimation through Graph Embeddings. *Proc. ACM Manag. Data* 3, 3 (SIGMOD), Article 228 (June 2025), 25 pages. https://doi.org/10.1145/3725365

1 Estimating table overlap

Tables are one of the most common formats used in data lakes and Web sources for organizing information that refers to a common schema [39]. For instance, a large amount of tabular data can be found in GitHub, which contains more than 10 million tables stored as CSV files [25], and Wikipedia, where more than 3 million HTML tables have been created throughout its history [2, 5].

In these large table corpora, it is frequent to find pairs of tables that share the same content across multiple cells, i.e., *overlapping* tables. This overlap, depending on its size and shape, can be a signal of table relatedness. For instance, a *vertical* overlap spanning several rows across a

Authors' Contact Information: Francesco Pugnaloni, Hasso Plattner Institute, University of Potsdam, Germany, francesco.pugnaloni@hpi.de; Luca Zecchini, University of Modena and Reggio Emilia, Italy, luca.zecchini@unimore.it; Matteo Paganelli, University of Modena and Reggio Emilia, Italy, matteo.paganelli@unimore.it; Matteo Lissandrini, University of Verona, Italy, matteo.lissandrini@univr.it; Felix Naumann, Hasso Plattner Institute, University of Potsdam, Germany, felix.naumann@hpi.de; Giovanni Simonini, University of Modena and Reggio Emilia, Italy, giovanni.simonini@unimore.it.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2025 Copyright held by the owner/author(s).
ACM 2836-6573/2025/6-ART228
https://doi.org/10.1145/3725365

R	Monument	City	Country	Year
r_1	TV Tower	Berlin	Germany	1965-1969
r_2	Big Ben	London	England	1843-1869
r_3	St. Davids	-	England	1915-1921
r_4	Colosseum	Rome	Italy	70-80
r_5	AquaClaudia	Rome	Unknown	52
r_6	Well of Life	Zagreb	Croatia	Unknown

S	Nation	Population	City
s_1	England	8,866,180	London
s_2	Italy	2 752 908	Rome
s_3	England	1 348	St.Davids
s_4	Unknown	52	Hum
s_5	Croatia	Unknown	Rastoke
s_6	Germany	3.755.251	Berlin

Fig. 1. Pair of tables that overlap in different ways. ($\langle$"*Unknown*", "52"$\rangle$, $\langle$"*Croatia*", "*Unknown*"$\rangle$) has area four and is the largest overlap obtainable without reordering rows and columns, while ($\langle$"*Berlin*", "*Germany*"$\rangle$, $\langle$"*Rome*", "*Italy*"$\rangle$, $\langle$"*London*", "*England*"$\rangle$) is the real largest overlap, it has area six and is obtained through a permutation of rows and columns. The overlap ratio is 6/18 (largest overlap area / area of the smaller table).

few columns may denote a pair of joinable tables. Similarly, a *horizontal* overlap can be a sign of unionability. Further, an overlap covering many rows and columns might represent a case of duplicate tables, table containment, or different versions of the same table [3].

Detecting overlaps is a challenging task for traditional set-based methods for data discovery, as it requires considering the alignment of cell values in a table and the structure of the tables themselves. To fill this gap, the concept of *largest overlap*, i.e., the largest common *rectangular* subtable existing between two tables under any row and column reordering, was recently introduced [54]. We present an illustrative example of largest overlap in Figure 1.

Identifying tables for which the largest overlap exceeds a certain area and/or presents a specific shape finds application in multiple real-world scenarios, from the detection of largely overlapping tables in a data lake, e.g., for subsequent operations of change propagation [3] or elimination of redundant information, to more specific tasks, e.g., the detection of copying dependencies across multiple sources [35] or the automated discovery of multi-column joins [54]. In addition, table overlap not only serves user-facing applications, but can also be used as a filtering or blocking method for downstream tasks, such as table reclamation [16].

In practice, all proposed use cases fall under two main scenarios: *(i) table matching*, i.e., retrieving all the clusters of related tables in a corpus, and *(ii) table querying*, i.e., given a *query table*, identifying a set of related tables in a corpus by building a ranking or using a similarity threshold. Since the first scenario requires comparing all possible pairs, it has a quadratic complexity, and even when working with datasets of a few thousand elements, a large number of comparisons needs to be performed. The second, instead, is linear with respect to the number of tables in the corpus. However, it can be equally challenging if computed in a brute-force manner, indeed Sloth [54], the only prior approach available for estimating the largest table overlap, takes more than two hours of machine time to compare 1 table with a data lake of 10K tables. Thus, in this paper, we analyze the problem of efficiently approximating the *maximum normalized table overlap*, referring to it as *overlap ratio*.

1.1 Table overlap ratio

We define the overlap ratio between two tables by extending the definition of the largest table overlap [54]. The intuition is to identify the sub-tables of maximum area, in terms of number of cells, that are shared between the two tables and normalize their area with respect to the area of the smaller table.

Given two tables R and S defined on the schemas X and Y respectively, we first define an attribute mapping M as the bijective function that links a subset of columns of the first table $X_M \subseteq X$ to another set $Y_M \subseteq Y$ of the second table. Each mapping determines a table overlap O_M, which corresponds

to the sub-table obtained from the intersection under bag semantics of the tuples in the columns linked by the mapping. We call the overlap area for the mapping M the table area of O_M, i.e., the number of cells contained in the table overlap generated by the mapping M. Among all the possible mappings, we are interested in identifying the ones with the maximum overlap area.

Example 1 (Largest table overlap). *Given the tables R and S in Figure 1 it is possible to identify several attribute mappings. Just to list a few, the mapping* M_1:⟨*Year*⟩→⟨*Population*⟩ *generates an overlap area of two, i.e., only two cells are shared between the two columns. The mapping* M_2:⟨*City, Country*⟩→⟨*City, Nation*⟩ *instead, represents a mapping of area six and since there are no other mappings that generate a larger area, it is a largest table overlap.*

Formally, we call $O(R, S)$ the set of table overlaps determined by all possible attribute mappings between R and S. We instead identify with $O^*(R, S) \subseteq O(R, S)$ the set of table overlaps with the largest overlap area, and their respective areas with $A^*(R, S)$.

Definition 1.1 (Overlap ratio). Let R, Y be tables defined on the schemas X, Y, A_R and A_S their table areas, and $A^*(R, S)$ their largest overlap area. We call

$$\theta(R, S) = \frac{A^*(R, S)}{min(A_R, A_S)} \in [0, 1]$$

overlap ratio between the tables R and Y.

Example 2 (Overlap ratio). *Table R and S have an area of* 6×4 *and* 6×3 *respectively, and their largest table overlap, deriving from* M_2*, is 6. Their overlap ratio is* $\theta(R, S) = \frac{6}{min(6\times4, 6\times3)} = \frac{6}{18} = 0.33$.

1.2 Efficient overlap ratio approximation

Although Sloth [54] provides a method to compute the largest overlap between pairs of tables, it has serious scalability issues. A possible approach to solving this problem is to consider methods that further approximate this calculation, such as considering tables as flattened sets of values and adapting a set-wise similarity measure for this purpose. The *Jaccard Similarity* [27] is generally used to compare textual data and defines the similarity of sets of elements as the ratio between the cardinality of their intersection and their union. It can be used to approximate the overlap ratio by finding the intersection under set semantics of values in a pair of tables and dividing their cardinality by the cardinality of their union. Since this approach uses the set semantics, it does not take into account the repetition of values in the tables. A solution to the problem is computing the Jaccard similarity under bag semantics, i.e., considering tables as bags of values that allow repetitions and dividing the cardinality of the bag intersection by the cardinality of the bag union. Since the number of values in the bag intersection is at most equal to the sum of the sizes of the two bags, it can predict at most a value of 0.5, hence, it cannot be used as it is. Two solutions are (1) scaling its result to the right interval by multiplying its prediction by two or (2) dividing the cardinality of the bag intersection by the size of the smaller bag.

Nonetheless, none of these approaches consider the structure of the tables and thus they fail when the table pairs have more than one common sub-table. In particular, the last method assumes that all values in the bag intersection are part of the same largest common subtable, always predicting an overlap greater or equal to the real one.

Example 3 (Approximate overlap ratio). *For the tables in Figure 1, the Jaccard similarity would predict an overlap of* $\frac{10}{27} = 0.37$ *under set semantics, an overlap of* $2 \cdot \frac{12}{42} = 0.57$ *under scaled bag semantics, and* $\frac{12}{18} = 0.67$ *under bag semantics normalized by the smaller table area. The true value of the overlap ratio between the tables is* $\frac{6}{18} = 0.33$*, and is overestimated by all methods.*

Data Discovery in Data Lakes: Operations, Indexes, Systems

Ziawasch Abedjan
BIFOLD & TU Berlin
Berlin, Germany
abedjan@tu-berlin.de

Mahdi Esmailoghli
HU Berlin
Berlin, Germany
mahdi.esmailoghli@hu-berlin.de

Sainyam Galhotra
Cornell University
Ithaca, USA
sg@cs.cornell.edu

ABSTRACT

Data discovery has gained significant traction in the database community resulting in various discovery operations, index schemes, and discovery systems. This tutorial explores the architecture and components of data discovery systems, focusing on indexing structures and scalable algorithms for typical operations, such as join and union discovery. While giving insights into individual algorithms, we point out open challenges for holistic systems, data discovery evaluation, and discovery in federated setups.

PVLDB Reference Format:
Ziawasch Abedjan, Mahdi Esmailoghli, and Sainyam Galhotra. Data Discovery in Data Lakes: Operations, Indexes, Systems. PVLDB, 18(12): 5455 - 5459, 2025.
doi:10.14778/3750601.3750694

1 INTRODUCTION

Data lakes, whether intentionally created or emerging as a byproduct of organizational processes or open data initiatives, share a common challenge: they are often underspecified and underdocumented [1, 13]. Unlike curated data warehouses, where the schema and purpose are clearly defined, data lakes typically lack predefined objectives. Despite this, there is significant interest in leveraging these repositories for augmenting in-use datasets or pure exploratory purposes. To support such applications, data discovery as a process becomes essential, enabling users to interact with the data lake not by relying on detailed schema knowledge but by using search and matching techniques. By searching with user-provided keywords or table artifacts, users aim to uncover datasets that are relevant to their use cases.

Data discovery is currently an active field of research, as building scalable solutions to effectively capture user intent, such as augmenting a given dataset fragment, presents various challenges. A key issue is the need to index thousands to millions of tables for fast value look-ups and efficient alignment to evaluate joinability [9, 35] and unionability [19] with the user (query) table. After identifying relevant tables, assessing the usefulness of the retrieved results remains a complex problem. Often, users rely on proxy metrics, such as the downstream machine learning performance of the augmented dataset, to assess its effectiveness [7, 8, 22]. Finally, in distributed data lakes, additional constraints, such as privacy regulations and pricing structures, further complicate the process. In such cases, the usefulness and relevance of datasets must often be assessed based on available metadata and summaries, rather than direct access to the full datasets.

This tutorial is based on our previous study of data discovery use cases and our experience in building systems and algorithms that address some of its emergent challenges. Our goal is to give a structured overview of data discovery software components and to dive into technical aspects that differentiate individual prototypes and approaches. A meta-goal is to bridge the language and perspective differences on what is counted as data discovery and how discovery systems in database research can be compared. Figure 1 depicts a conceptual architecture of data discovery systems that we explore in this tutorial. The user formulates a discovery task that typically aims at enhancing an existing table or table fragment through additional features, rows, or individual values. Such tasks are then typically carried our on top of common operations, such as keyword search, join discovery, union discovery, and others that are built on top of dedicated indexes. In our tutorial, we cover the depicted architecture as follows. We first survey and discuss common data discovery use cases from industry and academia. We then present a classification of data discovery systems with layers that distinguish the locality, languages, and index structures of such systems. Locality captures the accessibility of repositories on monoliths and in distributed settings. Discovery languages can differ depending on how many different discovery tasks a system covers. Accordingly, we then dive deep into concrete index structures and how they serve individual discovery operations and languages.

Previous Tutorials: There have been similar tutorials in the past that relate to ours. In particular, the tutorial by Paton and Wu at EDBT 2024 [21] also covers various aspects of dataset search and navigation. In contrast to their tutorial, we have a strong focus on algorithmic details on how index structures and operations have evolved and can be categorized with a projection on how these approaches relate, can extend to distributed scenarios, and can be combined inside holistic systems. There has been a tutorial from the information retrieval perspective on web table extraction [32]. Our tutorial covers more recent advances and discusses the topic from the database management perspective. The tutorial on data exploration [15] is related but does not focus on data lakes.

2 TUTORIAL OUTLINE

In this tutorial, we first discuss the types of data lakes that set the motivation for building data discovery systems. We further categorize existing systems depending on their scope with regard to data locality, i.e., centralized and distributed repositories. We then detail common operations and fundamental indexing schemes in state-of-the-art that support such operations. Finally, we will discuss methods for evaluating and benchmarking data discovery systems and outline open problems and potential future research.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 18, No. 12 ISSN 2150-8097.
doi:10.14778/3750601.3750694

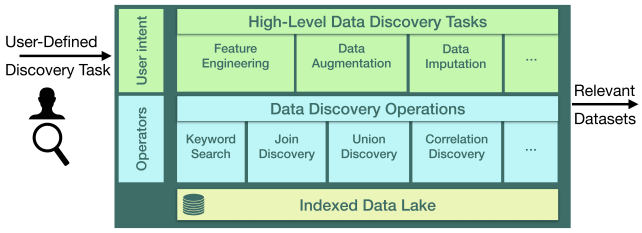


Figure 1: Vanilla data discovery system architecture

2.1 Data Lakes

With the awareness of how valuable data can be even without being currently under active usage, organizations started to accumulate datasets and make them accessible across silos. This is in contrast to the typical data warehousing approach, where handcrafted extract-transform-load pipelines are defined and documented to integrated datasets into a purposeful schema. Data lakes in contrast follow the integrate at application time paradigm. Often times the datasets not only are heterogeneous on schema level but can be subject to heterogenous data models or reside in heterogeneous storage systems. For the sake of scoping, we will focus on the algorithms and systems that have been designed for data lakes with tabular datasets.

2.2 Overview of Data discovery settings

Data discovery is a fundamental capability for organizations that manage large and often distributed collections of datasets. The need for robust data discovery systems has become even more pressing with the widespread adoption of machine learning and AI techniques, which depend critically on access to high-quality, relevant data. Numerous studies have explored different approaches to data discovery, with the optimal solution often varying based on the specific use case. This section provides an overview of the key architectural and interaction paradigms for data discovery.

Centralized vs. Federated Settings. Traditional data discovery systems typically assume *centralized access* to data, where all datasets are consolidated into a single repository, allowing the discovery engine to index and query them directly. However, this assumption is often impractical. In many cases, data is often dispersed across multiple repositories due to concerns related to data sovereignty, privacy regulations, and the high cost of data transfer.

To address these challenges, modern data ecosystems are shifting toward *federated* or *decentralized* discovery architectures. In such settings, data providers retain control over their datasets and share only metadata (or a summary of the dataset) with the discovery system. This model is common in commercial data marketplaces (e.g., Datarade, Dawex) and federated infrastructure initiatives (e.g., Gaia-X, Agora), where discovery must be performed over incomplete information, and access to data is negotiated only after identifying relevant resources.

Data Discovery Tasks and Use Cases. Users express their information needs in various ways, depending on the context and their familiarity with the data. Common interaction modalities include:

- **Keyword-based Discovery:** Users input textual queries along with optional filters. While straightforward and user-friendly, this method is often inadequate when users have nuanced requirements related to data semantics, coverage, or statistical characteristics.
- **Query-by-Example:** Users provide a small sample dataset and search for similar datasets based on structure or content. This approach is helpful when users struggle to verbalize their data needs but can illustrate them via examples.
- **Task-based Discovery:** Users specify a target task, such as training a classifier or evaluating a model, and seek datasets that are suitable for that purpose. This mode is especially relevant for machine learning workflows, where factors like class balance and feature alignment significantly impact performance.

In many real-world scenarios, users may only have a vague or evolving notion of what constitutes a “useful" dataset. Their needs may not be easily captured by keywords, examples, or tasks. To address this, recent work has focused on designing more expressive query languages that allow users to specify fine-grained predicates—such as schema constraints, statistical distributions, or semantic criteria—to better guide the discovery process.

2.3 Discovery Operations

Our tutorial will strongly focus on two major classes of operations that are common in data discovery research: Joins and Unions. note that each of the two can inhibit additional properties, such as fuzziness or conditional utility functions for the correspondingly joinable and unionable tables, as it is relevant for example for task-based discovery tasks. In our tutorial, we will define both operations and discuss the variations of each.

A join discovery operation aims at discovery tables that are joinable with a given user table. In the literature, variations, such as precalculated PK-FK relationships, single column joinability and multi-column joinability, as well as fuzzy joins have been considered. In our tutorial, we will cover the necessary algorithmic steps for each of the aforementioned variations, as well as their use cases and disadvantages. Furthermore, we will dive deep into the class of conditional join discovery approaches, where not only the joinability but also the utility of the obtained tables with regard to downstream tasks have been considered.

A union discovery operation aims at vertically expanding a table with more rows from tables inside a lake. Within this line of work, we will distinguish approaches that check for perfect unionability as in consistent with UNION in SQL as well as approaches that find partially unionable tables.

2.4 Indexing Data Lakes

An efficient index structure is necessary to support any of the aforementioned discovery operators. Index structures have long served as the foundation of traditional information systems, enabling efficient search and retrieval. In the era of large-scale data lakes and advanced data discovery techniques, these structures have evolved to meet two primary objectives: *scalability* and *customizability*. The large size and heterogeneity of data lakes require indexes that can efficiently scale to large volumes of data while supporting various

Table 1: Mapping between indexes and data discovery studies.

Index		Papers
Value (Cell)	Value → Location	[2, 4, 24, 29–31, 35]
	Location → Value	[2, 30, 35]
	Value → Class (Domain)	[24]
	Value → Embedding	[5]
Column	LSH Variations	[3, 5, 12, 14, 19, 28, 36]
	Column → Metadata	[2, 14, 24, 30, 35]
	Column Names → tables	[31]
	Column → Domain	[34]
	Column → Rank	[22, 23]
	Column → Embedding	[5, 11, 27]
Row	Row → Table	[2, 31]
Table	Table → Embedding	[31]
	Table → Metadata	[2]
Graph	V: Columns, E: Similarities	[4, 12, 16–18, 20, 26, 31, 33, 34]

custom search requirements. For instance, data discovery with the goal of benefiting a downstream machine learning (ML) task requires a task-specific similarity measure to find the most relevant data. In this example, the relevancy of a column is measured by how significantly the column correlates with a target column to be predicted in the ML task. These task-specific requirements demand customized index structures, leading to hybrid and more complex indexes.

Index structures for data discovery in data lakes can be categorized based on two key dimensions: query interface, determining the primary way users interact with the index, and search approximation, defining whether the index allows exact or approximate discovery.

Table 1 provides an overview of the state-of-the-art index structures, grouped into six categories based on their query interface: Value-based indexes, column-based indexes, row-based indexes, table-based indexes, and graph-based indexes.

Each of these index types serves a distinct purpose. Value-, column-, row-, and table-based indexes enable querying data lakes at different levels of granularity, from individual cell values to entire tables. Graph-based indexes facilitate exploratory search by capturing relationships between datasets, allowing users to navigate table connections.

Index structures can also be categorized based on whether they provide exact or approximate, i.e., fuzzy discovery. Exact indexes guarantee exact retrieval of the most relevant tables to the task at hand but may be computationally expensive. Examples of exact indexes are traditional inverted indexes. Approaches such as Josie [35] and MATE [9] benefit from exact indexes to find equi-joinable tables to a given key column. On the other hand, approximate indexes prioritize efficiency by sacrificing accuracy.

Hash- and sketch-based techniques have been widely adopted to improve scalability. For example, LSH-based indexes [6, 14, 36] are used for efficiently finding tables based on Jaccard, containment, and cosine similarities. However, significant research efforts focus on improving their accuracy and scalability, leading to variations such as LSH Ensemble [36], which reduces bias towards smaller tables, or Lazo [14], which uses OOPH sketches [25] to reduce the number of hashes required before constructing the LSH index. Sketch-based indexing techniques, such as Quadrant Count Ratio (QCR) [23], have also been employed to find correlating features to a target column.

High-dimensional vector indexes, such as HNSW, Inverted File Index (IVF), and Product Quantization (PQ), have recently been used to enable approximate data discovery based on semantic similarity between tables [11].

2.5 Evaluating Discovery Systems

One of the major challenges in data discovery research is the appropriate evaluation of proposed methods. Similar to other research problems in data integration, there is always a trade-off between resource consumption and quality of potential techniques. While measuring resource consumption and runtime is straightforward, it is often hard to set the proper criteria for the expected quality of data discovery results. In this tutorial, we want to delve into these challenges and discuss two different approaches that are currently carried out by researchers: 1) Close-world: A ground truth is created, often by fragmenting larger tables and trying to resynthesize the original tables. With this approach, metrics such as precision and recall can be calculated. 2) Open-world: Data discovery aims at finding serendipitous datasets that are useful for a downstream use case. Both approaches can bring insights into the capabilities of different techniques. In our tutorial, we want to highlight that the implication of each is different, which in turn might favor a certain class of techniques.

2.6 Future Directions

As our tutorial will show, research on data discovery has made significant progress in the past decade. Nevertheless, there are aspects that are still underexplored and impede the usage of data discovery solutions in practice. Among those, we will discuss the challenges in building holistic systems [10] that serve a variety of discovery tasks, data discovery in federated setups and under constraints as well as the expansion of existing techniques for additional modalities beyond relational tables.

3 TUTORIAL ORGANIZATION

We plan to carry-out the tutorial as an interactive lecture with live demonstration of discovery operations and systems. As several of the systems have been developed and reproduced at our labs, we can provide practical insights into several of the presented techniques. Overall, the structure as outlined in the previous section can be covered within 3 hours. We can trim the tutorial for a shorter session if needed, by focusing on fewer discovery operations.

Target audience. The topic of data discovery has a broad appeal within the data management community. We believe that anyone who is interested in data discovery, security, and privacy of data would be interested in this tutorial. More broadly, the tutorial is aimed at encouraging the DB community to work on novel problems in data discovery and make it more practical. All materials used will be released publicly, and the attendees will be given a hands-on experience to test the presented techniques.

Prerequisites. The background expected is that of an introductory course on data management. The tutorial has been carefully structured to accommodate both attendees unfamiliar with the topic and experienced participants by providing the required background knowledge and shared terminology.

GitTables: A Large-Scale Corpus of Relational Tables

MADELON HULSEBOS*, University of Amsterdam, Netherlands
ÇAĞATAY DEMIRALP, Sigma Computing, United States
PAUL GROTH, University of Amsterdam, Netherlands

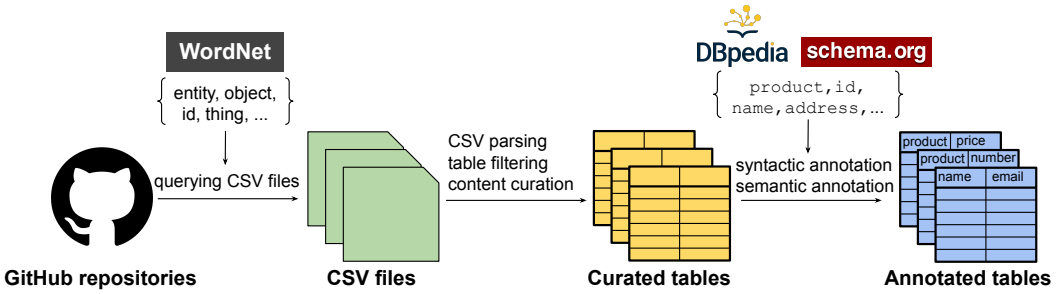


Fig. 1. The pipeline for creating GitTables consists of 1) querying CSV files from GitHub repositories based on WordNet, 2) parsing CSV files and curating tables, and 3) annotating tables with column semantics from DBpedia and Schema.org.

The success of deep learning has sparked interest in improving relational table tasks, like data preparation and search, with table representation models trained on large table corpora. Existing table corpora primarily contain tables extracted from HTML pages, limiting the capability to represent offline database tables. To train and evaluate high-capacity models for applications beyond the Web, we need resources with tables that resemble relational database tables. Here we introduce GitTables, a corpus of 1M relational tables extracted from GitHub. Our continuing curation aims at growing the corpus to at least 10M tables. Analyses of GitTables show that its structure, content, and topical coverage differ significantly from existing table corpora. We annotate table columns in GitTables with semantic types, hierarchical relations and descriptions from Schema.org and DBpedia. The evaluation of our annotation pipeline on the T2Dv2 benchmark illustrates that our approach provides results on par with human annotations. We present three applications of GitTables, demonstrating its value for learned semantic type detection models, schema completion methods, and benchmarks for table-to-KG matching, data search, and preparation. We make the corpus and code available at https://gittables.github.io.

CCS Concepts: • **Information systems** → *Relational database model*; *Information integration*; • **Computing methodologies** → *Information extraction*; *Neural networks*.

Additional Key Words and Phrases: relational tables, data management, deep learning, datasets

ACM Reference Format:
Madelon Hulsebos, Çağatay Demiralp, and Paul Groth. 2023. GitTables: A Large-Scale Corpus of Relational Tables. *Proc. ACM Manag. Data* 1, 1, Article 30 (May 2023), 17 pages. https://doi.org/10.1145/3588710

*Corresponding author (m.hulsebos@uva.nl)

Authors' addresses: Madelon Hulsebos, University of Amsterdam, Amsterdam, 1012 WX, Netherlands, m.hulsebos@uva.nl; Çağatay Demiralp, Sigma Computing, San Francisco, CA 94105, United States, cagatay@sigmacomputing.com; Paul Groth, University of Amsterdam, Amsterdam, 1012 WX, Netherlands, p.t.groth@uva.nl.

This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2023 Copyright held by the owner/author(s).
2836-6573/2023/5-ART30
https://doi.org/10.1145/3588710

1 INTRODUCTION

Deep learning (DL) models, in the past decade, have dramatically improved many long-standing computer vision and natural language processing tasks [25]. The practical success of DL has also spurred interest in its applications to tasks in other domains, including data management, ranging from data cleaning to annotation [45]. To train DL models for relational data, earlier work primarily relied on corpora consisting of tables scraped from HTML pages [10] such as WDC WebTables [46], the largest table corpus to date. These models have played an important role in facilitating data-driven data management research, with Web applications in particular [7, 49].

However, tables extracted from HTML pages on the Web (Web tables) provide a skewed representation of tables in the wild especially those residing in (enterprise) databases [24, 27]. The common attribute "ID", for example, does not appear among the twenty most frequent column names of the WDC WebTables corpus, and the dimensions of Web tables are significantly smaller than typical database tables [46]. It is therefore unsurprising that models based on Web tables have limited applications beyond the Web [7]. To broaden the impact of data-driven data management research, we need new table collections complementing existing corpora with tables resembling typical database tables.

To address this demand, we introduce GitTables: a corpus with 1M relational tables extracted from CSV files in GitHub repositories. We will keep the corpus growing to have at least 10M tables to facilitate the extension of deep transfer learning to the relational domain, analogous to how large corpora of natural language have stimulated pretrained language models like BERT [16] and GPT [6]. We expect GitTables to stimulate similar progress in data management tasks like data search and preparation.

Besides having representative tables, many data management applications benefit from understanding the semantics of table columns. An intelligent data exploration system, for example, could recommend a map chart to visualize two data columns representing countries and sales revenues. Upon encountering missing values, the system could also automatically fill the gaps using a knowledge base. The system could, for instance, impute "France" in an empty cell in a column with country values, if the neighboring cell in a column with capital cities is "Paris".

To facilitate such functionalities, we enhance table columns with semantic types from DBpedia and Schema.org (e.g. `address`) using a syntactic as well as semantic annotation method (an example in Figure 2). Each semantic type is also associated with the expected atomic data type (e.g. `string` and `number`), a description, and hierarchical type relations. These semantically rich annotations together with DL present a unique opportunity to learn table semantics as demonstrated with TURL [15] and TaBERT [48].

Our analysis of GitTables confirms the different nature of the tables: the tables have significantly larger dimensions (rows and columns). A machine learning model for detecting data shift between GitTables and Web tables accurately distinguishes table columns from each source corpus. This reflects the structurally different content of these corpora. The semantic type distribution also deviates significantly from the semantic distribution of Web tables, illustrating its distinctive topical coverage. We make GitTables available through https://gittables.github.io. We contribute:

(1) GitTables, a new large-scale corpus of 1M tables. To the best of our knowledge, GitTables is the first large-scale relational table corpus with topical coverage and content structurally different than tables extracted from HTML pages.
(2) A scalable column-annotation method using distant-supervision. We annotate the columns in GitTables with semantics consisting of semantic types, atomic data types, hierarchical relations, and descriptions, making GitTables the largest annotated table corpus to date.

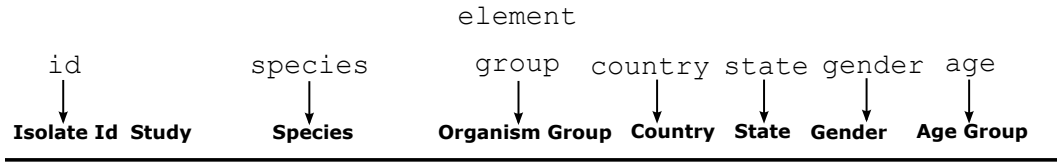


Fig. 2. Snippet of an annotated table in GitTables. Annotations are provided with confidence scores. For example, the confidence of the type `species` for the column "Species" is 1, while `element group` for "Organism Group" is 0.7. These scores enable filtering based on the intended use case.

Table 1. Existing large-scale relational table corpora in comparison with GitTables which has considerably higher dimensions, resembling actual tables in databases. We report statistics related to relational tables, leaving out, for example, entity tables.

Name	Table source	# tables	Avg # rows	Avg # cols
WDC WebTables [26]	HTML pages	90M	11	4
Dresden Web Table Corpus [17]	HTML pages	59M	17	6
WikiTables [3]	Wikipedia tables	2M	15	6
Open Data Portal Watch [29]	CSVs from Open Data portals	107K	365	14
VizNet [20]	WebTables, Plotly, i.a.	31M	17	3
GitTables	**CSVs from GitHub**	**1M**	**142**	**12**

(3) Three applications demonstrating the value of GitTables: 1) we train a semantic type detection model and obtain high prediction accuracy, 2) we use GitTables as a resource for schema completion methods in typical database contexts, and 3) we present a starting point for benchmark datasets for typical data management tasks.

2 RELATED WORK

Web initiatives such as Common Crawl, Wikipedia, and Open Data have been cost-effective resources for curating unstructured and structured data at scale [3, 31, 32]. Below we discuss large-scale table corpora sourced from these initiatives and review prior work annotating column semantics of tables sampled from these corpora.

2.1 Large-scale table corpora

WDC WebTables and Dresden Web Table Corpus. These Web table corpora [17, 26] extract tables from HTML pages in the Common Crawl corpus as inspired by [8]. They provide an abundance of relational tables ranging from 59M to 90M and have been instrumental in advancing applications like table augmentation and integration [7, 49]. However, Web tables generalize poorly due to their small dimensions and different content [24].

WikiTables. To provide high-quality tables with semantics that are easier to detect than those of arbitrary Web tables, WikiTables extracts approximately 2M tables from Wikipedia [3]. This corpus is primarily suitable for tasks such as question answering that rely on the quality of the table contents. Unsurprisingly, the tables in WikiTables are just as small as those in WDC WebTables.



LakeBench: A Benchmark for Discovering Joinable and Unionable Tables in Data Lakes

Yuhao Deng
dyh18@bit.edu.cn
Beijing Institute of Technology

Chengliang Chai
ccl@bit.edu.cn
Beijing Institute of Technology

Lei Cao
lcao@csail.mit.edu
University of Arizona/MIT

Qin Yuan
yuanq1020@gmail.com
Beijing Institute of Technology

Siyuan Chen
siyuanchen@bit.edu.cn
Beijing Institute of Technology

Yanrui Yu
yanruiyu@bit.edu.cn
Beijing Institute of Technology

Zhaoze Sun
sunzhaoze@bit.edu.cn
Beijing Institute of Technology

Junyi Wang
wangjunyi@bit.edu.cn
Beijing Institute of Technology

Jiajun Li
lijiajun@bit.edu.cn
Beijing Institute of Technology

Ziqi Cao
caoziqi@bit.edu.cn
Beijing Institute of Technology

Kaisen Jin
jks@bit.edu.cn
Beijing Institute of Technology

Chi Zhang
zhangchi315@bit.edu.cn
Beijing Institute of Technology

Yuqing Jiang
jiangyuqing@bit.edu.cn
Beijing Institute of Technology

Yuanfang Zhang
yfzhang@bit.edu.cn
Beijing Institute of Technology

Yuping Wang
wyp_cs@bit.edu.cn
Beijing Institute of Technology

Ye Yuan
yuan-ye@bit.edu.cn
Beijing Institute of Technology

Guoren Wang
wanggr@bit.edu.cn
Beijing Institute of Technology

Nan Tang
nantang@hkust-gz.edu.cn
HKUST (GZ)

ABSTRACT

Discovering tables from poorly maintained data lakes is a significant challenge in data management. Two key tasks are identifying joinable and unionable tables, crucial for data integration, analysis, and machine learning. However, there's a lack of a comprehensive benchmark for evaluating existing methods. To address this, we introduce LakeBench, a large-scale table discovery benchmark. It evaluates effectiveness, efficiency, and scalability of table join & union search methods. With over 16 million real tables, LakeBench is **1,600X** larger than existing datasets and **100X** larger in storage size. It includes synthesized and real queries with ground truth, totaling more than 10 thousand queries – **10X** more than used in any existing evaluation. We spent over 7,500 human hours labeling these queries and constructing diverse query categories for thorough evaluation. Our benchmark thoroughly evaluates state-of-the-art table discovery methods, providing insights into their performance and highlighting research opportunities.

PVLDB Reference Format:
Yuhao Deng, Chengliang Chai, Lei Cao, Qin Yuan, Siyuan Chen, Yanrui Yu, Zhaoze Sun, Junyi Wang, Jiajun Li, Ziqi Cao, Kaisen Jin, Chi Zhang, Yuqing Jiang, Yuanfang Zhang, Yuping Wang, Ye Yuan, Guoren Wang, and Nan Tang. LakeBench: A Benchmark for Discovering Joinable and Unionable Tables in Data Lakes . PVLDB, 17(8): 1925-1938, 2024.
doi:10.14778/3659437.3659448
PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/RLGen/LakeBench.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 17, No. 8 ISSN 2150-8097.
doi:10.14778/3659437.3659448

1 INTRODUCTION

Many governments have acknowledged the importance of providing open access to their data for the public good [30]. However, these open datasets are typically published in formats like image or JSON that don't fit neatly into relational databases. To make sense of this poorly maintained data, data scientists have to go through a pipeline of data curation tasks to discover, merge, and clean data [5, 9–11, 31]. As the first step of this pipeline, data discovery that finds relevant tables is critical for data scientists to effectively and efficiently conduct their work.

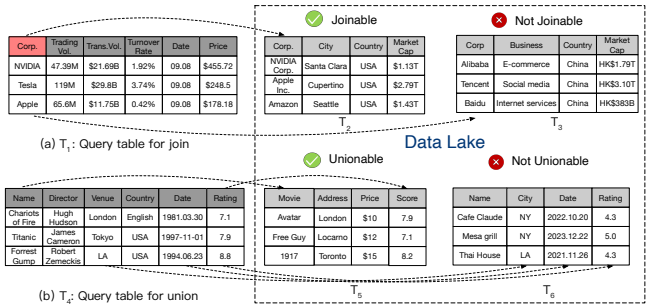


Figure 1: Table Discovery in Data Lake.

Table discovery from data lakes can be generally classified into two categories: keyword-based search [4, 16, 19, 20, 40] and table-based search [3, 13, 14, 17, 24, 32, 45, 46]. Keyword-based search over

tables often relies on the metadata of tables in the lake to identify tables or tuples relevant to the given keywords. However, in the data lake scenario, the metadata is often poorly maintained compared to the database management system (DBMS). Consequently, the incompleteness and inconsistency of metadata in the data lake become bottlenecks for keyword-based search. Thus in this work, we focus on the table-based search that can be further categorized into table join search and table union search. Recently, table-based search has attracted much attention in data management field to augment data instances [7, 27, 43] or attributes [6, 8, 26, 28, 39]. This benefits many downstream applications such as machine learning based data analysis.

Existing Works. To the best of our knowledge, no work has specifically focused on constructing a benchmark for the thorough evaluation of the existing table search methods. Although some works that target inventing new table search methods have done a great job in releasing their datasets and queries used in their evaluation, the scale of their datasets, the number of queries, and the diversity of the workload are not sufficient to evaluate the effectiveness, scalability, and efficiency of the table search methods in diverse scenarios. In particular, TUS [32] and Santos [24] evaluate their *table union search* methods with 1,000 queries on *small* data lakes which contain at most 11,090 tables – in total 11GB data, far from being sufficient to evaluate the scalability of the search algorithms and simulate the diverse real applications. Josie [45], targeting *table join search*, releases 1,000 queries to evaluate if their method is able to efficiently find the tables that have exact overlap with the query tables. In this setting, they do not have to label their queries, e.g., manually discover the tables joinable with the query tables. Therefore, these queries cannot be used to evaluate the effectiveness of other table join search methods that consider the semantic similarity of the data.

Design Goals. To fill this gap, in this work we aim to construct a comprehensive benchmark for table discovery, following the design goals below:

(1) *Scalability.* To thoroughly evaluate the efficiency and scalability of different methods, the benchmark has to involve data lakes of various sizes, especially large-scale ones. The size of the data lake is measured from two aspects: the storage size and the number of total columns. The storage size is highly related to the average table size and the number of tables. The number of total columns matters as well because almost all table discovery algorithms index and search over a large number of columns.

(2) *Variety.* The performance of table discovery largely relies on the characteristics of the datasets and queries. In particular, one algorithm might perform differently on different categories of queries. For example, given a query, a semantic-aware algorithm will show clear advantages if the table semantic matters the most in determining its unionable tables. Hence, plenty of diverse queries are critical to thorough evaluation.

(3) *Comprehensive comparison of solutions.* Understanding the pros/cons of existing table search methods and revealing research opportunities, requires a reliable implementation of all these methods, a thorough and fair experimental evaluation, and a deep analysis of the results.

Our Proposal. Guided by the above three design goals, we construct a benchmark, LakeBench– the first of its kind that supports the thorough evaluation of table union & join methods.

To achieve the first design goal, we collect over 1TB data – **100X** larger than the data lakes used in existing works, and the number of columns is up to **100 million**, on which we evaluate different table union/join search methods. Moreover, we label a sufficient number of diverse queries (more than 10 thousand) – **10X** more than any other works. To this end, we organize a team of 25 graduate students and spent more than *7,500 human hours* on labeling the queries. Because it is prohibitively expensive to manually identify tables in the data lake that are joinable/unionable to a given query, we design a candidate generation strategy to prune the tables that are most unlikely to be the ground truth.

To meet the second design goal, we construct data lakes with diverse characteristics by collecting tables from two data sources, namely OpenData [1] and WebTable [2]. WebTable has a large number of tables (more than 10^8) but each table is small. In contrast, OpenData consists of large tables (on average 1GB in size per table) but has fewer tables than WebTable. Besides, we generate both synthetic (splitting from large lake tables) and real queries (directly sampling from the data lake), which have different properties. We further categorize these queries at finer granularity w.r.t. the key factors that determine the quality of table search.

For the third design goal, we implement many key techniques such as column embeddings, approximate nearest neighbor (ANN) search index, etc., which are the building blocks of these approaches, ensuring that we are able to evaluate the scalability and accuracy of various approaches on different datasets and mostly reuse the code base. More importantly, we evaluate these approaches over the aforementioned fine-grained query categories, so as to reveal multiple insights w.r.t. the performance of different approaches over different categories. In addition, our analysis of the time and space complexity of the search algorithm connects the experiment results to theory, guiding our discussions on the results. In short, LakeBench makes it possible to thoroughly evaluate and understand existing table search methods.

Contributions. We make the following contributions:

(1) We build a comprehensive benchmark, LakeBench, for table discovery, featuring large-scale datasets and a sufficient number of diverse queries. Besides, it addresses critical limitations in existing benchmarking datasets. To be specific, *i*.) The lack of a specific dataset for joinable search with ground truth; *ii*.) The number of ground truth tables for individual queries is small, particularly notable in TUS and SANTOS, thus problematic in accurately assessing recall; *iii*.) Relying heavily on automatically generated ground truth, which may simplify the task and potentially miss ground truth because of lacking sufficient human annotation.

(2) We collect real tables (more than 1 TB in size) from multiple sources, which consist of 4 data lakes with various storage sizes and different numbers of table columns.

(3) We create 10,000+ table queries covering different characteristics and spend 7,500+ human hours on accurately labeling them.

(4) We thoroughly evaluate and analyze the state-of-the-art table join/union search approaches using these created queries on the benchmark datasets. Based on that, we reveal multiple insights and research opportunities. For example, we observe that although

the methods that fine-tune pre-trained language models in general show an advantage over other methods, their accuracy is still not very satisfactory in some cases. Therefore, designing better fine-tune strategies or fine-tuning large language models (LLM) might be a promising direction to go in the future.

2 PRELIMINARY

2.1 Problem Definition

We formally define the two table discovery tasks in a data lake.

Table Join Search. Suppose that a data lake $\mathcal{T}$ contains a large set of tables $\mathcal{T} = \{T_1, T_2, ..., T_N\}$, where each table $T_i, i \in [1, N]$ has n_i rows (tuples), m_i columns (attributes) and each cell value is denoted by c_{ij}. Given a query table T_q^J, as well as a specific column C_q^J of T_q^J, table join search is to find the target tables that can be joined with T_q^J on C_q^J, and we note the measure of the relevance score between two columns as $R(C_q^J, C_t)$, where C_t is a column of a target table $T_t \in \mathcal{T}$. The higher the score, the more likely the two columns can be joined (In this case, we can say that the two columns or two tables can be joined interchangeably), *i.e.*, $R(C_q^J, C_t)$ can also be represented as $R(C_q^J, T_t)$. Note that different methods have different criteria to compute the score, like the number of overlaps and/or semantic similarity. The formal definition is as follows.

Definition 1 (Top-k Table Join Search). Given $\mathcal{T}$, a query table T_q^J, the specific column C_q^J and a parameter k, top-k table join search aims to retrieve a subset $\mathcal{T}_q \subset \mathcal{T}$, $|\mathcal{T}_q| = k$ such that $\forall T \in \mathcal{T}_q$ and $\forall T' \in \mathcal{T} \setminus \mathcal{T}_q, R(C_q^J, T) > R(C_q^J, T')$.

Remark. Given a target table T_t, there may exist multiple columns that can be joined with T_q. In this case, we take the one with the highest score as the target column, *i.e.*, $C_t^J = \arg\max_{C \in T_t} R(C_q^J, C)$. Also, we focus on the two-way join rather than multi-way joins [15].

Table Union Search. Given a query table T_q^U, table union search aims at finding the top-k unionable tables from the data lake $\mathcal{T}$. At a high level, similar to table join search, table union search highly relies on the unionbility of the columns, *i.e.*, a pair of unionable tables should have multiple pairs of columns that can be unioned. Like table join search, column unionbility also takes the overlaps and/or semantics between columns into consideration. To be specific, given a target table T_t, we can measure the relevance of each pair of columns, *i.e.*, $R(C, C'), C \in T_q^U, C' \in T_t$, and compute a table-level relevance score as $R(T_q^U, T_t)$.

Definition 2 (Top-k Table Union Search). Given $\mathcal{T}$ and a query table T_q^U, top-k table union search aims to retrieve a subset $\mathcal{T}_q \subset \mathcal{T}$, $|\mathcal{T}_q| = k$ such that $\forall T \in \mathcal{T}_q$ and $\forall T' \in \mathcal{T} \setminus \mathcal{T}_q, R(T_q^U, T) > R(T_q^U, T')$.

More specifically, given a query table with a user-specified column, table join search finds the target tables that can be joined with the query table on the user-specified column.

Example 1 (Table join search.). *As shown in Figure 1(a), given a query table (T_1) and a specified column* `Corporation`*, Table T_2 in the data lake is joinable with T_1, i.e., the first attribute of T_2 can be joined with the specified column because i) the column names match and ii) a number of cell values have fuzzy overlaps (e.g., Apple and Apple Inc.). Joining these two tables together produces a table with richer features, which is beneficial when used to train a machine learning model to for example predict the stock* `Price` *of the companies in the table. On the other hand, although in T_3, the column name and the contents of its first attribute are semantically similar to the specified column, it is not joinable because there is no overlapping value.*

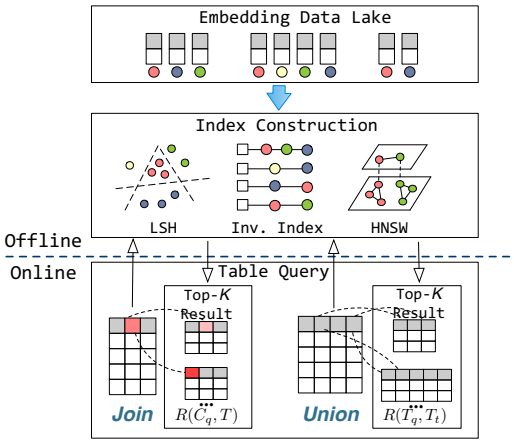


Figure 2: Table Discovery in Data Lake.

For table union search, given a query table, it aims to find target tables that are unionable with the query.

Example 2 (Table union search.). *As shown in Figure 1(b), given a query table (T_4), T_5 can be unioned with T_4 because they are all about the information of movies and two pairs of attributes are highly correlated (T_4.*`Name` *v.s. T_5.*`Movie` *and T_4.*`Rating` *v.s. T_5.*`Score`*). However, when it comes to T_6, although T_6 has three attributes (T_6.*`City`*, T_6.*`Date` *and T_6.*`Rating`*) aligned respectively with attributes (T_4.*`Venue`*, T_4.*`Date` *and T_4.*`Rating`*) of T_4, T_6 is not unionable with T_4. This is because the two tables are not in the same context: T_4 is about movies, while T_6 is about restaurants.*

Examples 1 & 2 show that effectively discovering joinable and unionable tables faces multiple common challenges, including (1) the (semantic) schema similarity of columns, (2) the overlap between columns, and (3) the contextual information across multiple columns in a table. Moreover, as a data lake is typically large-scale, designing efficient and scalable discovery algorithms is of high complexity. Although many approaches have been proposed to address the above problem from different perspectives, these approaches are yet to be thoroughly evaluated due to the lack of a comprehensive benchmark.

Remark. Note we return the top-k results rather than asking users to set a cut-off threshold w.r.t. the relevance score and returning all tables that have a relevance score higher than this threshold. This is because setting a threshold appropriate to the given query is harder than setting the k parameter. Therefore, almost all [3, 14, 17] existing works focus on retrieving the top-k results.

2.2 Table Discovery Process

Most of the table discovery methods consist of the following three modules: data lake embedding, index construction, and online table

Rottnest: Indexing Data Lakes for Search

Ziheng Wang*+ Stanford University
Sasha Krassovsky* Anthropic PBC
Conor Kennedy*, Alex Aiken Stanford University
Weston Pace LanceDB, Inc.
Rain Jiang*, Huayi Zhang, Chenyu Jiang, Wei Xu Bytedance, Inc.

***Abstract*—Data lakes have become widely popular in managing enterprise data. Their widespread integration with query engines has allowed them to displace specialized data warehouses as the single source of truth for enterprise data. While the columnar storage format and block min-max indices allow query engines to achieve competitive performance on relational data analytics queries, they are not yet suitable for other search-oriented queries like full text and vector nearest neighbor search. We present Rottnest, a general system that builds additional lightweight indices on top of data lakes. We show that our system is more cost efficient compared to un-indexed data lakes or specialized databases across several orders of magnitude of total query loads and operating time horizons.**

I. Introduction

Data lakes have become mainstream in analytical data processing. They typically consist of Parquet files on object storage such as AWS S3 organized by a table format like Delta Lake, Iceberg or Hudi [1]–[5]. The Parquet files are queried on-demand with engines such as Trino, Databricks, Snowflake, or DuckDB [6]–[10]. For OLAP SQL workloads, the performance of such query engines is often better than specialized data warehouses [11], [12]. The widespread support of Parquet among data science and machine learning tools have made data lakes ideal for the analytical "single source of truth" in modern enterprise data stacks.

However, data lakes are still unsuitable for emerging workloads that are **search-oriented**. These workloads, such as high-cardinality time series, text, and embedding analytics typically need to quickly drill down to a very small subset of the data and perform complex aggregations on this subset. In most cases, the minimal indexing available in current Parquet-based data lakes do not allow efficient evaluation of the filter conditions (e.g. UUID match, text regex, approximate nearest neighbors), making these workloads inefficient for query engines like SparkSQL or Trino [6], [13]. As a result, the lakehouse paradigm breaks for these workloads – organizations have to duplicate their data to a specialized data management system like Clickhouse, ElasticSearch or Qdrant to perform efficient search analytics, as shown in Figure 1.

Modern data lakes store data primarily as immutable Parquet files in object storage, optimized for large-scale analytics. Data lakes like Delta Lake and Apache Iceberg added transactional support to data lakes by maintaining lightweight metadata files alongside the Parquet files [4], [5], [14]. These systems achieve ACID properties efficiently through a combination of metadata logging and background compaction operations - transaction logs track file-level changes while periodic compactions merge small files to optimize read performance. Building on these principles, we explore if we could support novel search workloads through additional lightweight index files on top of transactional data lakes. We build Rottnest, a system that augments data lakes with external indices resident on object storage for workloads such as high cardinality analytics, full text search, and vector embedding search.

Although individual components of Rottnest's novel architecture such as inverted indices and LSM-style compaction are well studied, our key insight is that combining them enables efficient search over data lakes with low query cost and storage overhead. Rottnest's architecture is enabled by a novel consistent-on-demand indexing protocol that can be bolted on to any data lake and object storage backend. We also propose a novel evaluation methodology that demonstrates Rottnest's effectiveness across a wide range of operational scenarios.

Just as data lakes provide "good enough" transactional semantics for offline big data workloads without replacing OLTP databases, Rottnest provides "good enough" search capabilities for offline workloads without aiming to replace specialized search systems in latency-sensitive online serving. We believe it can be the most compelling option for most offline workloads such as historical log analytics and LLM pretraining data exploration.

In summary, we make three contributions:

1) A novel bolt-on, consistent-on-demand protocol for concurrent index maintenance. Our protocol, with formal correctness proofs, enables consistent indexing even as the underlying data lake undergoes operations like compaction and deletion. Unlike existing alternatives, it only relies on strong read-after-write consistency, making it compatible with all major cloud providers.
2) A new system architecture to support search-oriented workloads over data lakes using our lazy indexing protocol that combines in-situ querying of Parquet files directly and object-store native indices to deliver low query cost and low storage overhead.
3) A novel evaluation framework using physics-inspired *phase diagrams* to analyze total cost of ownership (TCO) across different query loads and time horizons. This methodology reveals insights about system trade-offs that would be difficult to obtain through traditional performance evaluations, and can be applied to evaluate other cloud data systems where TCO is the relevant metric.

* Equal contribution. + Currently at Anthropic PBC.

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00139

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00139

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:38:49 UTC from IEEE Xplore. Restrictions apply.

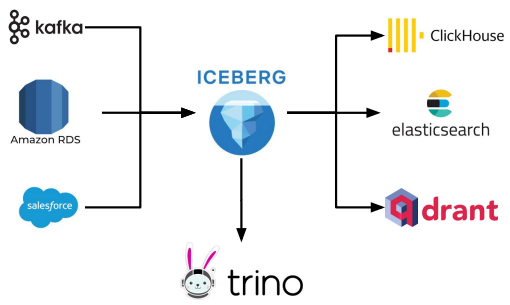


Fig. 1. Typical enterprise data stack. For workloads where directly querying the data lake with a query engine like Trino is too inefficient, a specialized system like Clickhouse, ElasticSearch or Qdrant is used.

II. Motivations

In this section, we describe the access characteristics of Parquet-based data lakes to explain why search-based workloads can be challenging. We also describe existing approaches commonly taken to address those challenges.

A. Parquet Data Lakes on Object Storage

Organizations have started using object-storage-based data lakes based on Parquet files as the single source of truth for enterprise data, due to their wide ecosystem integrations, compelling cost characteristics and transactional semantics. As depicted in Figure 1, operational data is first ETLed into the data lake from OLTP databases, CRMs, and custom connectors. Once the data lands in the data lake, a query engine such as Trino or SparkSQL is used to query the data lake directly for analytical workloads.

Parquet files are well optimized for OLAP workloads against object storage. A Parquet file can consist of multiple row groups, where each row group in turn consists of a collection of column chunks. A column chunk in a row group consists of all the values of a column in the row range corresponding to the row group. Min-max statistics on the column chunks assist query engines in performing predicate pushdown, reading only column chunks necessary for a particular query. These column chunks, which range from several to hundreds of MB in size, can be downloaded in parallel via byte-range requests against object storage. With VM instance bandwidth reaching 100Gbps in major public clouds, analytics on Parquet files residing in object storage has reached interactive speeds.

B. Search Workloads

While data lakes are designed for SQL OLAP use cases, they still fall short in emerging search-oriented workloads, such as:

- **High-cardinality filtering**, such as filtering on a UUID column. Examples include observability workloads (filtering by Kubernetes pod name) and blockchain analytics (filtering by transaction hash).
- **Substring search**, such as log search or text analytics. For example, to detect if evaluation datasets are leaked in the pretraining corpus, one could perform a substring search against the training records, which might be stored as a text column in a data lake.
- **Vector embedding search**, i.e. approximate nearest neighbor search. Examples include retrieval-augmented-generation (RAG) and recommendation systems.

While query engines can efficiently use Parquet-based data lakes for OLAP workloads, they struggle to do so on these new workloads. There are two main issues:

1) **Useless indices.** In search workloads, it is often infeasible to keep the data sorted based on a high-cardinality column and maintain insertion performance. For example, time series data such as IT monitoring and blockchain logs are naturally sorted by timestamp, and sorting them based on UUID tags amount to an extremely expensive transpose. Natural sort-orders do not exist for text or vector embeddings. These two factors render min-max indices on column chunks useless for search workloads.
2) **Read granularity.** Even if an appropriate index exists, column chunks corresponding to text or binary data types typically tend to be tens to hundreds of MBs in size due to Parquet's design (further explored in Section V-A). This means for highly selective search queries, current query engines have to retrieve many MBs of data from object storage just to return a result that is tens of bytes.

"Lakehouse" query engines today like Spark, Trino, or DuckDB today typically have to perform expensive full-column scans which read GBs to TBs of data to perform search queries, making them very expensive.

C. Existing Approaches

Today practitioners commonly employ two approaches to tackle such search workloads, copy data (ETL) and brute force scan. We also mention an emerging third approach, which is new data formats.

1) Copy Data: Practitioners today commonly copy data from the data lake back into specialized systems tailor-made for their workload, such as Clickhouse [15], ElasticSearch [16], vector databases like Qdrant [17] (Figure 1). While this approach guarantees the best performance for those queries by leveraging specialized systems, it largely negates the benefits of the data lake for the data in question. These systems tend to be compute-storage integrated and are expensive to manage. In addition, the data pipelines duplicate data and reintroduce data consistency and staleness issues that data lakes seek to address in the first place.

While we believe that this is the best approach if the search queries are frequent or require strict latency SLAs, e.g. a search engine like Google, we observe that many important workloads do not have such requirements. For example, an LLM pretraining data exploration system might serve only up to 100 internal users while query latencies up to a minute are acceptable. Similarly, a log management system might serve only a particular SRE team where query latencies up to a few seconds are acceptable.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:38:49 UTC from IEEE Xplore. Restrictions apply.

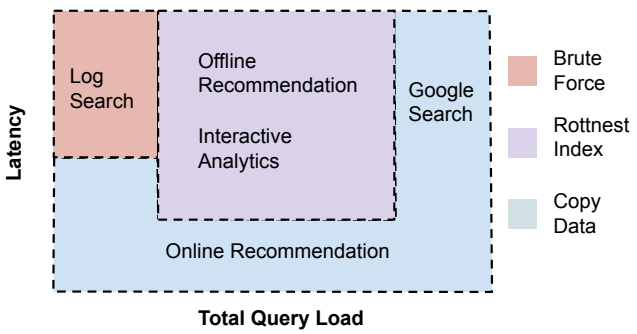


Fig. 2. The most economical approach given latency and throughput requirements of the application. Some example applications are shown.

2) Brute Force Scan: Another popular approach is to simply pay the high IO and compute cost of these queries by horizontally scaling the query engine, such as with Spark SQL [13], [18]. Cloud vendors are especially incentivized to take this approach as they typically charge based on the amount of data scanned. For example, AWS Athena has been heavily used by cybersecurity professionals to query logs, with a pricing model that reflects the brute-force scanning approach.

This approach is the simplest and most economical if querying is highly infrequent, since it incurs no upfront indexing or storage costs. However, it simply defers those costs to query time. While we believe this may be the best approach for workloads that might not ever read most of the data (e.g. log analytics), if most data will be read at least a couple of times over its lifetime, this quickly becomes expensive. In addition, this approaches incurs a minimum latency of typically a few hundred milliseconds due to its need to spin up tasks on demand and access object storage, making it unsuitable for worklaods with strict latency SLAs.

3) New File Format: A third approach is to replace Parquet files with new file formats, such as Lance and Nimble [19], [20]. While these new file formats are purposefully designed to excel at these new search-style workloads, they have not yet achieved the widespread ecosystem integrations that made Parquet appealing in the first place. For example, all three major data lake formats only support Parquet today [2], [3], [5]. As a result, it is impractical today for most organizations to adopt them as the ground truth data store. This means in practice organizations still have to keep data in Parquet, making this no different than the copy-data approach.

Another similar approach is recent modifications to the Parquet file itself, for example column indices and bloom filters [21], [22]. While these approaches could be somewhat effective in some search queries that involve high cardinality tags, they do not help for text or vector searches. More importantly, they assume a Parquet writer that generates these additional indexing structures, which in practice is quite rare.

D. Motivation for a New Approach

The limitations of existing approaches suggest the need for a new design that simultaneously satisfies:

- **Lower indexing and storage cost** compared to copying data into dedicated indices that require expensive persistent RAM/disk resources.
- **Cheaper querying** compared to the brute force approach.

Satisfying these two objectives would provide the most economical solution for a large class of offline or human-interactive workloads with medium total query load, some examples of which are shown in Figure 2. Our objective of easy integration with existing data lakes to reduce data silos leads us to two more design goals:

- **Non-interference guarantee** with existing data lake operations. As the data lake lives at the center of an organization's data architecture, the system must not impact the performance or complexity of existing data lake ingestion, maintenance, or read workloads.
- **Broad compatibility** with all major cloud providers, which might offer varying consistency properties for their object storage offerings.

III. Rottnest Search Indices

Rottnest addresses these four objectives by ***maintaining lightweight index files on top of data lakes.*** Inspired by how data lakes like Delta Lake or Iceberg achieve ACID properties on top of Parquet files in object storage by maintaining log files [3], [4], [23], we propose a novel lazy indexing protocol that constructs secondary indices on data lakes which satisfies the latter two design goals.

Instead of updating indices upon write, Rottnest maintains consistency with the underlying data lake on-demand. Rottnest index maintenance and querying can be performed completely independently of other data lake operations. This approach shares similarities with Postgres concurrent indices and background compactions in LSM-based databases like RocksDB [24], [25]. Like RocksDB, this means the newest data often remains unindexed and requires full scanning during queries. While our protocol is conceptually similar to Azure Hyperspace [26], a key difference lies in the consistency primitives used. Hyperspace relies on atomic rename operations, which are not universally supported across major cloud providers like AWS S3. In contrast, Rottnest's protocol requires only strong read-after-write consistency, making it compatible with all major cloud object stores.

Building on our novel lazy indexing protocol, Rottnest's overall search architecture satisfies our first two design goals: minimizing both index construction/storage costs and query costs. Optimizing along these two objectives allows Rottnest to maximize the area of the purple-shaded area of Figure 2 where it wins over the brute force and copy data approaches. In the red-shaded area, brute force scanning requires no indexing cost but incurs high search costs - lower index costs help Rottnest reach its break-even point with fewer queries. In the blue-shaded area, dedicated systems like ElasticSearch require high upfront costs to store indices in expensive persistent SSDs/memory but offer cheap searches - lower query costs allow Rottnest to remain competitive at higher query loads while also meeting stricter latency requirements.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:38:49 UTC from IEEE Xplore. Restrictions apply.

Table Discovery in Data Lakes: State-of-the-art and Future Directions

Grace Fan
Northeastern University
fan.gr@northeastern.edu

Jin Wang
Megagon Labs
jin@megagon.ai

Yuliang Li
Megagon Labs
yuliang@megagon.ai

Renée J. Miller
Northeastern University
miller@northeastern.edu

ABSTRACT

Data discovery refers to a set of tasks that enable users and downstream applications to explore and gain insights from massive collections of data sources such as data lakes. In this tutorial, we will provide a comprehensive overview of the most recent table discovery techniques developed by the data management community. We will cover table understanding tasks such as domain discovery, table annotation, and table representation learning which help data lake systems capture semantics of tables. We will also cover techniques enabling various query-driven discovery and table exploration tasks, as well as how table discovery can support key data science applications such as machine learning and knowledge base construction. Finally, we will discuss future research directions on developing new table discovery paradigms by combining structured knowledge and dense table representations, as well as improving the efficiency of discovery using state-of-the-art indexing techniques, and more.

CCS CONCEPTS

• **Information systems → Top-k retrieval in databases**; *Mediators and data integration*; *Database design and models.*

KEYWORDS

Dataset Discovery, Data Lake, Data Integration, Unionable Tables

ACM Reference Format:
Grace Fan, Jin Wang, Yuliang Li, and Renée J. Miller. 2023. Table Discovery in Data Lakes: State-of-the-art and Future Directions. In *Companion of the 2023 International Conference on Management of Data (SIGMOD-Companion '23), June 18–23, 2023, Seattle, WA, USA.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3555041.3589409

1 INTRODUCTION

We have witnessed in the last decades a drastic growth in the number of open and shared datasets coming from governments, academic institutes, and private companies. These massive collections of structured or unstructured datasets, which we refer to as *data lakes*, open up new opportunities for innovation, economic growth, and social benefits [39]. As of 2022, the size of US open data (data.gov) has reached a total of 335k datasets contributing to $3 trillion of the US economy [9]. WebDataCommons [30] has also made 233M tables extracted from the open web publicly available. Data lakes store a wide variety of open-domain knowledge from sources such as Wikipedia, news articles, and other online sources, as well as data that is private to an organization or purchased from data brokers. However, while data lakes are invaluable data sources, their large size and complexity can make it difficult for users to find and access the specific data that is required by specific downstream applications. To address this issue, the data management community has been building *data discovery systems* [3, 6, 18, 20, 34, 40, 48], with table discovery from data lakes as the primary application. Data discovery systems help users quickly and easily explore and analyze large datasets, enabling them to gain insights and knowledge that can support decision making and other activities.

This tutorial aims at providing a comprehensive overview of the most recent developments from the database community that enable the aforementioned data discovery methodologies and systems. While there are some previous studies about discovery from other data formats, such as images and JSON [36, 55], we focus on the discovery from tabular data, which is a primary data format in data lakes. We will organize our tutorial by taking an architectural view of table discovery systems depicted in Figure 1. The architecture highlights key system components (in green) and sub-tasks (in blue) that are essential building blocks of an end-to-end data discovery pipeline from raw data lake tables to end-users and downstream applications.

Firstly, we will have an overview of several table understanding tasks. Given an input data lake, table discovery systems typically enrich the raw data lake tables via automatically annotating and indexing techniques to enable better understanding of table semantics. Annotations such as semantic column types can be further used to improve the performance of downstream applications (e.g., table search). We will focus on the tasks of (1) *table annotation* [34] for generating useful meta-data about tables, (2) *domain discovery* [33, 45] for identifying novel domains beyond standard DB data types, (3) machine learning techniques for learning *embeddings* [14, 50] of data items in tables (including columns, rows, and entire tables).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGMOD-Companion '23, June 18–23, 2023, Seattle, WA, USA
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9507-6/23/06…$15.00
https://doi.org/10.1145/3555041.3589409

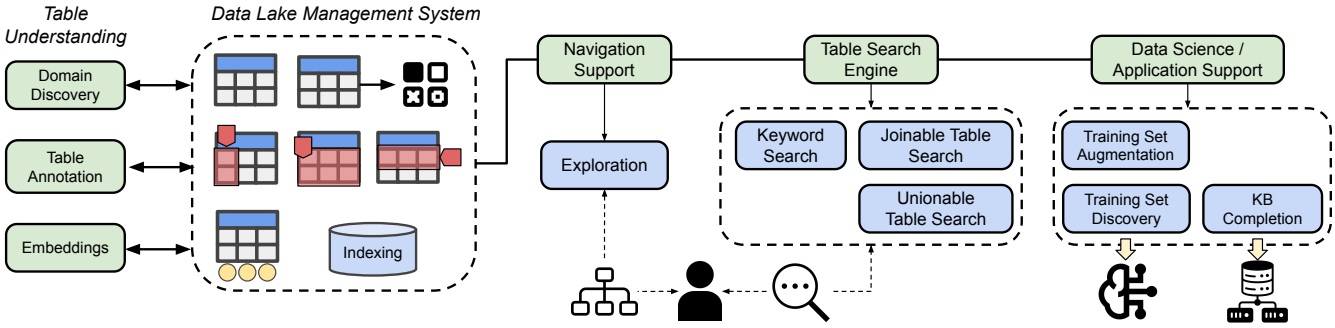


Figure 1: Overview of table discovery in data lakes.

The table discovery system serves the data lake tables to the end-users (e.g., data scientists) by enabling them to query and explore tables that are relevant to their downstream analytic tasks. This is typically achieved via *query driven discovery* [43]: given a user query, retrieve the top-k relevant tables from the data lake. Alternatively, some discovery systems support *data navigation* allowing a user to navigate through the tables in a data lake using a (discovered) organization for the tables [42] or by exploring individual tables followed by tables related through a foreign key or other relationships [18]. And query and navigation can be combined to create organizations over the set of tables retrieved by search [46] in an online manner.

For query driven discovery, we will survey the classic query types includingkeyword search or meta-data search, which often use table schema including attribute names [4, 47, 54] as supported by popular systems such as Google dataset search [3, 21]. Importantly, rather than keywords or table schema, an emerging trend in table discovery systems is to support querying data lakes using table values (e.g., columns or an entire table). Previous studies have identified two types of important search queries: joinable table search [12, 15, 16, 48, 59, 60] and unionable table search [2, 12, 17, 27, 44]. Joinable table search aims at augmenting a query table with additional attributes, while unionable table search is motivated by the need to find tables with new tuples to extend a query table. We will be covering the classic schema-based and value-based approaches, as well as the latest approaches based on knowledge graphs [27] and pre-trained language models [17] that achieve state-of-the-art performance.

We will also cover an emerging trend of performing data discovery to support a variety of data science tasks. One typical example is the creation and augmentation of training datasets. High-quality training datasets are often times a prerequisite for the success of ML models. Data lakes provide a unique opportunity for harvesting training data. Among those techniques, we will primarily introduce (1) data augmentation [8] that aims at finding new features to improve the ML models' performance and (2) training set discovery/construction [58] which focuses on constructing training data from a fixed set of tables via learning representations.

Finally, we will present challenges and future research directions for table discovery. We will cover four main aspects. First, for table understanding which is core to table discovery, we will discuss the possibility of combining two lines of already successful approaches: *knowledge* based approaches [27, 53] and *machine learning* based approaches [14, 17, 23]. We will challenge the community to find synergies between the two approaches by bringing together both rich structured knowledge from KB's and highly effective dense representations from ML. Second, we will further discuss the challenge of boosting the scalability of discovery systems, especially the issues related to indexing. We will elaborate on why indexing is still a challenging unsolved problem for data lakes and how some recently developed techniques on data structures [25], sketching [48], and vector indices [38] can potentially help. Finally, we will discuss the challenge of query-time table annotation and a potential research direction for connecting table discovery with graph mining.

2 TUTORIAL OUTLINE

In this section, we give an overview of the topics that will be covered in the tutorial. They are depicted in Figure 1 as the main components of a data discovery system (in green) and applications (in blue) that are related to table discovery.

2.1 Overview

A popular methodology for table discovery is query-driven discovery [12, 43, 60], where the user starts with a query and aims to find dataset(s) relevant to the query. The query could be either keywords or tables, while the query type could be exact match, joinable, unionable, etc. Existing studies support keyword search over the metadata of tables for users who intend to find tables relevant to a topic (Section 2.3). However, the schemas of tables in data lakes are often unreliable, either missing or incomplete often due to data being shared in primitive formats like CSV [43]. Even when metadata is available, the tables often come from a huge variety of sources meaning the metadata may be inconsistent using different vocabularies and naming conventions. Thus, two important lines of work have emerged. The first we call *Table Understanding*, which are tasks that seek to recover some of the semantics of tables. The second is data-driven table discovery based on a query table (rather than metadata), specifically joinable table search and unionable table search. Yet, data-driven table discovery still has ambiguity in the definitions, which have evolved over time. For example, early work [12] defines related tables as schema-complements and entity-complements based on a defined subject attribute that explains

the table entities. Valentine [28] uses four different notions of joinability and unionability depending on the alignment between the schema of returned tables with that of the query table. Bogatu et al. [2] simultaneously finds joinable and unionable tables by employing five different metrics. Thus, definitions of finding related tables can be very broad, leading to many variations of joinable and unionable table search, which are discussed in Sections 2.4 and 2.5, respectively. Also, there is another aspect of table discovery called navigation [46], which can return a collection of tables in a hierarchical structure that includes not only the contents but also the relationship of tables in the results.

Besides, the outcome of table discovery can benefit many related data science applications. For instance, in the problem of tabular data classification, joinable table search [15] can help extract richer features from the data lakes to enrich the original tables in the training data. And table stitching [29] has been proven rather useful in supporting knowledge base completion.

2.2 Table Understanding

An early influential table discovery system [34] makes use of a variety of table annotations to perform search. These include annotating cells with entities (from an ontology) that the cell mentions, annotating columns with ontology types, and annotating pairs of columns with ontology relationships [34]. The approach uses a supervised probabilistic graphical model to collectively learn all types of annotations. Venetis et al. [51] achieves similar goals by leveraging a proprietary ontology to label columns and binary relationships. More recent work has used labeled training data and supervised learning to label columns with a set of types. These studies start from a set of tables annotated with a fixed set of 78 types and train a machine learning model to apply on unlabeled tables. Sherlock [23] uses a feature-based method to learn column representations for semantic type detection; while Sato [56] makes further improvements by incorporating topic modeling methods (that use row context) for semantic type detection.

Domain discovery is a related problem that aims to identify sets of terms that represent instances of a semantic concept or domain from a given collection of tables. Rather than annotating columns with types, domain discovery collects all values that belong to the same domain. Existing solutions [33, 45] are unsupervised and leverage co-occurrence information to cluster domain terms. In addition Li et al. [33] select a representative as the domain or semantic type.

Recently there is a new trend of utilizing pre-trained language models like BERT to support table annotation applications [14, 50, 52]. The idea is to sequentialize tabular data items by inserting special tokens and use this representation of tables as input to perform fine-tuning over pre-trained language models. This has been done for a variety of annotation applications, including column type detection, relation extraction, cell filling, row population and schema augmentation [14]. Notably, such studies achieve state-of-the-art performance on a variety of benchmarking tasks due to the superior power of language models. We will revisit this observation when we talk about future challenges (Section 3).

2.3 Keyword Search

For users who intend to find tables about a certain topic, some data discovery systems support keyword search to satisfy this need. Keyword search was first used to find tuples within databases [10, 22, 26], but this soon developed into keyword search to find relevant tables, especially in a data lake setting. In this case, a user provides topic keywords and the data discovery system finds relevant tables where different systems have defined relevance by proposing different similarity measurements. The OCTOPUS [4] system supports keyword search over web tables using document keyword search to find pages and then computes the correlation between the keywords and each element (cell) in any table extracted from the document. OCTOPUS returns clusters of tables that share the same schema. In addition, it infers additional attributes by joining tables. In contrast, Pimplikar and Sarawagi [47] use keyword queries describing columns and identify a mapping between the columns of a table and query keywords using table headers, the text surrounding a table on a webpage, and the data within a table. To better facilitate keyword search, Google Dataset Search [3, 21] focuses on integrating and normalizing metadata of tables from various data sources. It works on cleaning, standardizing, and inferring the metadata of each dataset and the relationships among different datasets, while organizing them into a knowledge graph to facilitate retrieval. Keyword search is done only over the metadata, not the data within a table.

2.4 Joinable Table Search

Except for keyword search, many table discovery systems use the data values themselves to find tables that are related to the column values of the given query table. One problem that several approaches aim to solve is joinable table search, with the goal of finding relevant tables to join and augment the query table with additional attributes. InfoGather [54] allows augmentation of a query table with new attributes by first performing schema matching and then either augmentation by attribute names or example tuples, or discovery of related attributes to a given set of entities. Das Sarma et al. [12] defines joinable tables as schema-complements, where schema matching is conducted using attribute names, types, and values to find joinable attributes and non-matching attributes to augment with the query. Lehmberg et al. [31] conducts joinable table search on web tables with an aligned subject attributes by measuring the Jaccard similarity between the data values, metadata, and table contexts.

Although early work tended to use Jaccard similarity to measure the syntactic similarity between attributes, Jaccard has been shown to be bias attributes with smaller cardinality [1]. This is impractical in the case of data lake tables, where there is a large skew in the domain sizes. LSH Ensemble [60] studies the problem of domain search to solve the problem of joinable table search. For domain search, it uses set containment to find attributes with high overlap (containment) in values. In this way, it finds joinable tables by finding domains that highly contain the query domain. By employing a Locality Sensitive Hashing (LSH) index [13] and a data partitioning scheme that optimally minimizes false negatives, LSH ensemble is able to find joinable tables on a large scale even among attributes with skewed cardinality distributions. This line of work continues